© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_16

16. Cloud Power-Ups: Bedrock, Vertex, and Azure OpenAI

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to explore the capabilities and features provided by the major cloud vendors for generative AI (Gen AI) and use them with your Java-based applications.

Introduction

Generative AI is transforming how applications interact with users, enabling them to produce human-like text, images, and other media on demand. As organizations look to integrate these capabilities into their Java-based systems, cloud platforms have become the de facto choice for hosting, managing, and scaling foundation models. This chapter explores how AWS—through its Amazon Bedrock service—empowers developers to build generative AI applications that are secure, enterprise-ready, and deeply integrated into existing AWS infrastructures. The chapter will start by placing Gen AI in the context of today's major cloud providers; then it will dive into the specifics of getting started with Bedrock, from experimentation in the Playground to production-grade deployments with guardrails, retrieval-augmented generation (RAG), and Spring AI integration.

State of Gen AI in the Cloud

In mid-2025, the big three clouds—AWS, Microsoft Azure, and Google Cloud—treat generative AI as a native platform feature. AWS Bedrock front-ends more than 40 foundation models and layers guardrails safety filters across them, Azure OpenAI Service has introduced GPT-4.5 along with a "stored completions" API for end-to-end provenance, and Google's Vertex AI counters with Gemini 2.5 in both Flash and Pro variants. The competitive conversation has shifted from deploying ever-larger models to offering differentiated latency, modality support, and built-in safeguards.

At the developer layer, abstractions are rapidly converging. Frameworks such as Spring AI let teams flip a single property to swap between Bedrock, Azure, or OpenAI endpoints, while each cloud now previews native model-routing services that automatically choose the cheapest or most capable engine for a given prompt. Retrieval-augmented generation is effectively a checkbox: Bedrock Knowledge Bases, Azure AI Search Vector Index, and Vertex AI Agent Engine all bundle vector storage, embeddings, and grounding evaluation so developers can ship RAG pipelines without stitching tools together.

With core capabilities commoditized, the battleground has moved to governance and cost-to-quality optimization. Guardrails, Azure content filters, and Vertex policy tags harden default defenses, yet enterprises are equally hungry for smaller, task-specific models that slash token spend without sacrificing accuracy. Success now hinges less on where a model is hosted and more on how well teams evaluate outputs, enforce policy, and blend specialized models into a single, audit-ready workflow.

Generative AI in AWS: Amazon Bedrock

Amazon provides the Amazon Bedrock service to get started with building Gen AI applications quickly. With enterprise-grade security, privacy, and access to industry leading Foundation Models, AWS makes it easy to onboard and build Gen AI use cases. It makes sense to leverage the foundation models hosted on AWS through Amazon Bedrock if your organization already uses AWS for its applications.

Amazon Bedrock is a fully managed service from Amazon that helps you build Gen AI applications for end-to-end development focusing on security, data privacy, and support for commercial licensing using industry-standard Foundation Models. All this within the same environment of AWS you are already comfortable with. It is serverless, and you pay only for the services on the go. For more details, you can refer to the documentation at **https://aws.amazon.com/bedrock/**.

To experiment and play with Amazon Bedrock, make sure to sign up for a free AWS account at https://signin.aws.amazon.com/signup?request_type=register. Amazon Bedrock is not covered under the free tier at this point of time when I check at https://aws.amazon.com/free/, as shown in Figure 16-1. Always refer to this page to check for any changes when you are trying this.

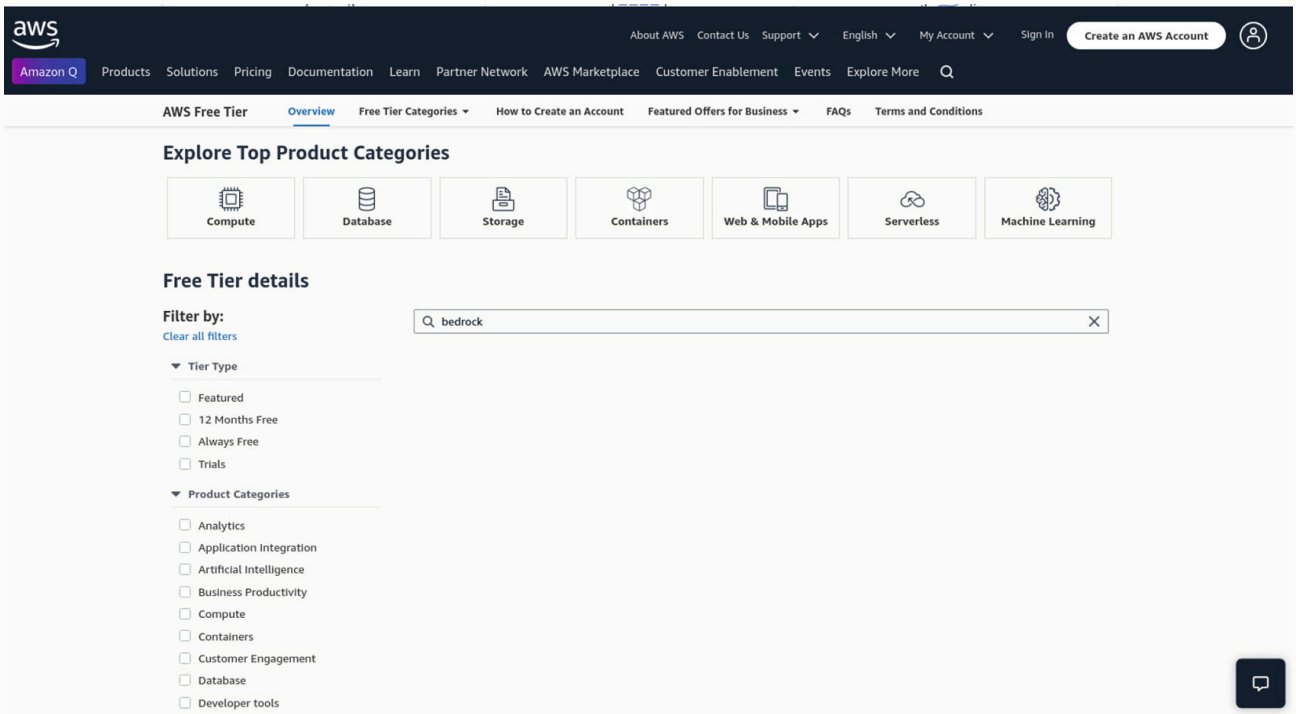


Figure 16-1 Amazon Bedrock unavailability in AWS Free Tier

You can access Bedrock by doing a quick search with the keyword in the top navigation bar, as shown in Figure 16-2.

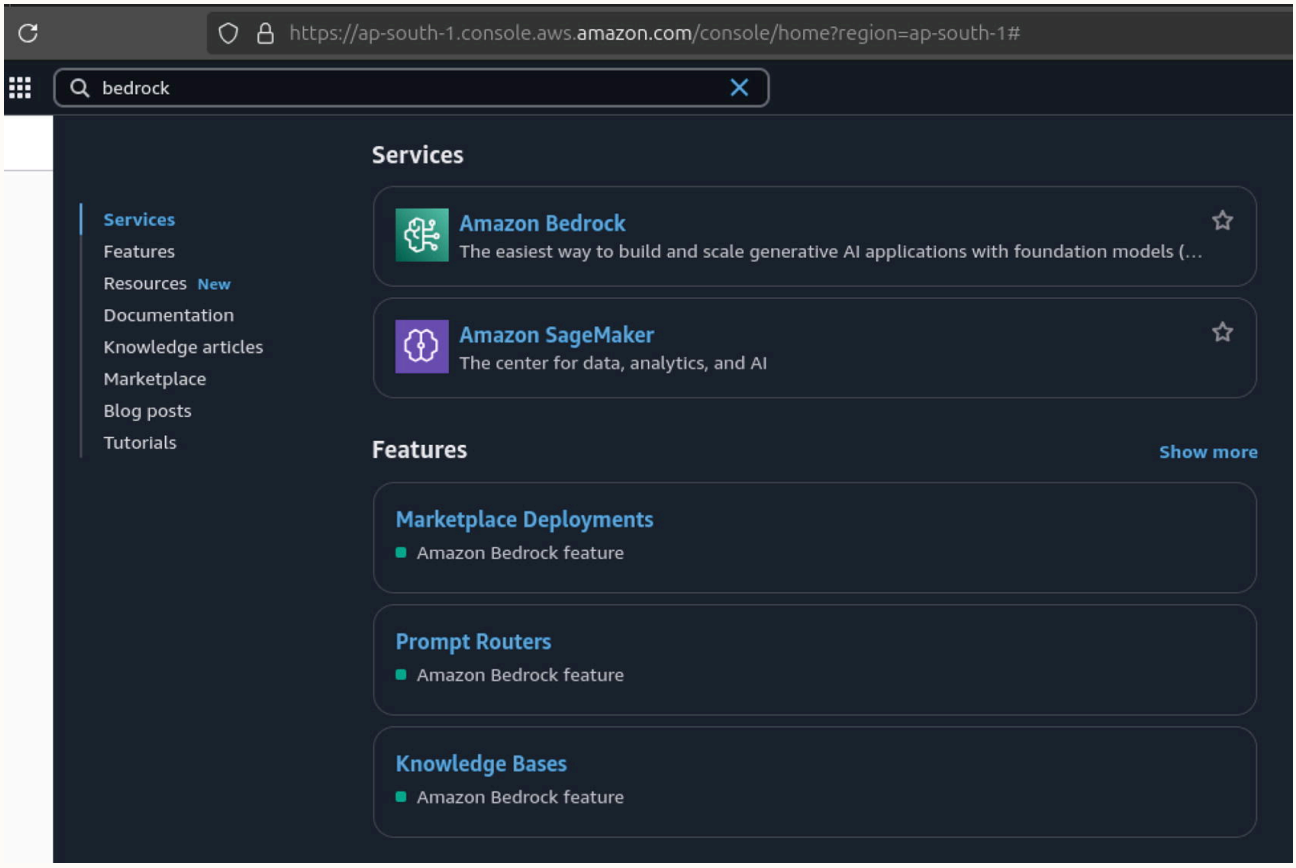


Figure 16-2 Amazon Bedrock access from the navigation search

On accessing Amazon Bedrock from the console, you can see the different features available, as shown in Figure 16-3.

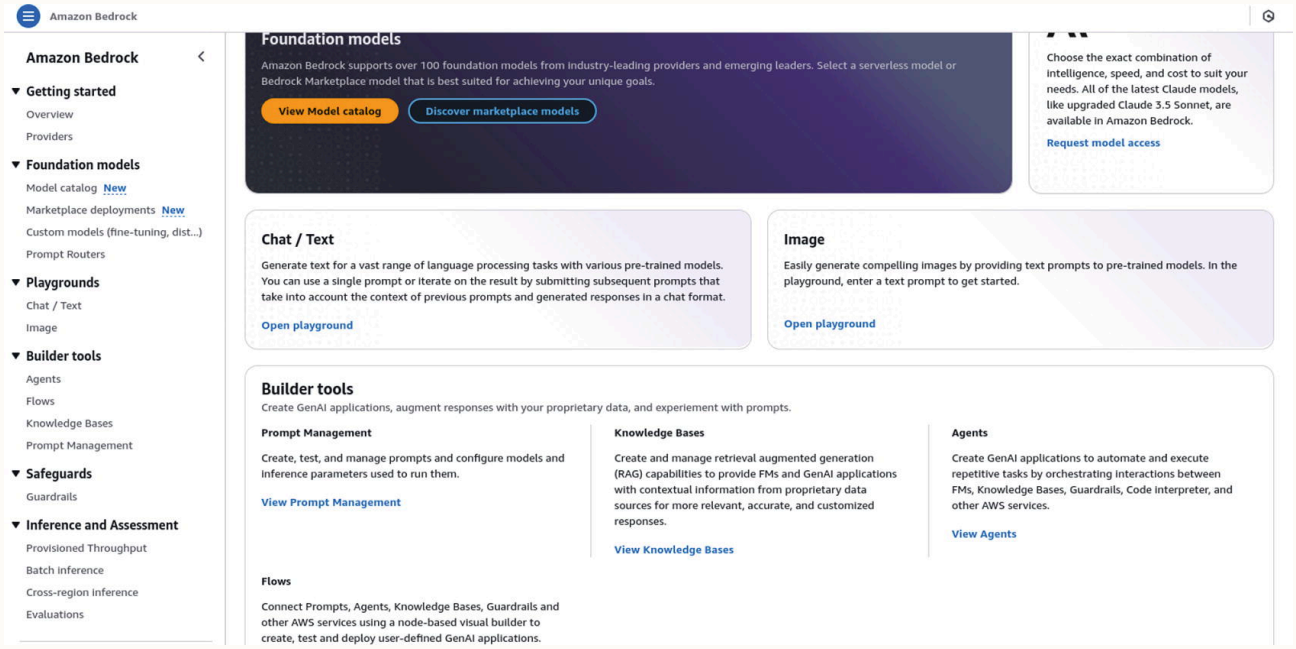


Figure 16-3 Amazon Bedrock home page in AWS Console

As with any cloud provider, the UI and features around Gen AI is always innovating, so the UI may have changed while you are reading this content. In the left navigation menu, you can see some of the features that Bedrock provides at the current time:

- Access to Foundation models
- Playground to try different Foundation models with your prompts in text/image formats.
- Builder tools focusing on prompt management, agentic AI, and knowledge base setup.
- Guardrails to set up security around the LLM ecosystem
- Inference endpoints to host and serve your foundation models and evaluations to test the Model response

I won't go into the theory of all these in detail, but you will learn about them as you try some of them while building your application project. This is your goal as shared by business and your stakeholders:

> *As the industry is evolving, we have decided to extend our domain to the aircraft manufacturing industry to start manufacturing our own aircrafts. To enable our engineers to build certified products specific to FAA/EASA certification guidelines, we want to build an application that will answer any question they have relevant to the part they are engineering to make it certification worthy.*

For this use case project, you will focus on the FAA for the time being, and it can be adapted similarly for EASA too. All documents relevant to the certification process and information is available for public consumption as Code for Federal Regulations (CFRs) at https://www.ecfr.gov/current/title-14/chapter-I/subchapter-C/part-21/subpart-H?toc=1, as shown in Figure 16-4.

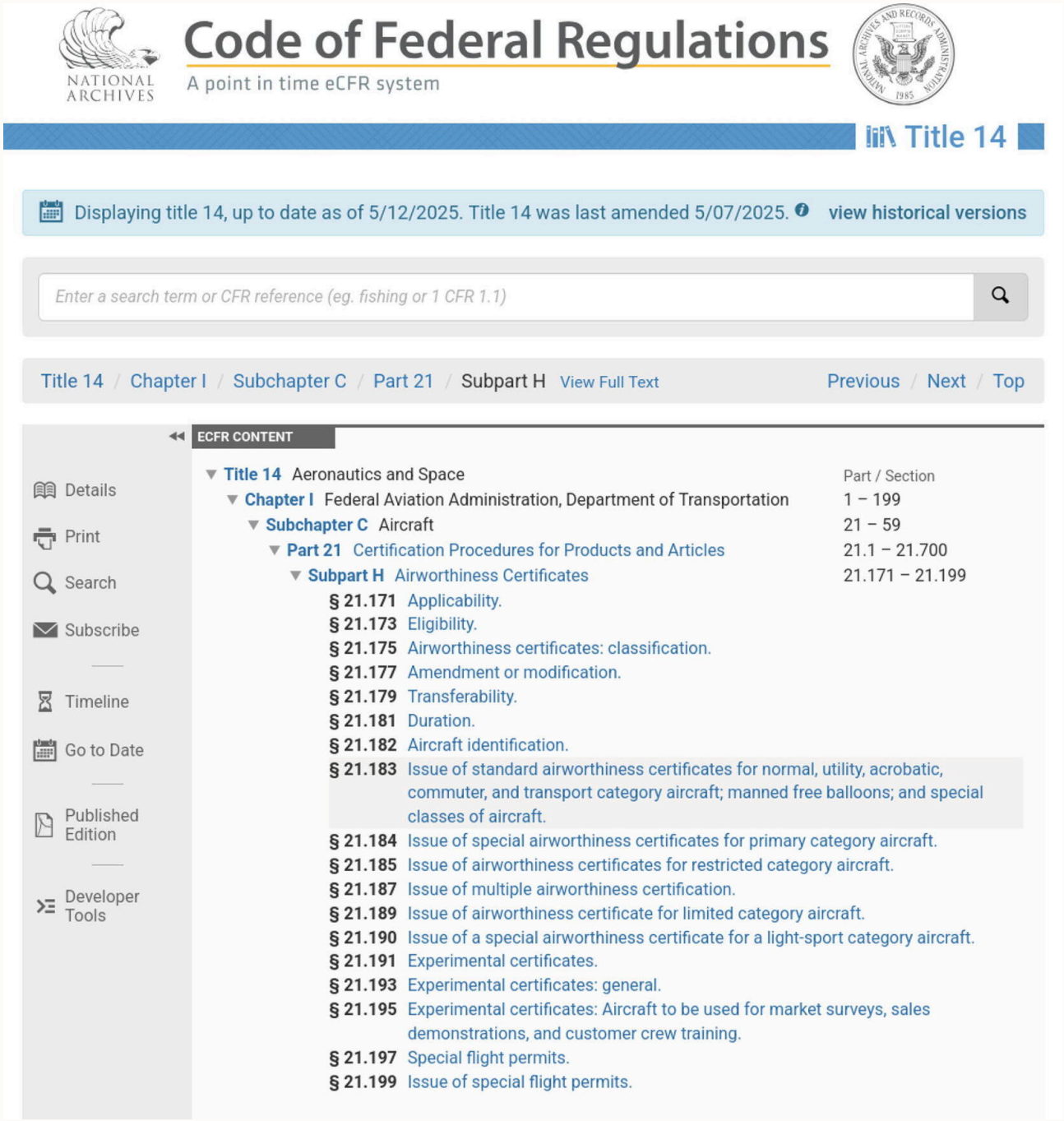


Figure 16-4 FAA CFRs

This documentation contains a variety of technical information and may be challenging to search for if you do not have the exact terms. This is where LLM and RAG play an important role to find the semantic documents and respond to your engineers.

> **Note**The purpose of this application is informational and for learning purposes with no liability or responsibility for using this in any commercial setup.

Accessing a Foundational Model in Bedrock Playground

Let's first explore the foundational models available in Bedrock, as shown in Figure 16-5. As you can see, there are a variety of models available to choose from, including some of the ones produced by Amazon. Always review the license for the model you intend to use. You will need to accept the license agreement and terms of agreement before using

the model. This may not be applicable to all models, though. Also check for the region where you are using the model from, since in some regions all models may not be available.

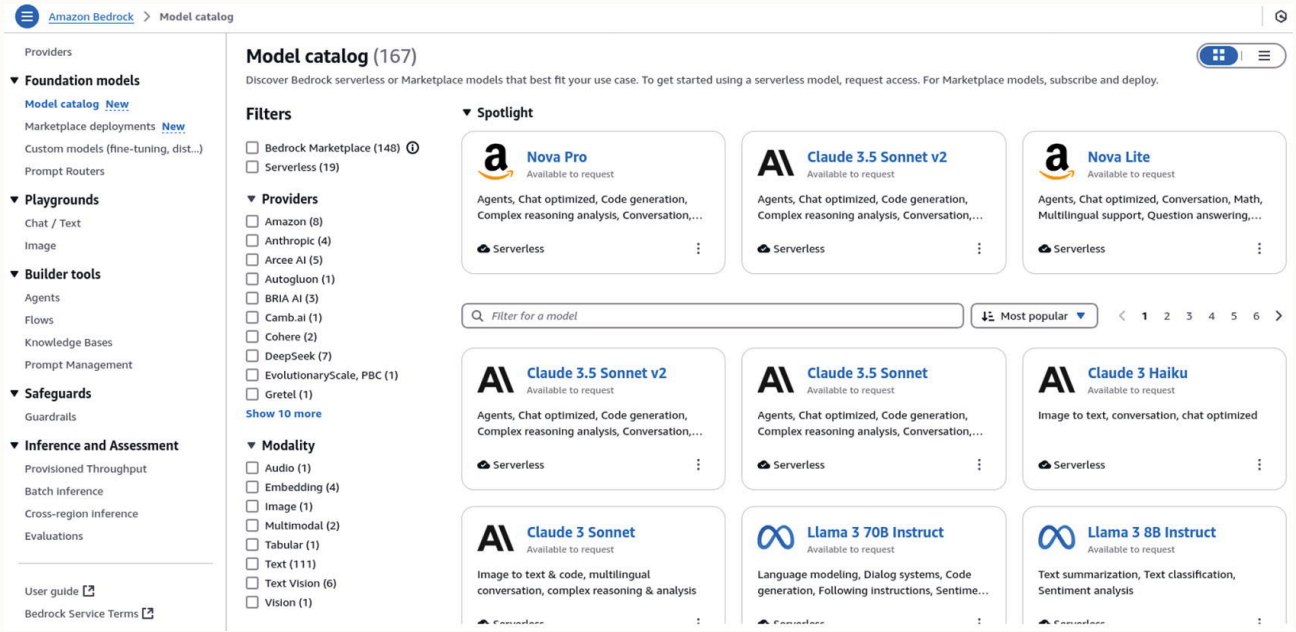


Figure 16-5 Foundation models in Amazon Bedrock

This is an important step to choose your model based on your needs and the cost. Let's explore the AWS Nova model, for instance. Figure 16-6 shows the home page of the AWS Nova model.

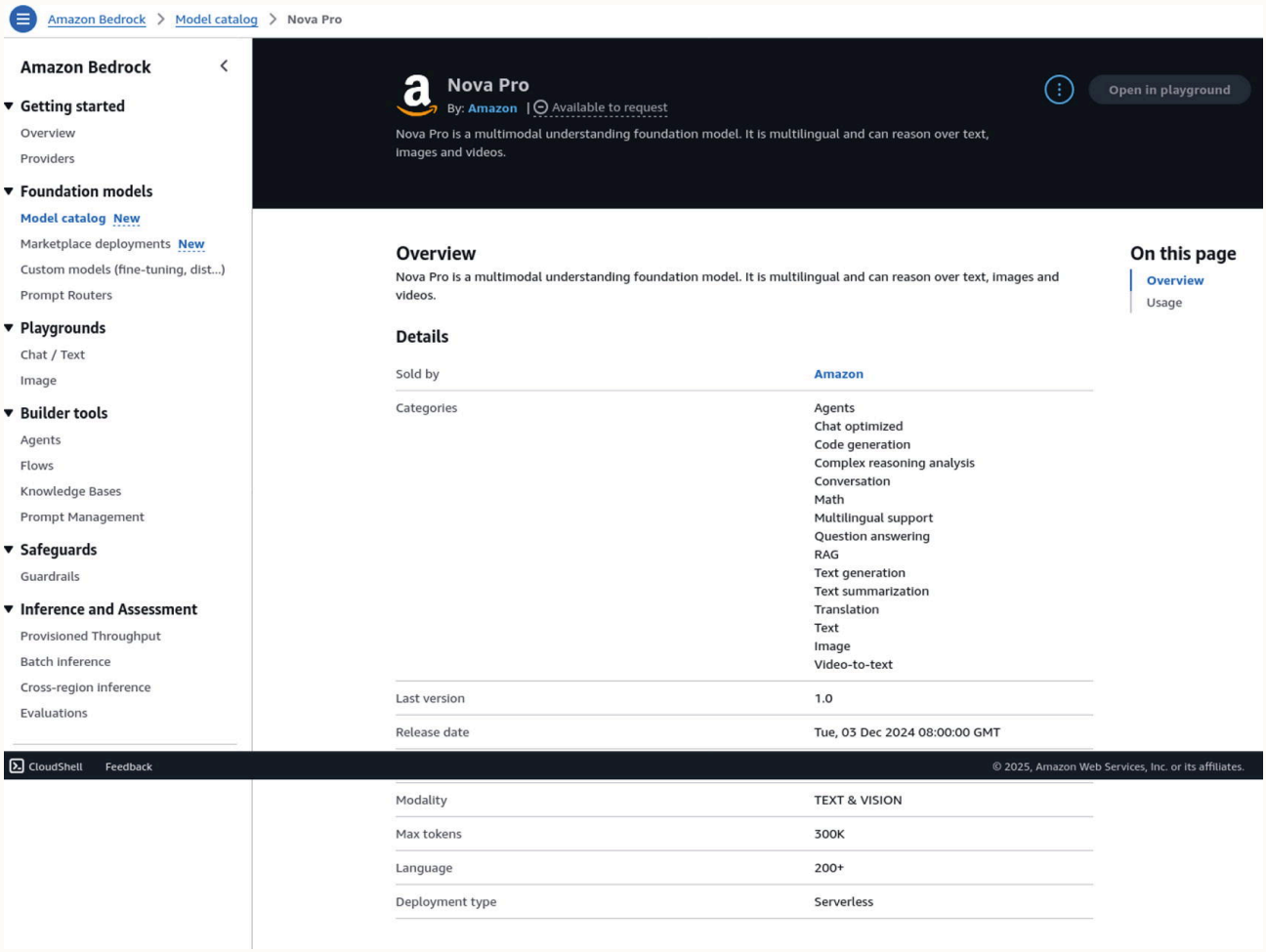


Figure 16-6 AWS Nova model

As shown, at the top it says the model can be requested for use, after which it will be available, and the "Open in playground" option in the right top will be enabled. Currently, I am visiting the page from the ap-south-1 region. The home page contains some important information such as an overview, details as to who has created this model, the categories of tasks the foundation model can help with, its version, the date it was released on, the input modalities it supports, the max number of tokens, the languages supported, and finally the deployment type, which is serverless. This is useful information and can help determine the model to start with. Let's request to use the model and check it out in the playground.

After clicking the "Available to request" link, you will see a pop-up sharing additional information and a link to request model access with a note that billing will start after access grant and for using the model, as shown in Figure 16-7. You will be charged only for the requests you make using the model and not for requesting access to it.

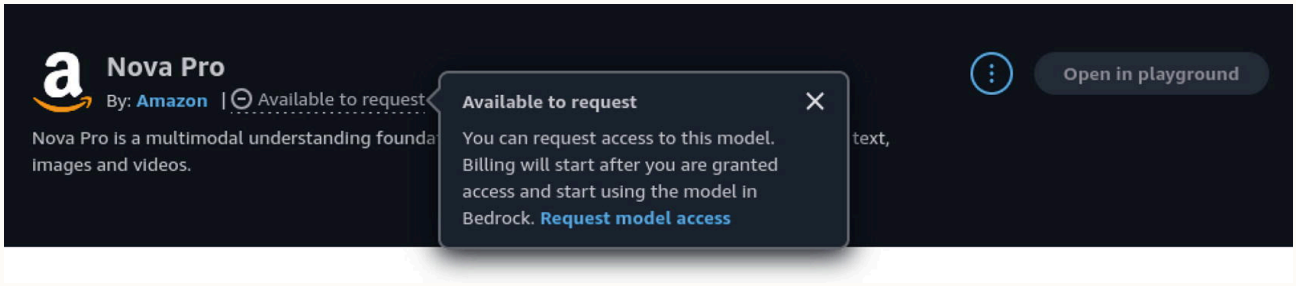


Figure 16-7 Requesting model use

On the model access page, you can click the "Available to request" link of the model of choice or enable all models, as shown in Figure 16-8. My recommendation is to read the EULA for the specific model and enable it on a use-case basis.

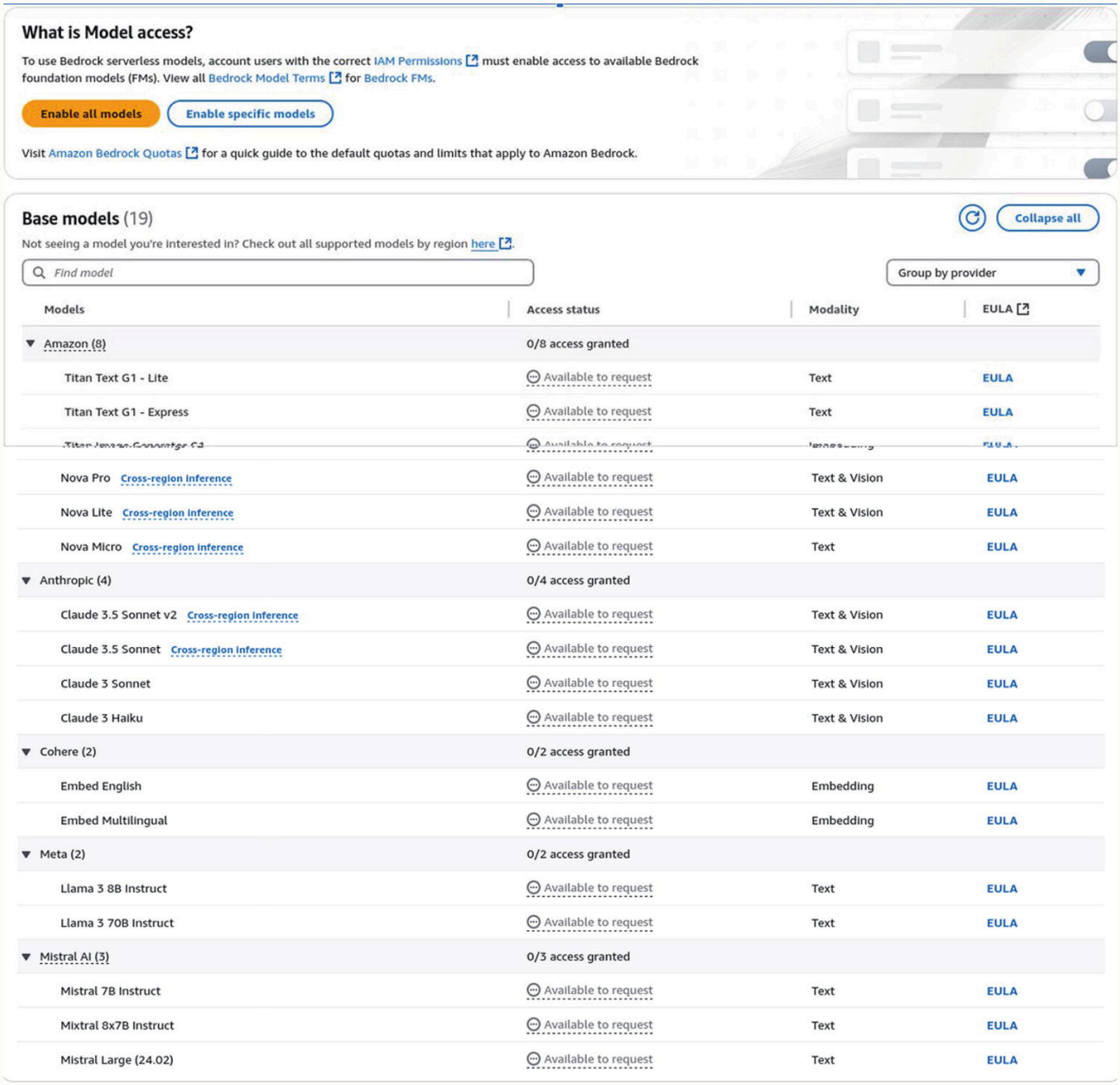


Figure 16-8 Model access

Since your use case is focused and starting with text inputs, you can use Nova Micro, keeping it light and use case specific, as shown in Figure 16-9 and Figure 16-10.

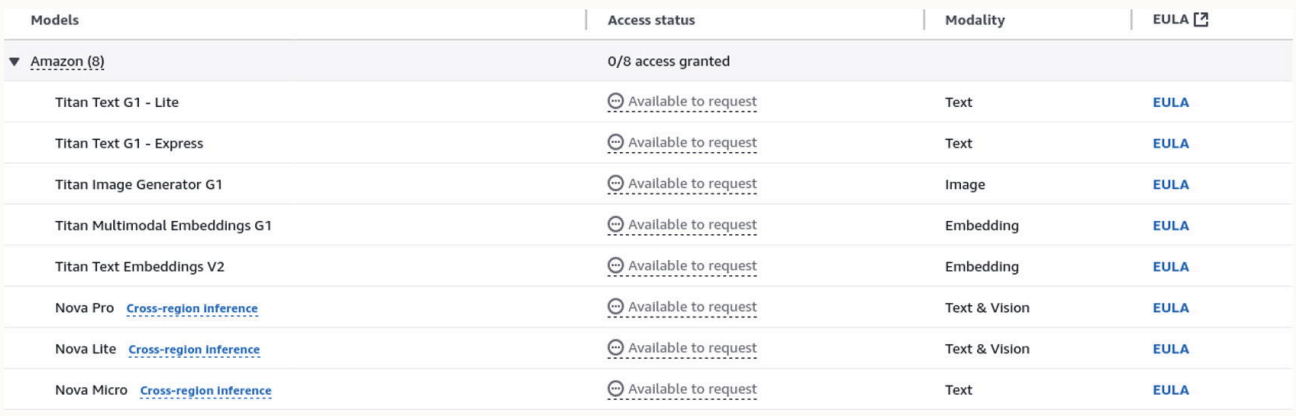


Figure 16-9 Nova-specific models

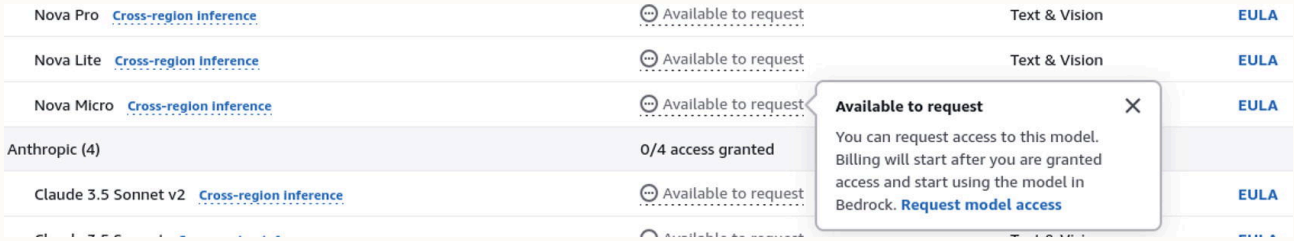


Figure 16-10 Request Nova micro model

The model is selected. Then click the next button, as shown in Figure 16-11.

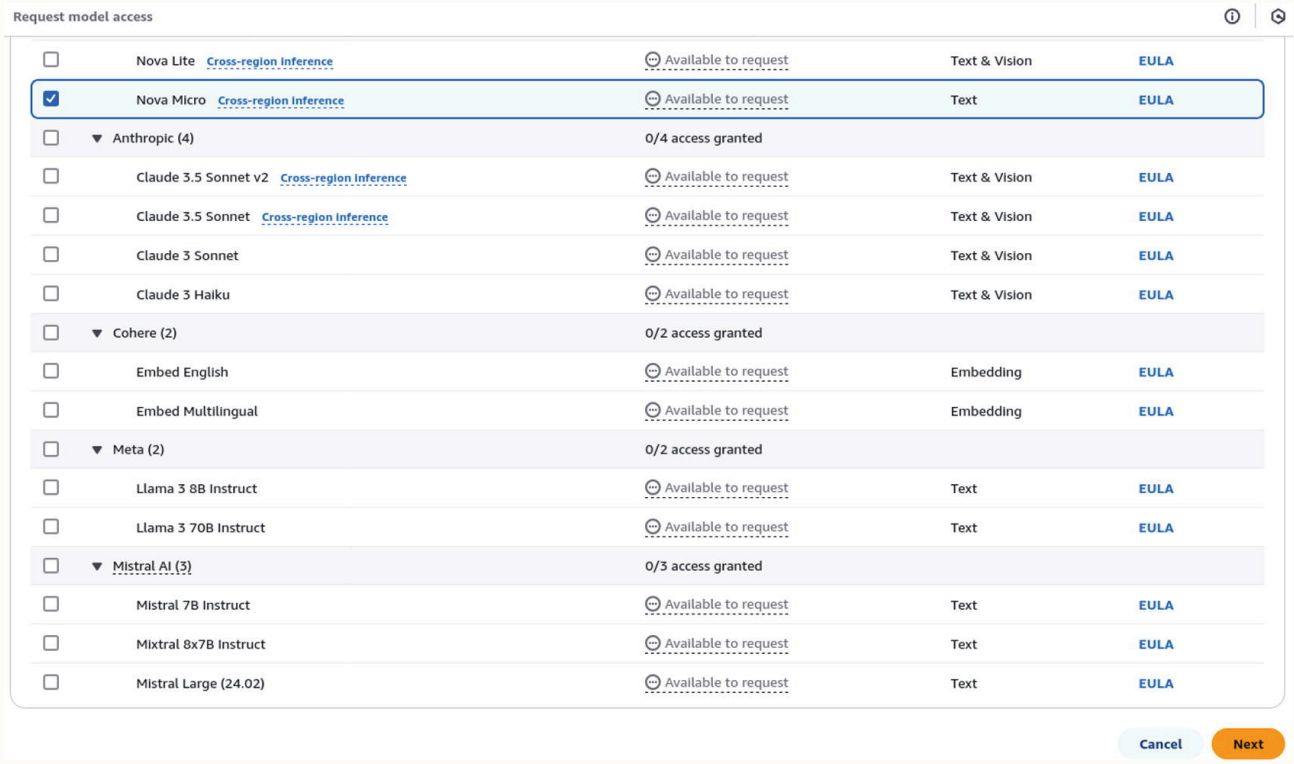


Figure 16-11 Nova model selection

Finally, submit the request for use, as shown in Figure 16-12.

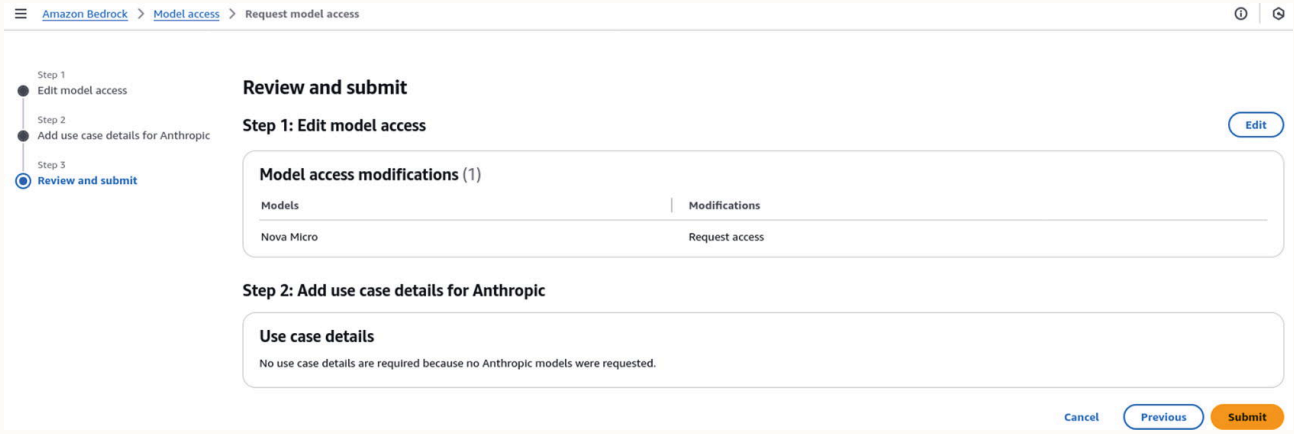


Figure 16-12 Submitting a request for Nova Model use

Once submitted, you will receive a notification saying the time for access grant to the requested model. For some models, it needs to be verified by the end vendor for the use

case you want to use it for. In this case, as you can see in Figure 16-13, you have been granted immediate access to it.

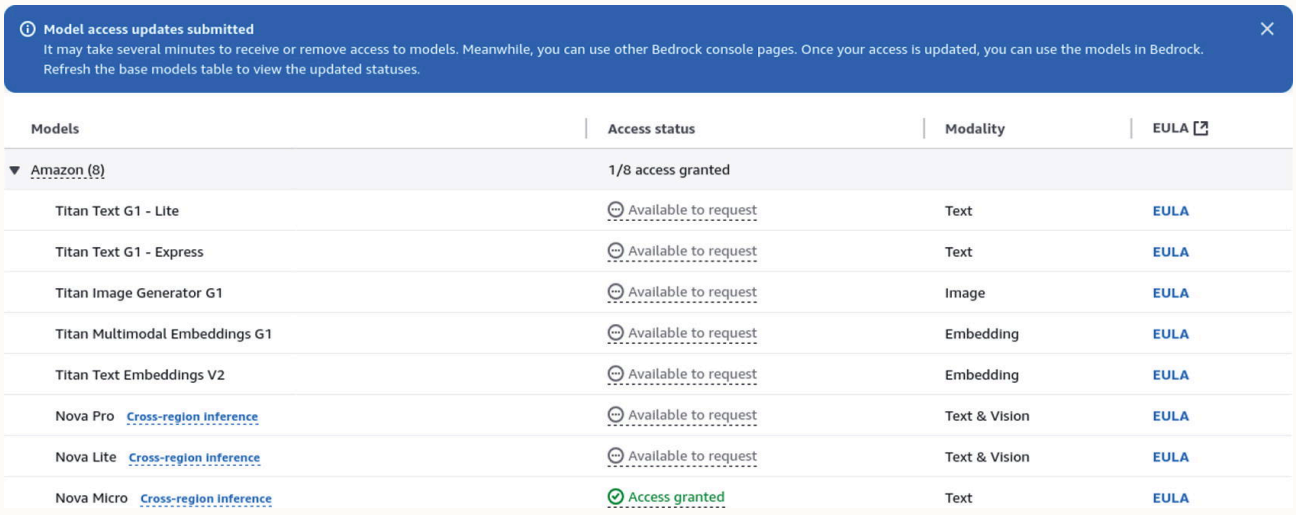


Figure 16-13 Access Granted page

Now this model is available to play with in the playground page. Let's visit the playground page from the left menu, as shown in Figure 16-14.

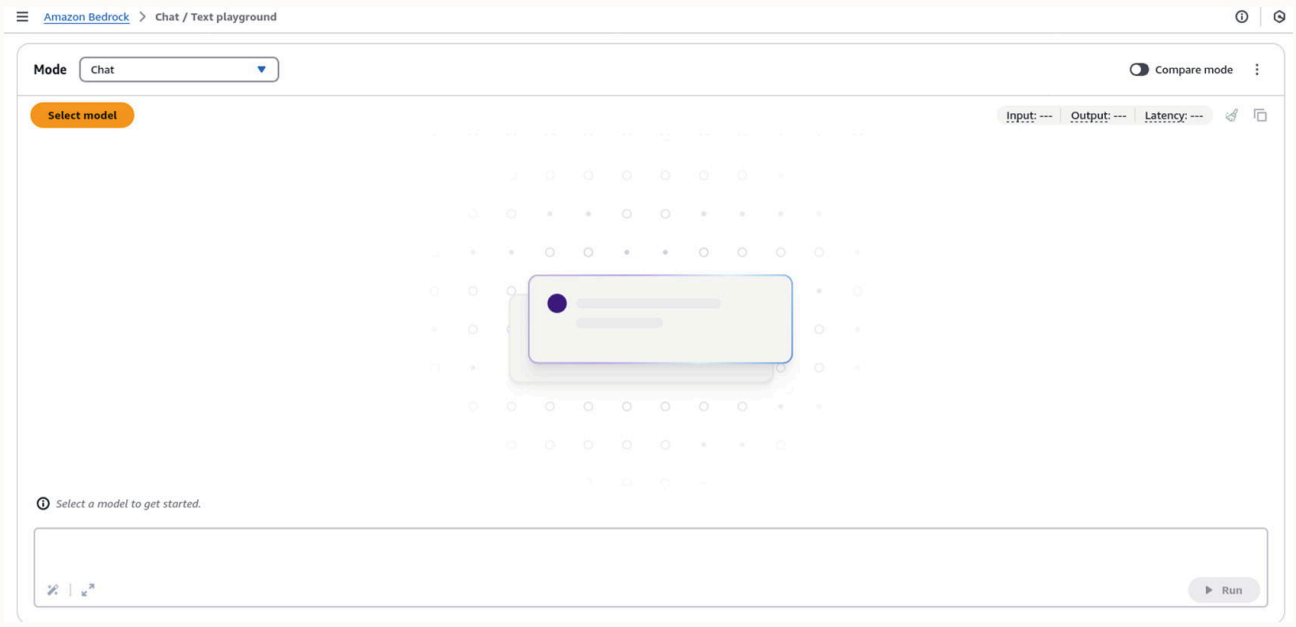


Figure 16-14 Bedrock playground page

There is no model selected yet. Let's click the select model button and select your access-granted Nova Micro Model and specific region you are trying in, as shown in Figure 16-15.

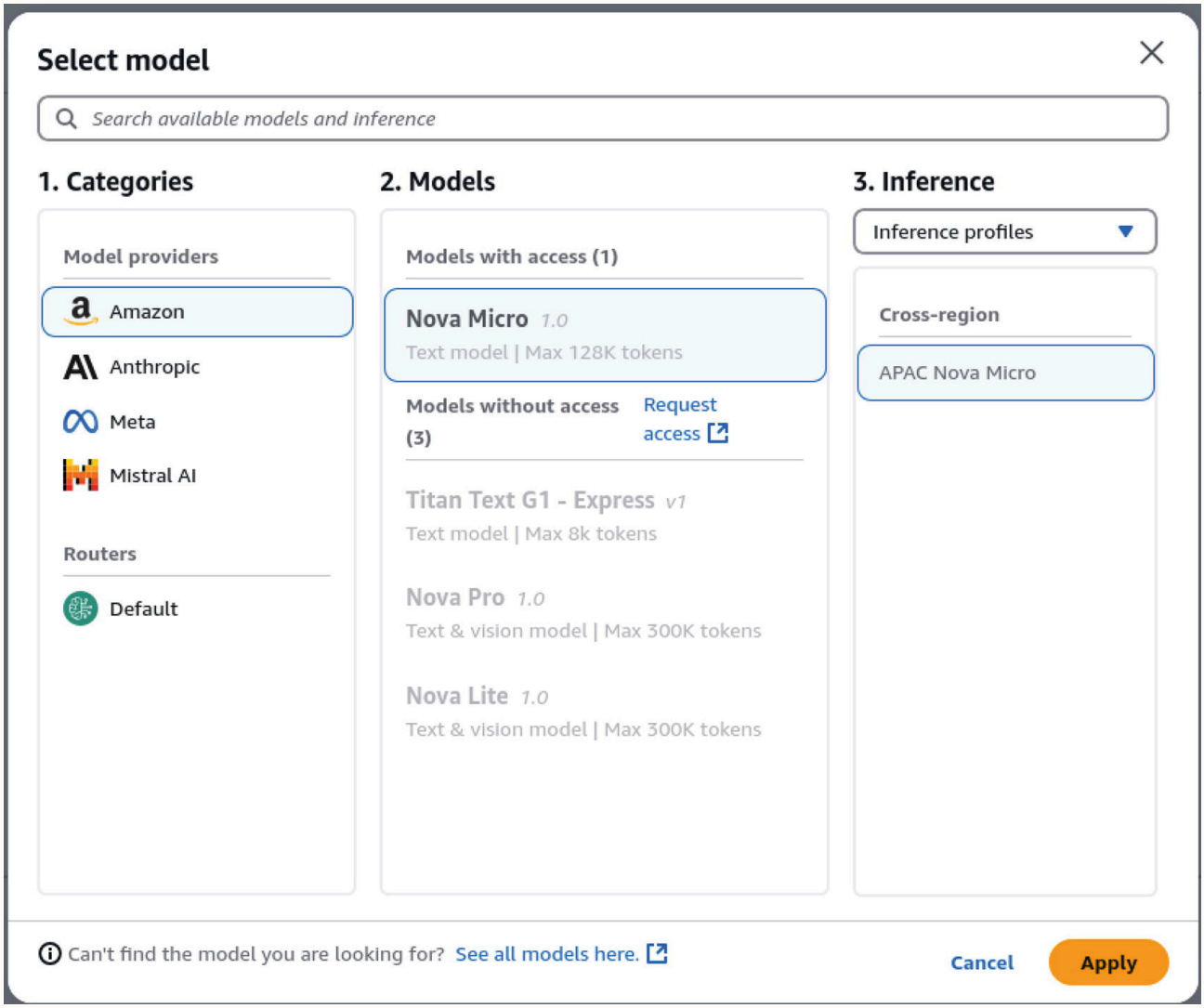


Figure 16-15 Model selection for the playground

Now that your model is selected, the playground is active for trial, as shown in Figure 16-16. Reviewing this, you can see options for system prompts, temperature, Top P, setting the response length, any stop sequence that the model needs to check for to stop producing after it encounters the sequence helpful for early termination of the response, and finally guardrails setup. There is also an option to compare more than one model so as to see the response from multiple models to help you see the output for the same query in the same interface.

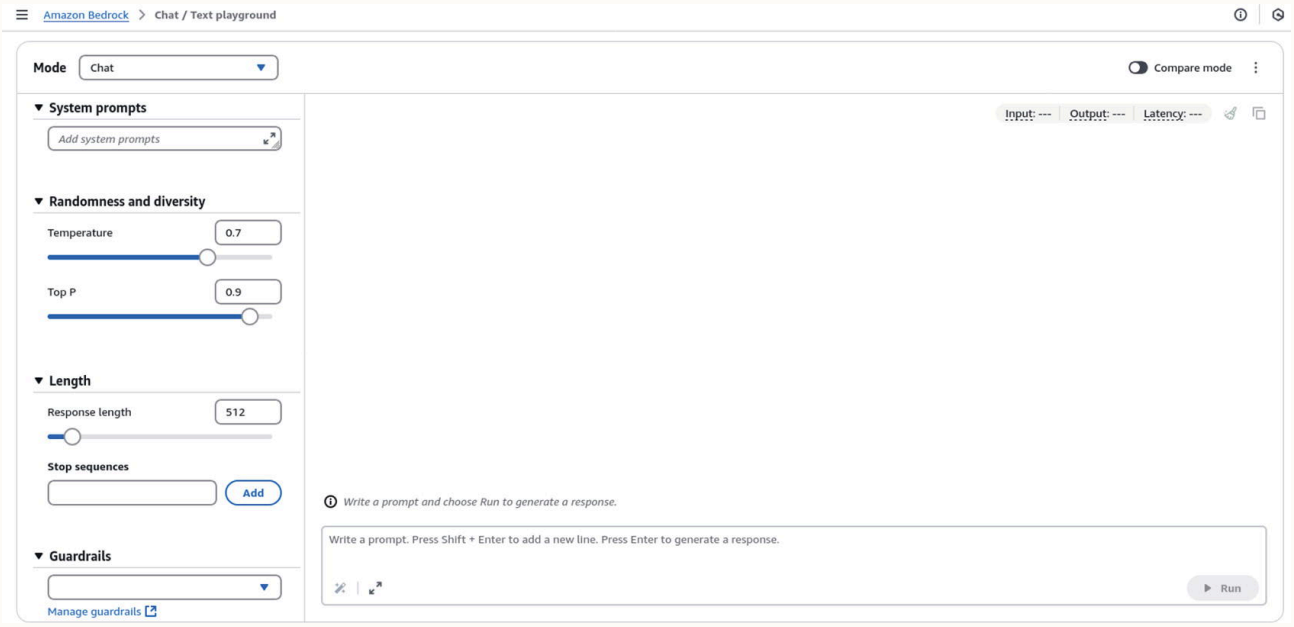


Figure 16-16 Model-activated playground

Let's play with it by using a sample query of "Tell me about airworthiness certificates" and see the response from the model, as shown in Figure 16-17. You can further play with the different options and other prompts and ask the model to act in a specific context or role.

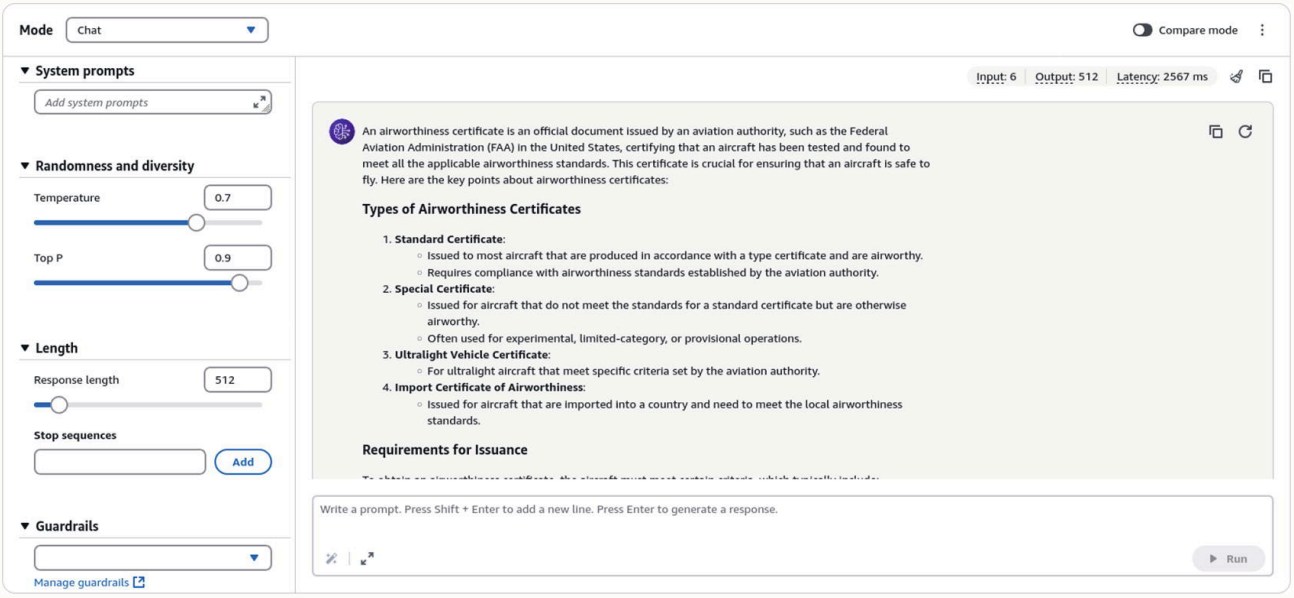


Figure 16-17 Model response in playground

Building a Sample Project Using Amazon Bedrock SDK

You will start building an application now. There are two approaches to build it. You can use the AWS Java SDK for the Amazon Bedrock service, or you can use an existing Java framework's support. Both Spring AI and LangChain4j support integration with Bedrock.

Using AWS SDK to Connect with Amazon Bedrock

Create an initial project at Spring Initializr with a web dependency, as shown in Figure 16-18.

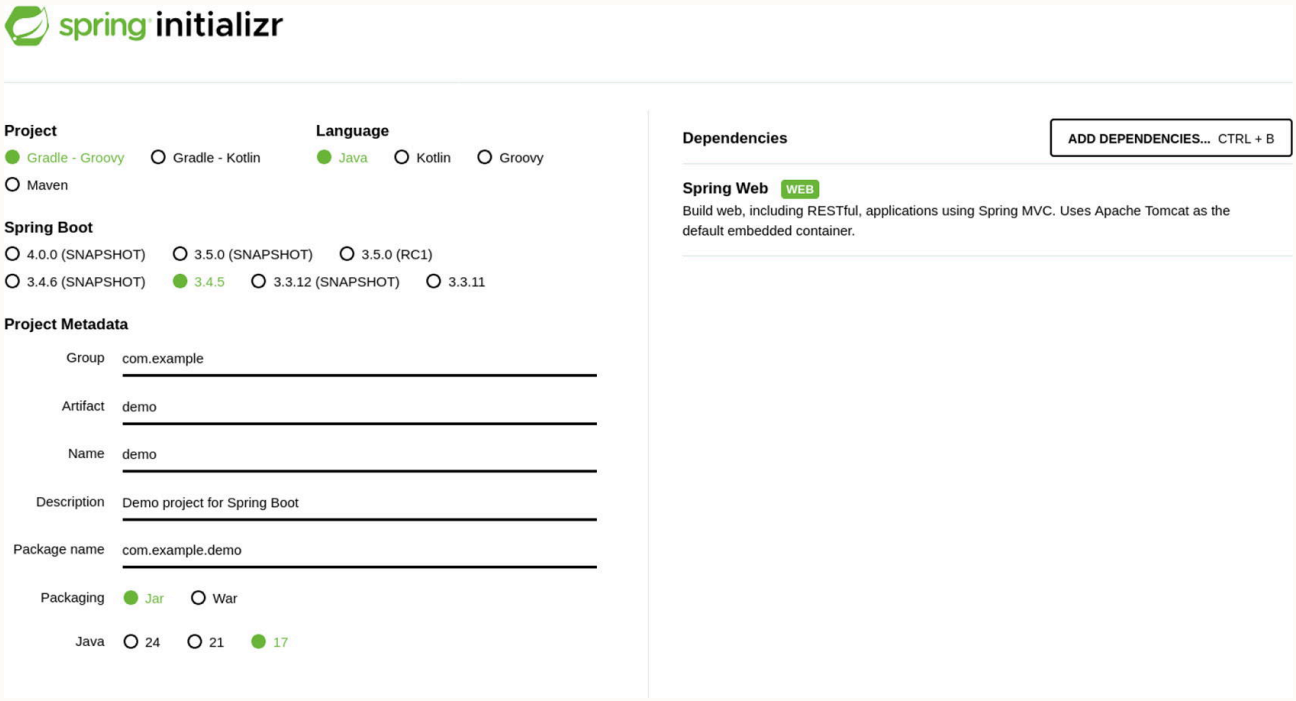


Figure 16-18 Spring Initializr dependency

Download and unzip the sample project at the directory of your choice. You will need to add the AWS SDK for the Java dependency to your build.gradle file.

```
// https://mvnrepository.com/artifact/software.amazon.awssdk/bedrockruntime
implementation("software.amazon.awssdk:bedrockruntime:2.31.45")
```

The build.gradle file should now look like this:

File: ./build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.4.5'
    id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
repositories {
    mavenCentral()
```

```
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'software.amazon.awssdk:bedrockruntime:2.31.45'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') {
    useJUnitPlatform()
}
```

The next step is to create an AWS configuration bean to connect to Amazon Bedrock, as shown next. Replace the region in the application.properties file with the region where you had previously set up the Nova model for use.

File: ./src/main/resources/application.properties

```
spring.application.name=demo
aws.region=ap-south-1
```

File: ./src/main/java/com/example/demo/config/AwsConfig.java

```
package com.example.demo.config;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
@Configuration
public class AwsConfig {
  @Value("${aws.region}")
   private String awsRegion;
  @Bean
   public BedrockRuntimeClient bedrockRuntimeClient() {
      return BedrockRuntimeClient.builder()
            .credentialsProvider(DefaultCredentialsProvider.create())
            .region(Region.of(awsRegion))
            .build();
   }
}
```

Next, you will set up a few data objects that will represent your request and response objects to be used in your controllers and services layer.

File: ./src/main/java/com/example/demo/dto/ConverseRequestDto.java

```
package com.example.demo.dto;
public class ConverseRequestDto {
    private String inputText;
    public ConverseRequestDto() {}
    public ConverseRequestDto(String inputText) { this.inputText = inputText; }
    public String getInputText() { return inputText; }
    public void setInputText(String inputText) { this.inputText = inputText; }
}
```

File: ./src/main/java/com/example/demo/dto/ConverseResponseDto.java

```
package com.example.demo.dto;
public class ConverseResponseDto {
    private String outputText;
    public ConverseResponseDto() {}
    public ConverseResponseDto(String outputText) { this.outputText = outputText; }
    public String getOutputText() { return outputText; }
    public void setOutputText(String outputText) { this.outputText = outputText; }
}
```

The next step is to set up the Service layer, which will take the input from the controller and send a request to the Amazon Bedrock runtime service. You will need the modelId of the model you have enabled previously. This you can see from the model page in the model catalog listing, as shown in Figure 16-19. You can use this as the modelId if you are leveraging provisioned throughput, as shown in the bottom left in the Inference and Assessment section.

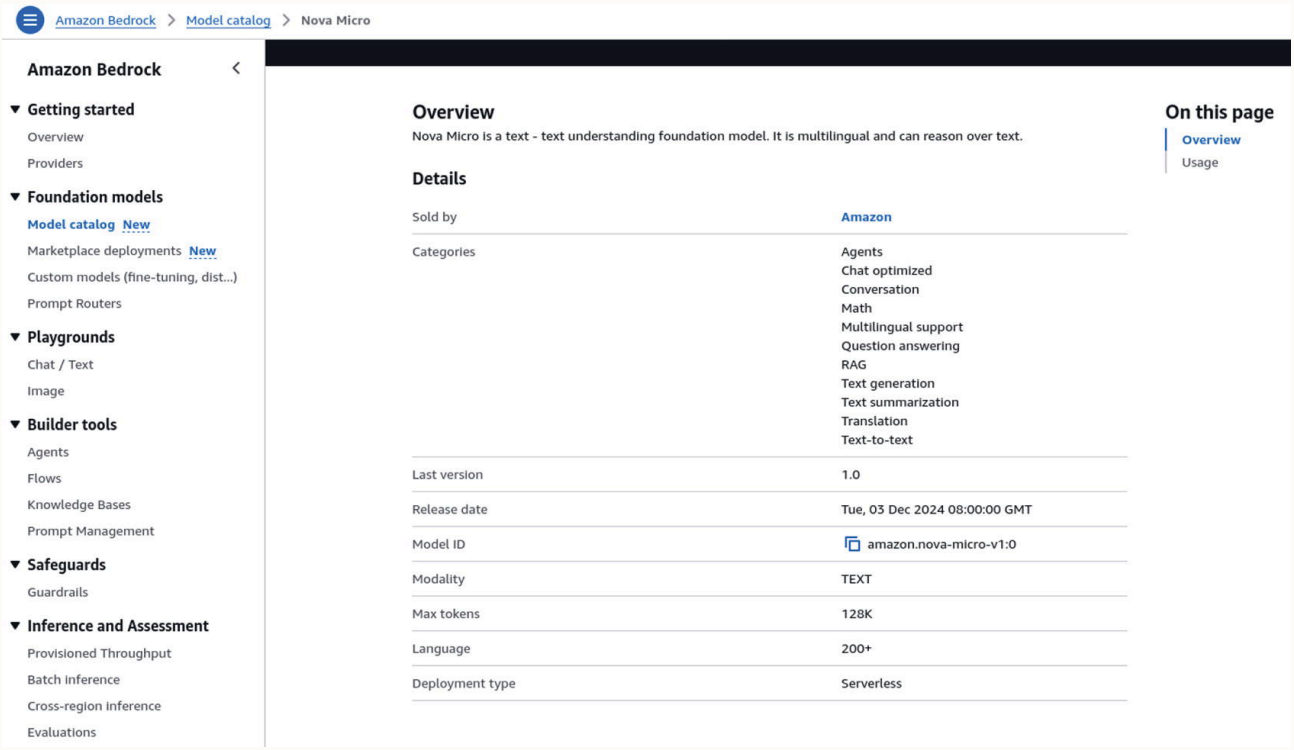


Figure 16-19 Model ID of the model

For this demo, you will use the cross-region inference, as shown in Figure 16-20. Instead of targeting a foundation model directly, you can leverage a cross-region inference profile to distribute requests across multiple regions.

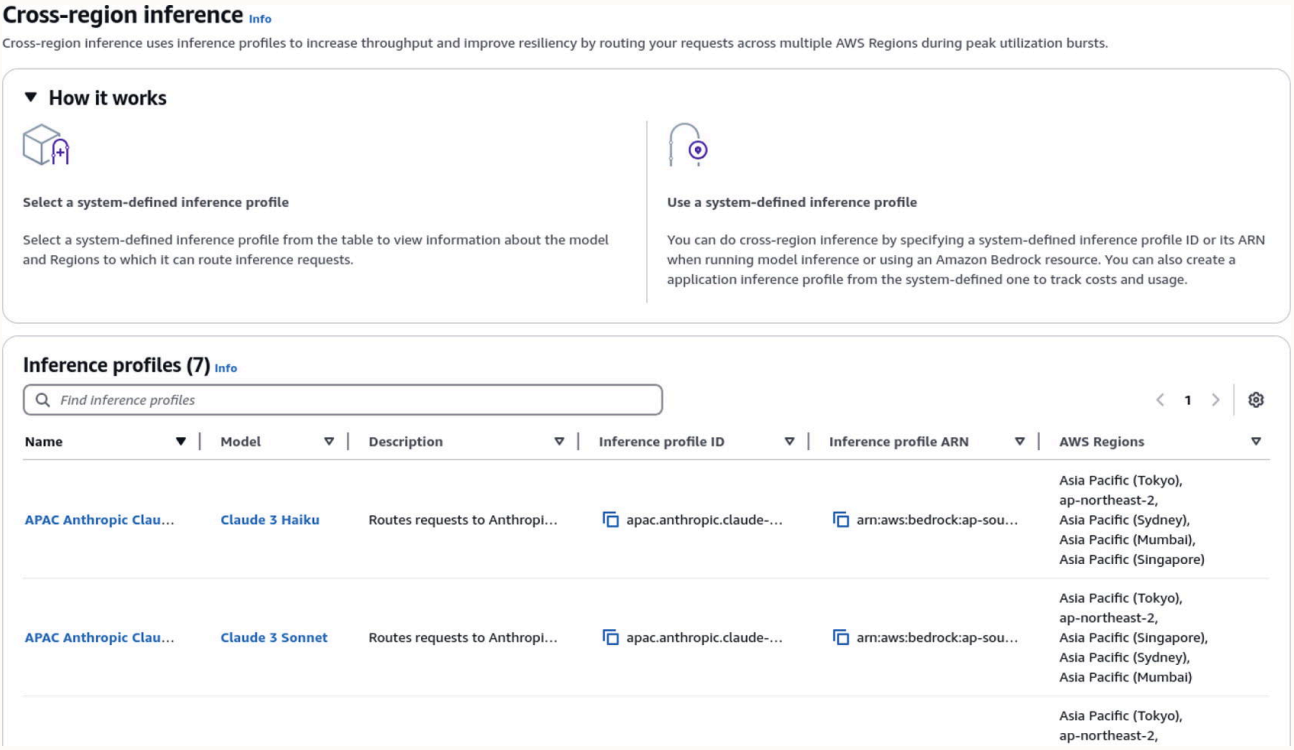


Figure 16-20 Cross-region inference

Copy the inference profile ARN value for the Nova Micro model, which will look something like the following based on your account ID and region where it was enabled previously:

arn:aws:bedrock:ap-south-1:xxxxxxxxxxxx:inference-profile/apac.amazon.nova-micro-v1:0

We will store the modelId value in the application.properties file and refer to the value in the service file.

File: ./src/main/resources/application.properties

```
spring.application.name=demo
aws.region=ap-south-1
aws.bedrock.modelId=arn:aws:bedrock:ap-south-1:xxxxxxxxxxxx:inference-profile/apac.amazon.nova-micro-v1:0
```

The service component does two things. First, it requests at runtime for injection of the Bedrock client bean.

```
public BedrockService(BedrockRuntimeClient client) {
   this.client = client;
}
```

We define a converse method that takes in the user input query as an argument.

```
public String converse(String inputText) {
```

There are two steps here. First, create your Message object from the user input.

```
Message message = Message.builder()
        .role(ConversationRole.USER)
        .content(ContentBlock.fromText(inputText))
        .build();
```

Second, create a request object with modelId and inference properties like temperature and maxTokens.

```
ConverseRequest request = ConverseRequest.builder()
        .modelId(modelId)
        .messages(message)
        .inferenceConfig(cfg -> cfg
                .maxTokens(500)
                .temperature(0.5F)
        )
```

```
        .build();
```

Finally, you will invoke the call to the client.converse method to invoke your request and fetch the text from the returned response.

```
 try {
    ConverseResponse response = client.converse(request);
    return response.output().message().content().get(0).text();
} catch (SdkClientException e) {
    throw new RuntimeException("Failed to invoke Bedrock model: " + e.getMessage(),
e);
}
```

Here is the source code for the service component:

File: ./src/main/java/com/example/demo/service/BedrockService.java

```
package com.example.demo.service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;
@Service
public class BedrockService {
  private final BedrockRuntimeClient client;
  @Value("${aws.bedrock.modelId}")
   private String modelId;
  public BedrockService(BedrockRuntimeClient client) {
      this.client = client;
   }
  /**
    * Sends the given text to Bedrock and returns the model's reply.
    */
   public String converse(String inputText) {
      Message message = Message.builder()
            .role(ConversationRole.USER)
            .content(ContentBlock.fromText(inputText))
            .build();
     ConverseRequest request = ConverseRequest.builder()
            .modelId(modelId)
            .messages(message)
            .inferenceConfig(cfg -> cfg
                  .maxTokens(500)
```

```
                .temperature(0.5F)
            )
            .build();
        try {
            ConverseResponse response = client.converse(request);
            return response.output().message().content().get(0).text();
        } catch (SdkClientException e) {
            throw new RuntimeException("Failed to invoke Bedrock model: " + e.getMessa
ge(), e);
        }
    }
}
```

Now that you have the foundational components set up, you will use them from the controller layer.

File: ./src/main/java/com/example/demo/controller/BedrockController.java

```
package com.example.demo.controller;
import com.example.demo.dto.ConverseRequestDto;
import com.example.demo.dto.ConverseResponseDto;
import com.example.demo.service.BedrockService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/bedrock")
public class BedrockController {
  private final BedrockService bedrockService;
  public BedrockController(BedrockService bedrockService) {
      this.bedrockService = bedrockService;
  }
  @PostMapping("/converse")
   public ResponseEntity<ConverseResponseDto> converse(@RequestBody ConverseR
equestDto req) {
      String reply = bedrockService.converse(req.getInputText());
      return ResponseEntity.ok(new ConverseResponseDto(reply));
  }
}
```

Before you start your application, you have to create a user in AWS IAM with the Amazon Bedrock Full Access policy, as shown in Figures 16-21, 16-22, and 16-23. For a quick setup here, we are using Full access, but for production access you should have a fine-grained access policy.

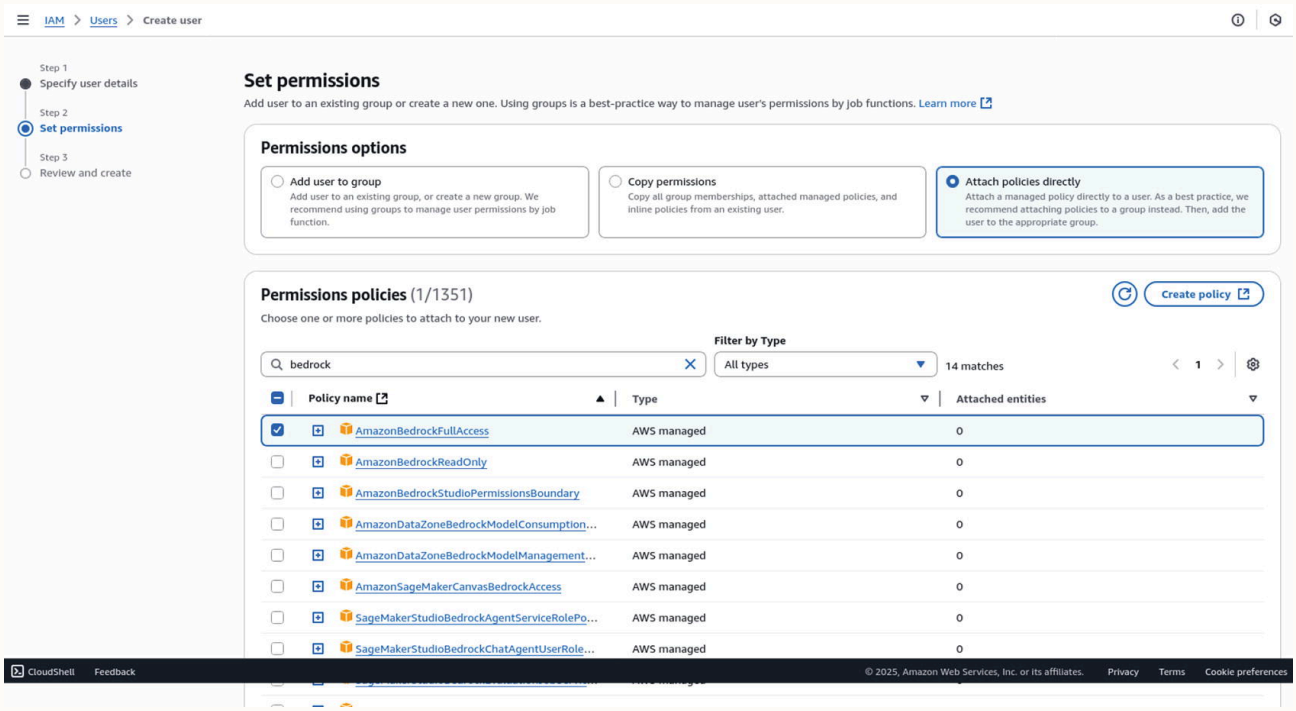


Figure 16-21 Assigning the Bedrock policy to a user

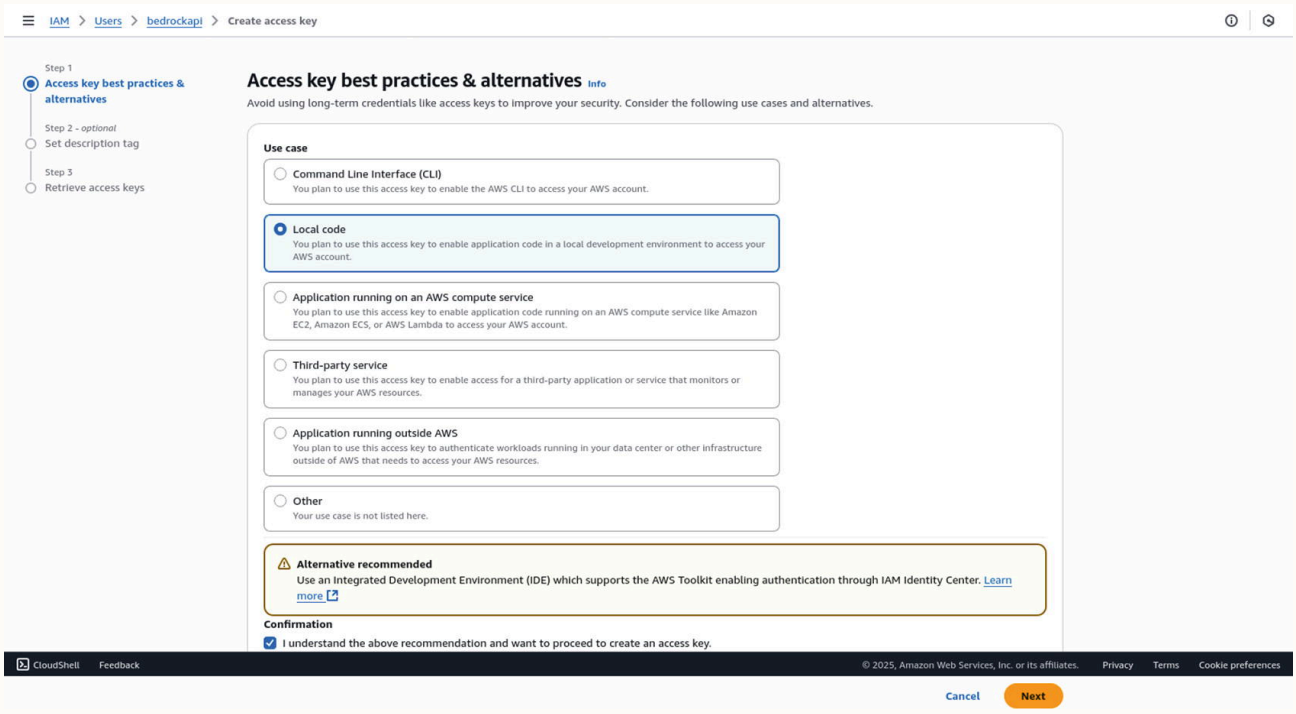


Figure 16-22 Used from local code

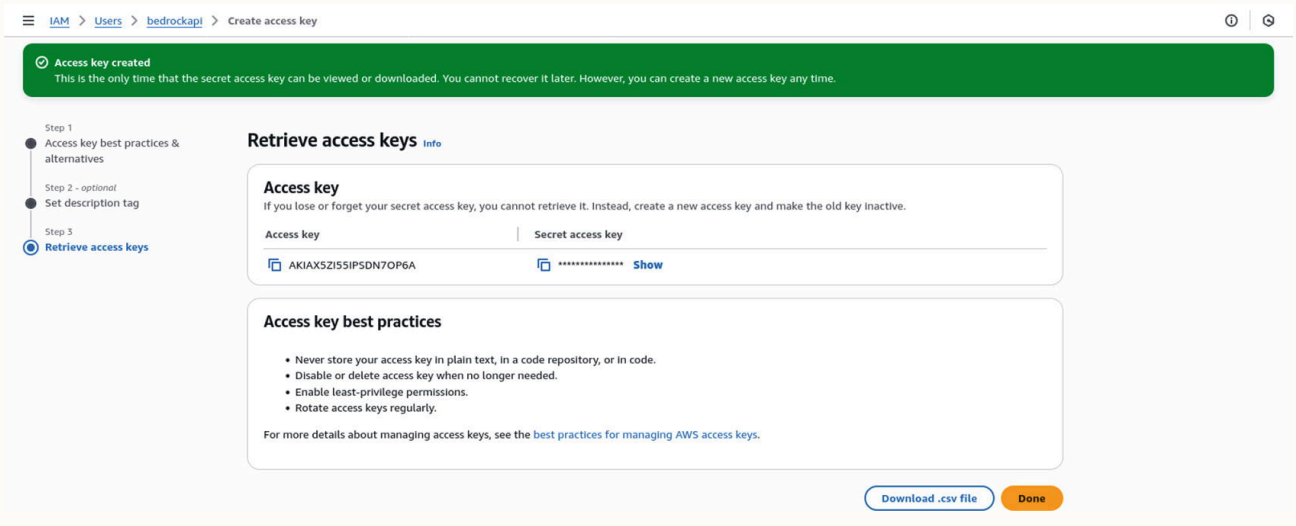


Figure 16-23 Save the access key from security credentials in the user interface

Set the access key and secret key in your machine by running aws configure and setting them.

Let's boot up your application and see it in action by making a curl request as shown here:

```
curl -X POST localhost:8080/api/bedrock/converse   -H "Content-Type: application/js
on"   -d '{"inputText":"Describe a hello world program in one line."}'
```

Output:

```
{"outputText":"A \"Hello, World!\" program in one line typically prints \"Hello, World!
\" to the console. Here's an example in Python:\n\n```python\nprint(\"Hello, World!
\")\n```\n\nThis single line of code uses the `print` function to output the string \"He
llo, World!\" to the standard output."}
Wow, our first application with Amazon Bedrock. Cool enough!
```

Extending and Integrating Your Application with Guardrails

Guardrails are rule-based safety nets placed around AI models to ensure they behave in predictable, policy-compliant ways. Rather than letting foundation models respond freely, you define boundaries—what content is allowed, which topics to avoid, and how to handle sensitive or factual information. By enforcing these constraints automatically, guardrails help you maintain trust, prevent misuse, and keep your AI-driven features aligned with your organization's guidelines. Guardrails operate at two critical checkpoints:

1. **Before the call**
   All incoming prompts—including the user's request and any contextual data—are first inspected and sanitized. This "pre-inference" filtering ensures that only well-formed, unbiased, and ethically acceptable queries ever reach the model.
2. **After the call**
   Once the model produces an output, that response is subjected to a second layer of checks. Here, guardrails verify that the content is factually accurate, on-topic, and free from unwanted bias before it's delivered to the user.

Amazon Bedrock Guardrails: Enabling Responsible AI at Scale

Amazon Bedrock Guardrails is a turnkey framework of configurable protections that you can layer over any generative model—whether it's a native Bedrock foundation model, a fine-tuned version, or a third-party endpoint. Here's how it helps you build safer, more reliable AI services:

1. **Factuality through contextual grounding**
   Before delivering a response, Bedrock Guardrails can check the model's output against your source data (for example, documents supplied to a retrieval-augmented generator or transcripts in a summarization task). Any answer that doesn't directly tie back to the reference material—hallucinations, contradictions, or invented facts—can be flagged or filtered out, so users see only answers grounded in known truth.
2. **Logic-driven verification with Automated Reasoning (Preview)**
   Go beyond keyword-blocking: Automated Reasoning applies formal, mathematics-inspired checks to verify that each answer conforms to a policy document you provide (such as an HR handbook or technical specification). It not only spots logical inconsistencies or fabrications but also generates an explanation of why the response is valid, giving you provable guarantees that the AI isn't straying into fiction.
3. **Custom topic exclusion**
   You can define, in simple natural-language terms, any subject areas or domains that your application must steer clear of—investment advice in a banking assistant, for instance. Bedrock Guardrails then inspects both user queries and model outputs, automatically blocking or sanitizing any content that ventures into those off-limits categories.
4. **Comprehensive multimodal filters**
   To protect against hate speech, graphic violence, explicit imagery, and other forms of harmful content, guardrails applies adjustable toxicity checks across both text and images. These filters also guard against prompt-injection or "jailbreak" attacks by screening inputs and outputs for malicious or unintended instructions.
5. **Sensitive-data detection and redaction**
   Privacy is paramount. Guardrails can automatically recognize patterns for personally identifiable information (PII) or any custom-defined sensitive data (email addresses, credit-card numbers, etc.). Depending on your needs, the system can either reject inputs containing such data or strip it out of the model's responses—ideal for scenarios like redacting customer details from call-center summaries.

Let's build guardrails around your Gen AI application. As shown in Figure 16-24, there are three steps to integrate guardrails.

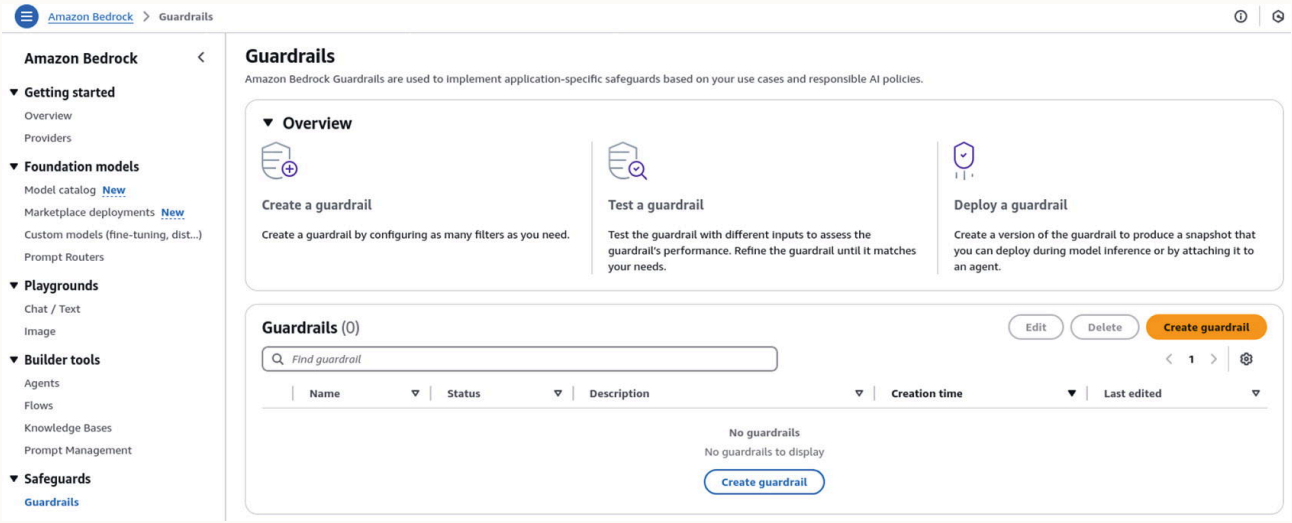


Figure 16-24 Amazon Bedrock guardrails

The steps are as follows:

1. Create a guardrail.

   Define a guardrail by setting up as many filters as needed.

   Click the "Create guardrail" button. This takes you to Figure 16-25 form where you need to share a few details like name, description, and message to respond with if the input contains any information against the guardrail rules.

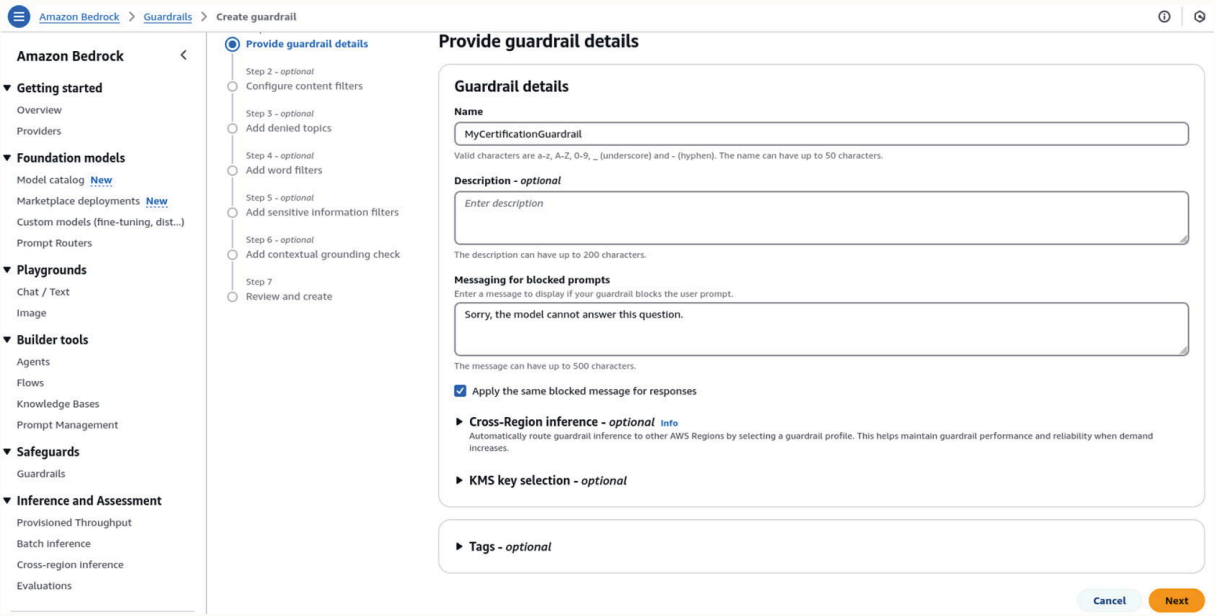


Figure 16-25 Guardrail metadata

Then you need to configure the content filters. Let's enable the filters for harmful categories and prompt attacks.

Harmful categories: Activate safeguards to identify and block dangerous user inputs and model outputs. Raising the filter's sensitivity will tighten screening and reduce harmful content across all categories. These can be enabled for both text and image, and different thresholds can be chosen, as shown in Figure 16-26. Let's keep the defaults enabled for your app.

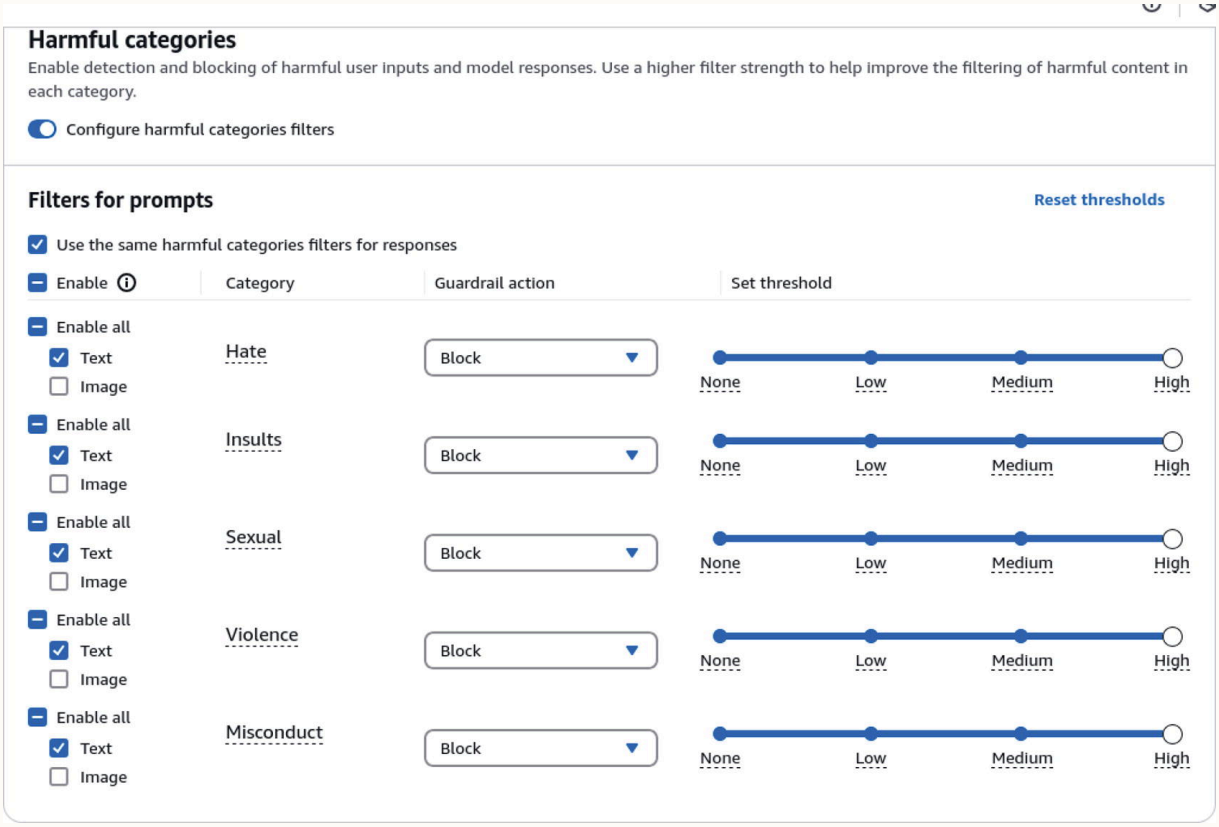


Figure 16-26 Harmful categories options

Prompt attacks: Turn on protections that catch and block any user message attempting to override system instructions. To prevent legitimate system prompts from being misidentified, tag inputs so the filters apply only to user-originated text. The values can be enabled for threshold similarly in this category too, as shown in Figure 16-27 and Figure 16-28.

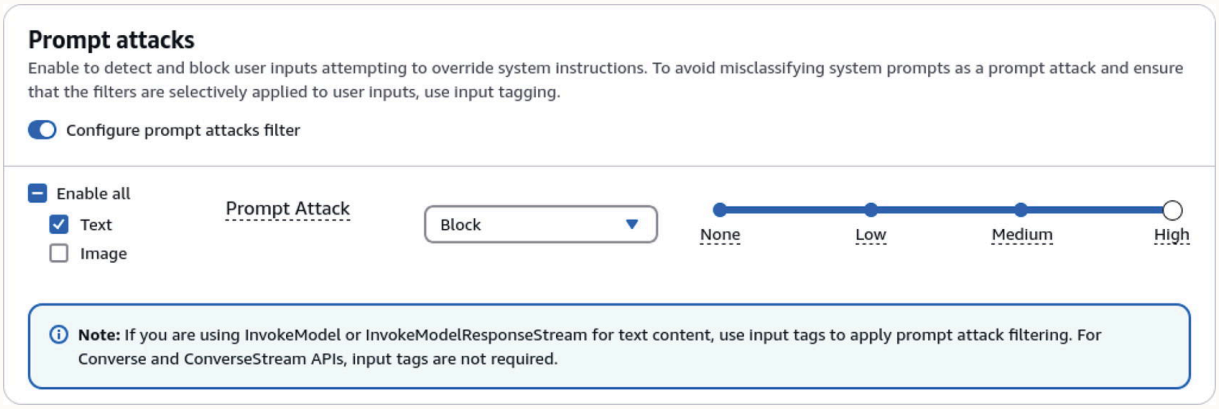


Figure 16-27 Prompt attacks configuration

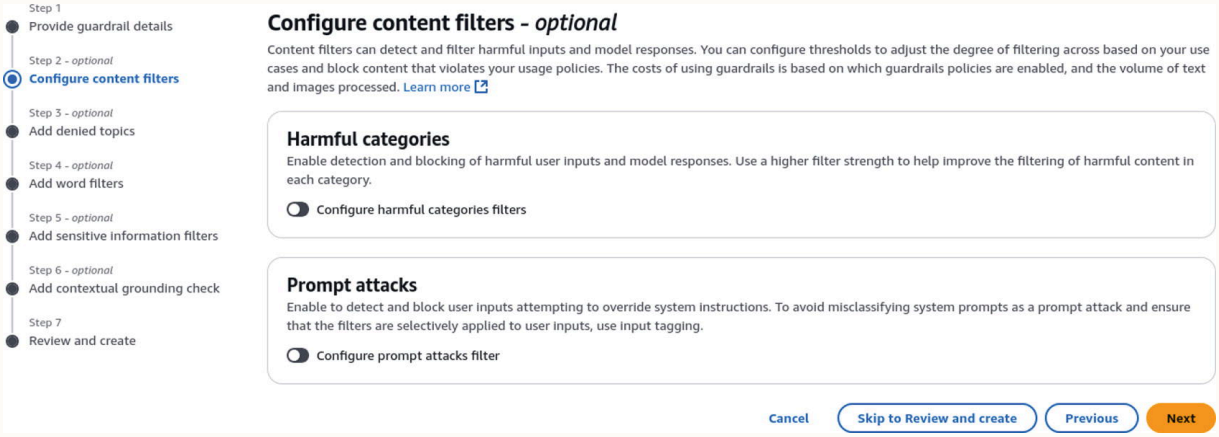
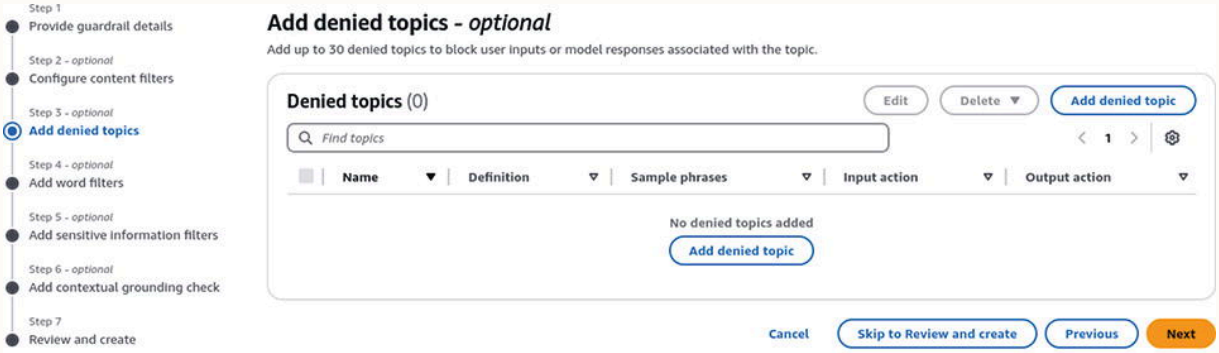


Figure 16-28 Configure content filters

After this, you can enable any denied topics like do not allow the user to ask for any existing patent-related information. It should all be from the certification technical details information. You can proceed, as shown in Figure 16-29.



Add denied topic

Name

Enter name

Valid characters are a-z, A-Z, 0-9, underscore (_), hyphen (-), space, exclamation point (!), question mark (?), and period (.). The name can have up to 100 characters.

Definition

Provide a clear definition to detect and block user inputs and FM responses that fall into this topic. Avoid starting with "don't".

Example - Investment advice refers to inquiries, guidance, or recommendations regarding the management or allocation of funds or assets with the goal of generating returns or achieving specific financial objectives.

The definition can have up to 200 characters.

Input

☑ Enable

Input action

Choose what action the guardrail should take on user inputs before they reach the model.

Block

Output

☑ Enable

Output action

Choose what action the guardrail should take on model outputs before displayed to users.

Block

▶ **Add sample phrases - *optional***

Cancel Confirm

Edit denied topic

Name

patent info deny

Valid characters are a-z, A-Z, 0-9, underscore (_), hyphen (-), space, exclamation point (!), question mark (?), and period (.). The name can have up to 100 characters.

Definition

Provide a clear definition to detect and block user inputs and FM responses that fall into this topic. Avoid starting with "don't".

Information relevant to showing any existing design, technicalitites for a IP or patent.

The definition can have up to 200 characters.

Input

☑ Enable

Input action

Choose what action the guardrail should take on user inputs before they reach the model.

Block



Figure 16-29 Denied topics configuration

After this comes the profanity filter, as shown in Figure 16-30. Activate this option to automatically filter out profanity from both user inputs and AI-generated responses. It relies on a globally maintained list of offensive terms, which is periodically updated. Also, new custom words specific to your application can also be uploaded manually or through a file or S3 location.

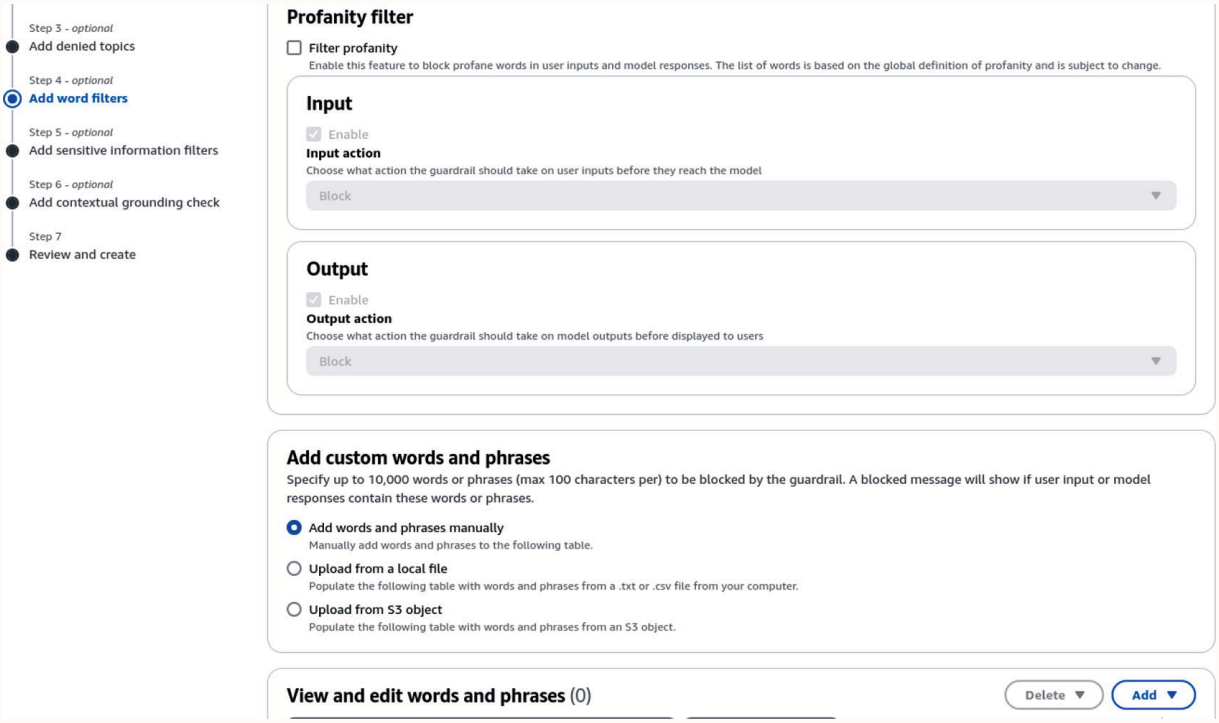
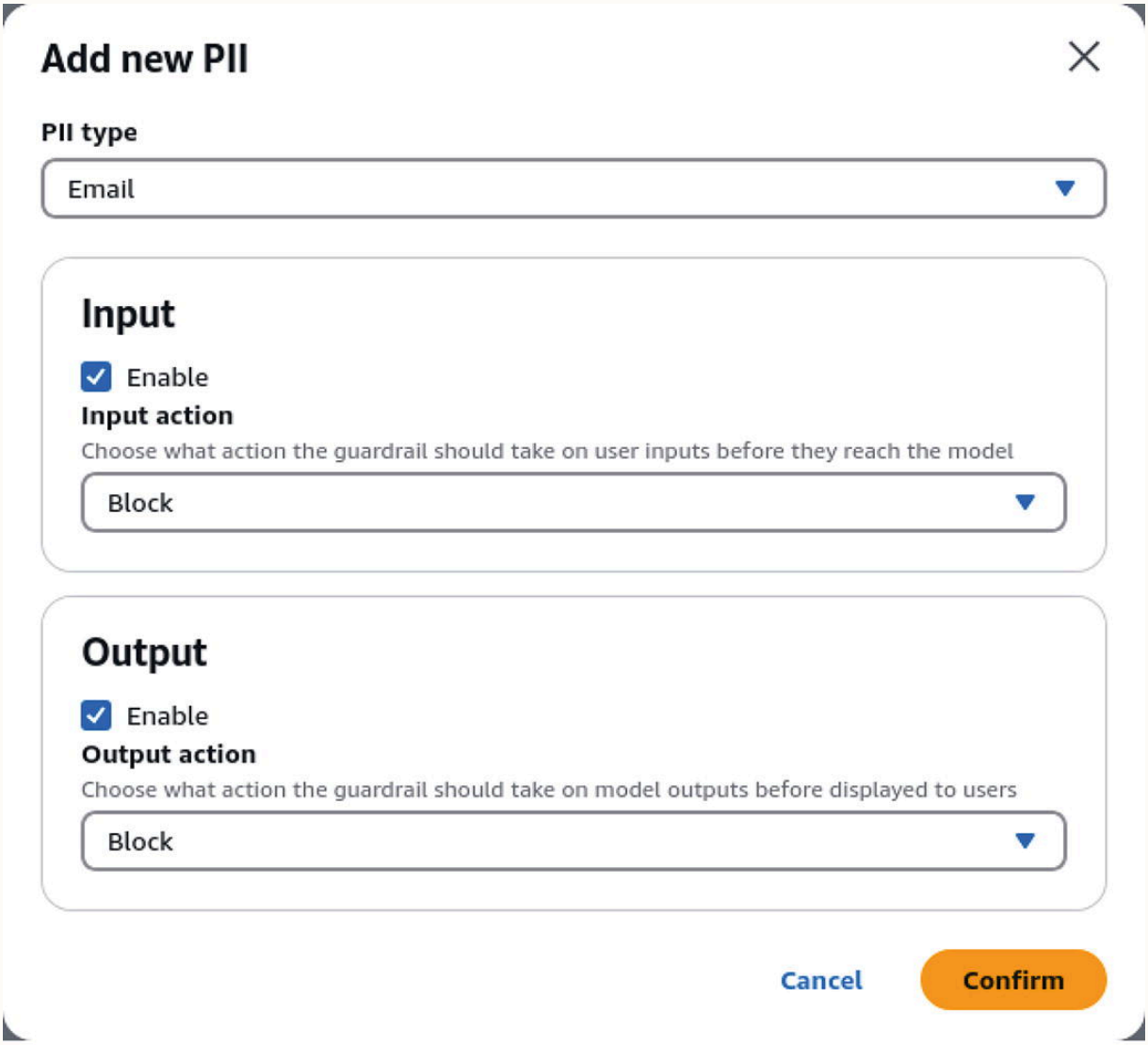
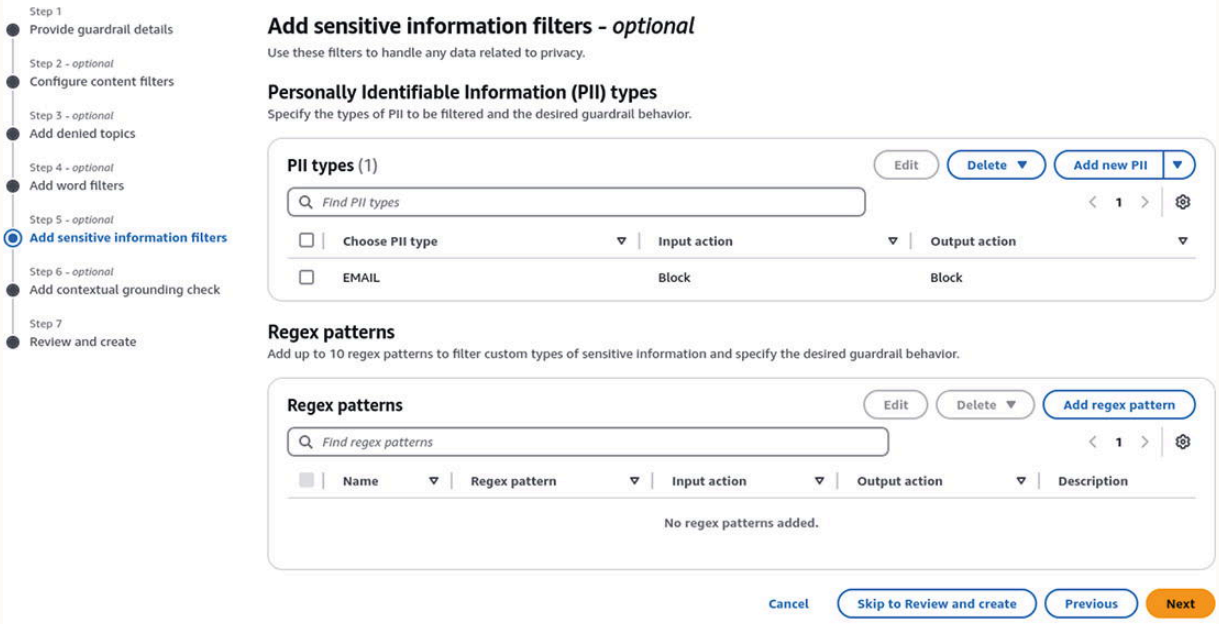
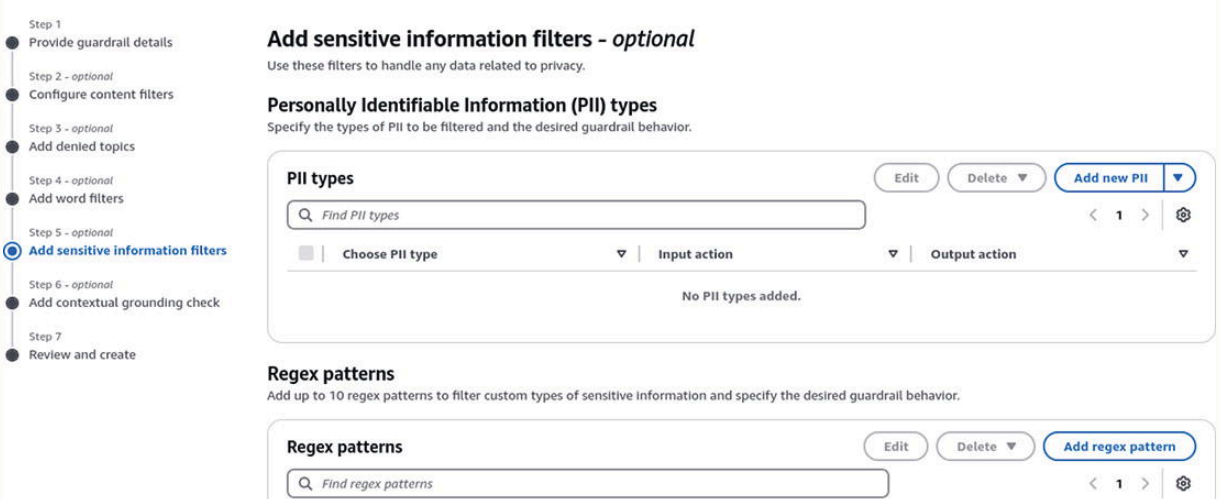


Figure 16-30 Profanity filter configuration

The next step is to add any sensitive information filters like PII categories. In Figure 16-31, I have chosen the email address category. You can choose from varied other categories based on your requirement, as shown in Figure 16-31.


Add new PII
PII type
Email
Input
Enable
Input action
Choose what action the guardrail should take on user inputs before they reach the model
Block
Output
Enable
Output action
Choose what action the guardrail should take on model outputs before displayed to users
Block
Cancel
Confirm


Step 1
Provide guardrail details
Step 2 - optional
Configure content filters
Step 3 - optional
Add denied topics
Step 4 - optional
Add word filters
Step 5 - optional
Add sensitive information filters
Step 6 - optional
Add contextual grounding check
Step 7
Review and create
Add sensitive information filters - optional
Use these filters to handle any data related to privacy.
Personally Identifiable Information (PII) types
Specify the types of PII to be filtered and the desired guardrail behavior.
PII types (1)
Edit
Delete
Add new PII
Find PII types
1
Choose PII type
Input action
Output action
EMAIL
Block
Block
Regex patterns
Add up to 10 regex patterns to filter custom types of sensitive information and specify the desired guardrail behavior.
Regex patterns
Edit
Delete
Add regex pattern
Find regex patterns
1
Name
Regex pattern
Input action
Output action
Description
No regex patterns added.
Cancel
Skip to Review and create
Previous
Next


Step 1
Provide guardrail details
Step 2 - optional
Configure content filters
Step 3 - optional
Add denied topics
Step 4 - optional
Add word filters
Step 5 - optional
Add sensitive information filters
Step 6 - optional
Add contextual grounding check
Step 7
Review and create
Add sensitive information filters - optional
Use these filters to handle any data related to privacy.
Personally Identifiable Information (PII) types
Specify the types of PII to be filtered and the desired guardrail behavior.
PII types
Edit
Delete
Add new PII
Find PII types
1
Choose PII type
Input action
Output action
No PII types added.
Regex patterns
Add up to 10 regex patterns to filter custom types of sensitive information and specify the desired guardrail behavior.
Regex patterns
Edit
Delete
Add regex pattern
Find regex patterns
1

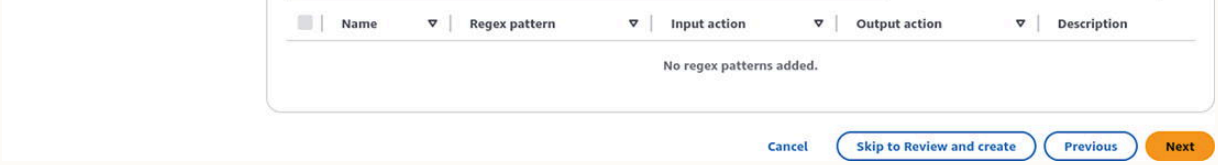


Figure 16-31 PII configuration

The next step is to enable grounding check if any context source is provided, as shown in Figure 16-32. This is important since you will be using RAG. Let's enable checks for grounding and relevance.

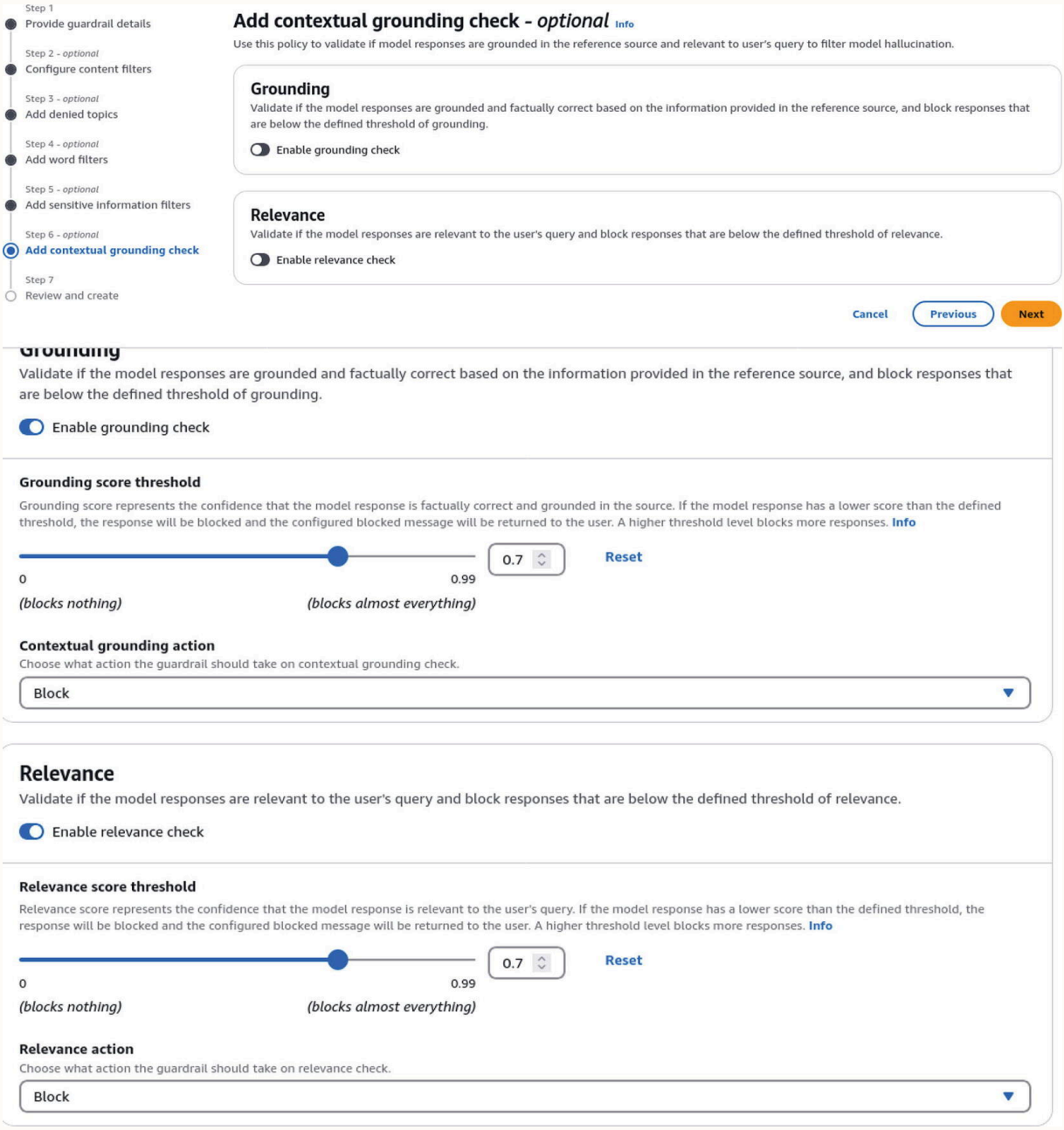


Figure 16-32 Grounding check configuration

Finally, on the review page, review all the changes made and click "Create guardrail," as shown in Figure 16-33.

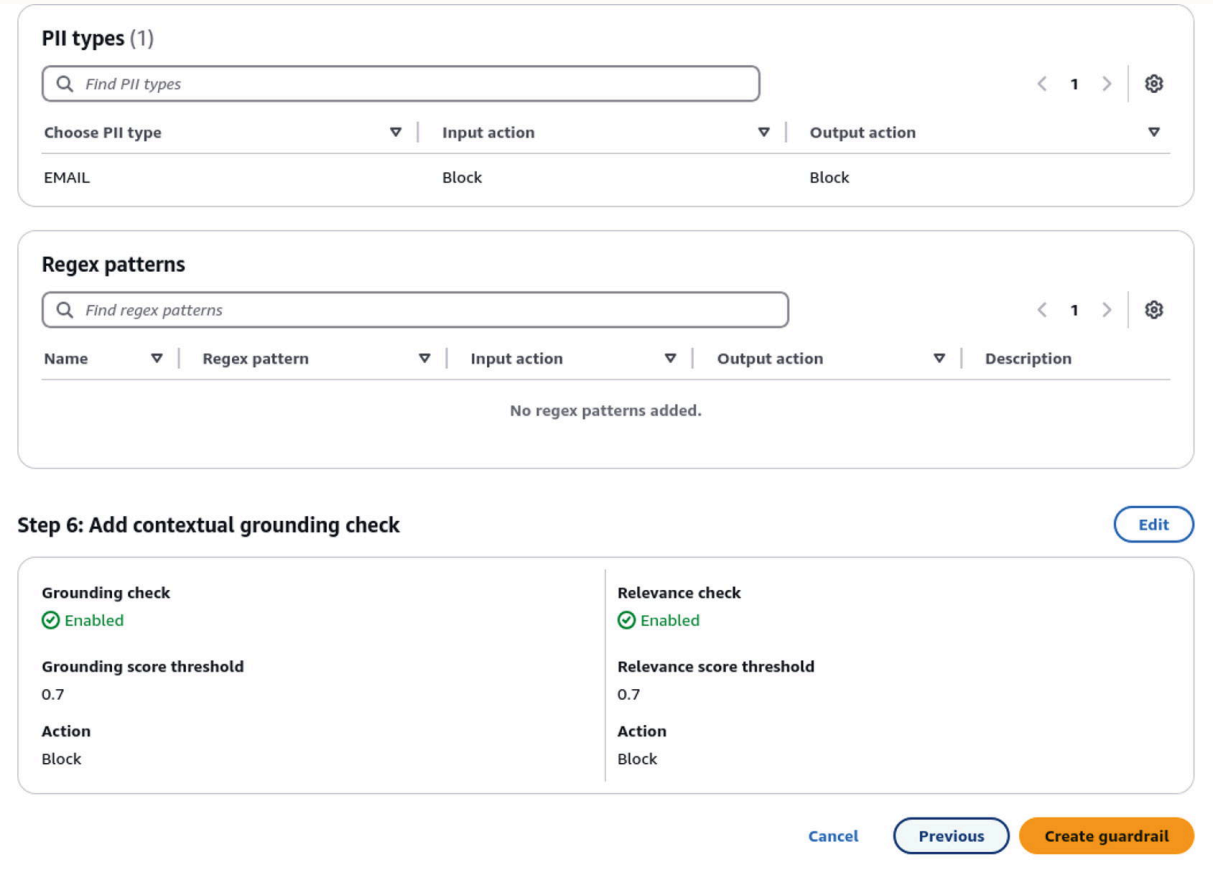


Figure 16-33 Review and create page

2. Test the guardrail.

Once your guardrail is created, you can test it in the same interface (see Figure 16-34).

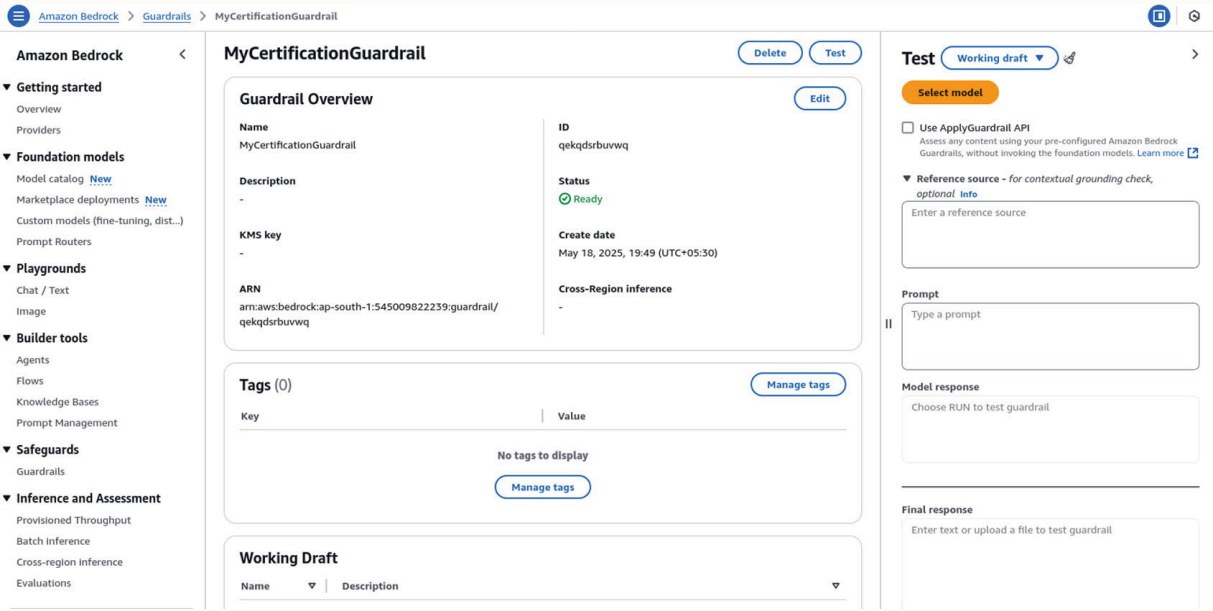


Figure 16-34 Guardrail overview page with test section

Select the Nova micro model from the options list (see Figure 16-35).

Test

Working draft

Nova Micro *1.0*
APAC Nova Micro

☐ Use ApplyGuardrail API
Assess any content using your pre-configured Amazon Bedrock Guardrails, without invoking the foundation models. Learn more

▼ **Reference source** - *for contextual grounding check, optional* Info

Enter a reference source

Prompt

Type a prompt

Model response

Choose RUN to test guardrail

Final response

Enter text or upload a file to test guardrail

Figure 16-35 Testing the Nova Micro model with guardrails

For contextual grounding check, you will take a snippet of information from the FAA website at **https://www.ecfr.gov/current/title-14/chapter-III/subchapter-A/part-400/section-400.2**.

The regulations in this chapter set forth the procedures and requirements applicable to the authorization and supervision under 51 U.S.C. subtitle V, chapter 509, of commercial space transportation activities conducted in the United States or by a U.S. citizen. The regulations in this chapter do not apply to the following:

(a) Space activities carried out by the United States Government on behalf of the United States Government;

(b) The launch of an amateur rocket as defined in § 1.1 of chapter I of this title; or

(c) A launch of a tethered launch vehicle that meets all the following criteria:

(1) Launch vehicle. The launch vehicle must—

(i) Be unmanned;

(ii) Be powered by a liquid or hybrid rocket motor;

(iii) Not use any of the toxic propellants of Table I417-2 and Table I417-3 in Appendix I of part 417 of this chapter; and

(iv) Carry no more than 5,000 pounds of propellant.

(2) Tether system. The tether system must—

(i) Not yield or fail under—

(A) The maximum dynamic load on the system; or

(B) A load equivalent to two times the maximum potential engine thrust.

(ii) Have a minimum safety factor of 3.0 for yield stress and 5.0 for ultimate stress.

(iii) Constrain the launch vehicle within 75 feet above ground level as measured from the ground to the attachment point of the vehicle to the tether.

(iv) Display no damage prior to the launch.

(v) Be insulated or located such that it will not experience thermal damage due to the launch vehicle's exhaust.

(3) Separation distances. The launch operator must separate its launch from the public and the property of the public by a distance no less than that provided for each quantity of propellant listed in Figure **16-36**.

TABLE A—SEPARATION DISTANCES FOR TETHERED LAUNCHES

Propellant carried (lbs.)	Distance (ft.) of the public and property of the public from the launch point
1-500	900
501-1,000	1,200
1001-1,500	1,350
1,501-2,000	1,450
2,001-2,500	1,550
2,501-3,000	1,600
3,001-3,500	1,650
3,501-4,000	1,700
4,001-4,500	1,750
4,501-5,000	1,800

Figure 16-36 Certification information in tabular format

Fill in the details in the form and ask for the patent idea that you had asked the guardrail to block. Let's see its output when you run it. Figure 16-37 shows the output.

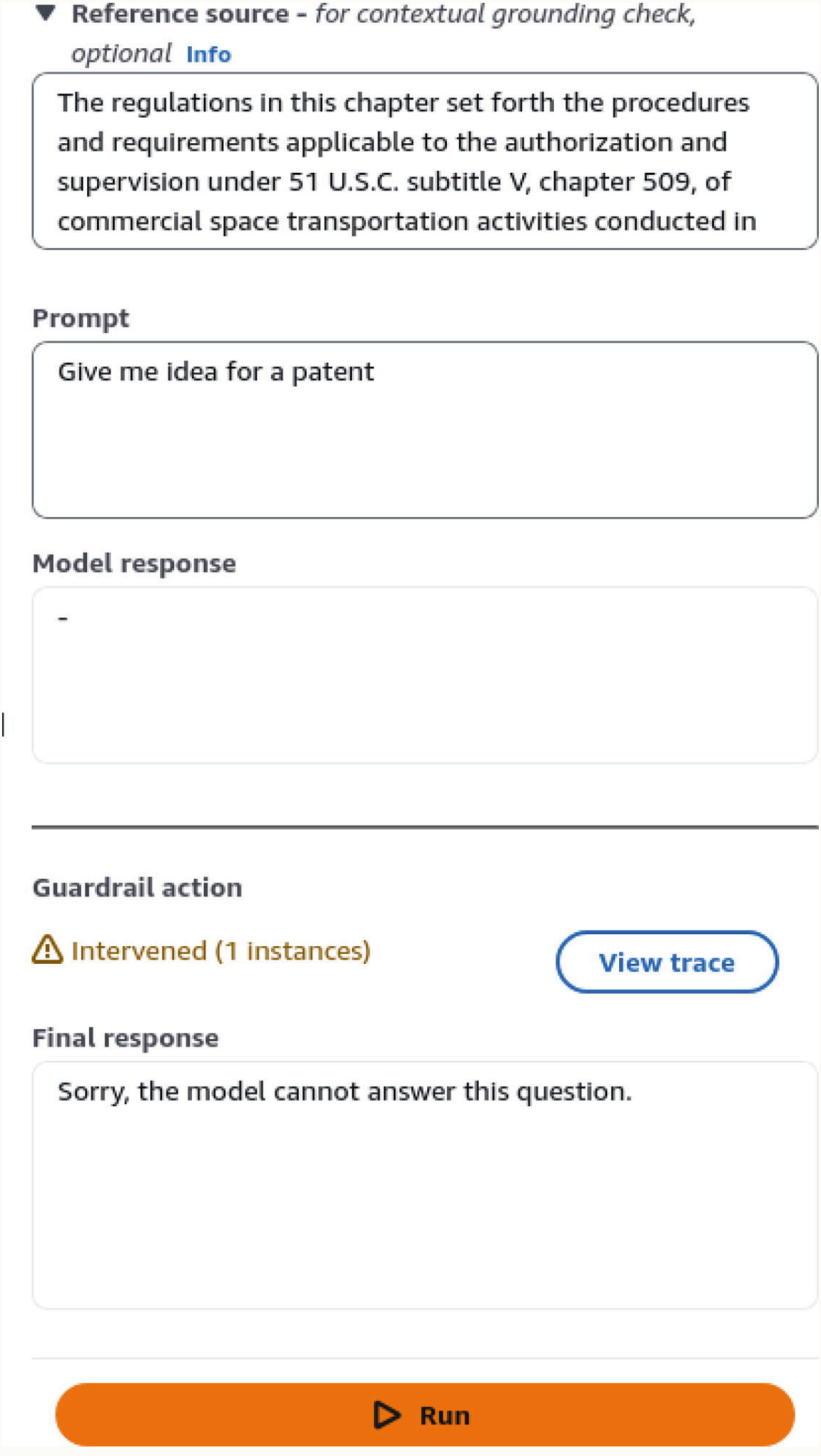


Figure 16-37 Testing guardrail actions

It seems to be working since the model did not share any response, but the final response was intervened by the guardrail action. You can view the trace of which actions were used (see Figure 16-38).

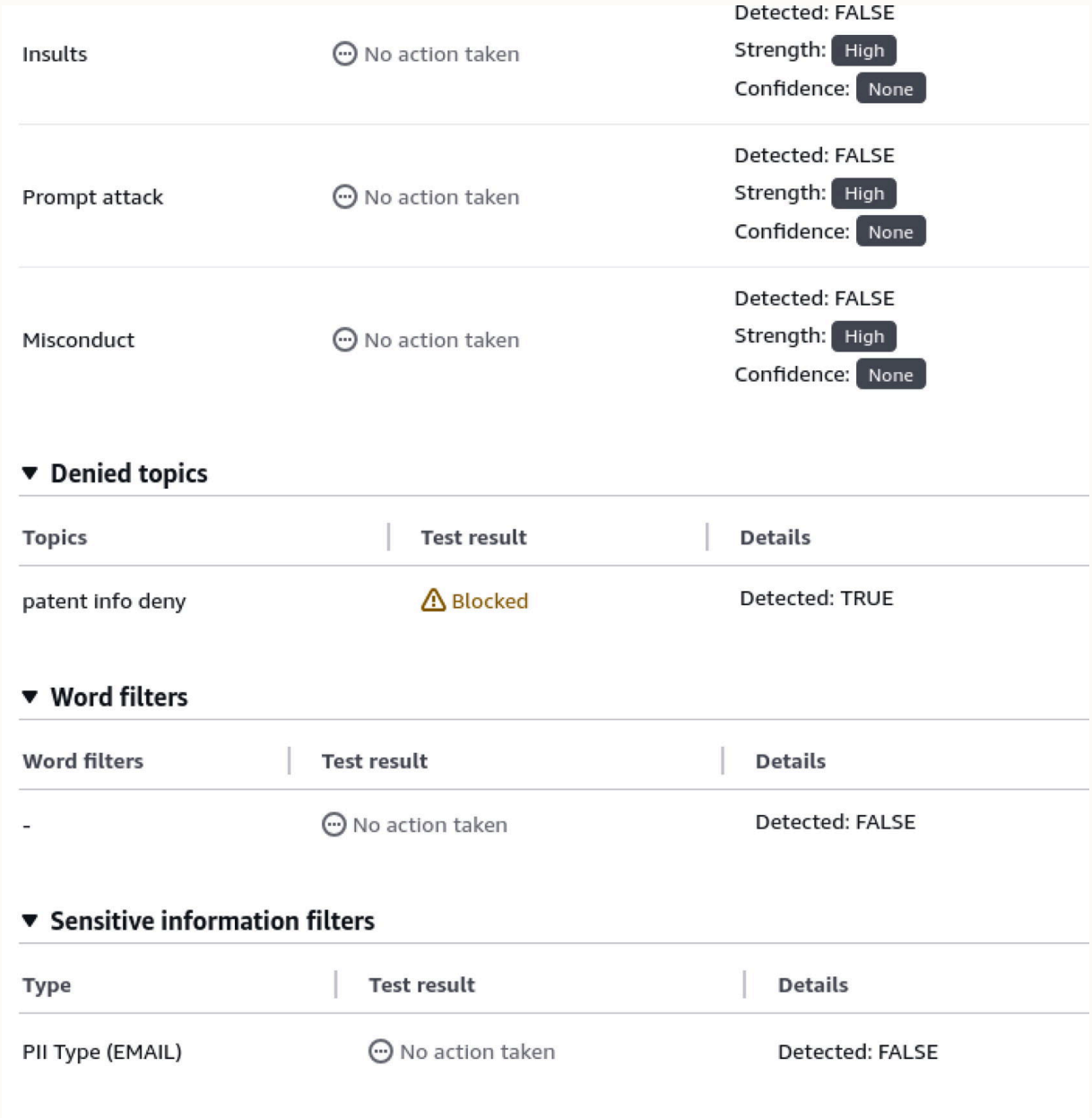


Figure 16-38 Overview of different detections through the guardrail

As you can see, other filters were detected as false, but the denied topics were activated for patent info deny category. This is really helpful, and you can test other scenarios before accepting this guardrail for deployment.

3. Deploy the guardrail.

Once you have tested the guardrail, you can deploy it by creating a first version of it (see Figure 16-39).

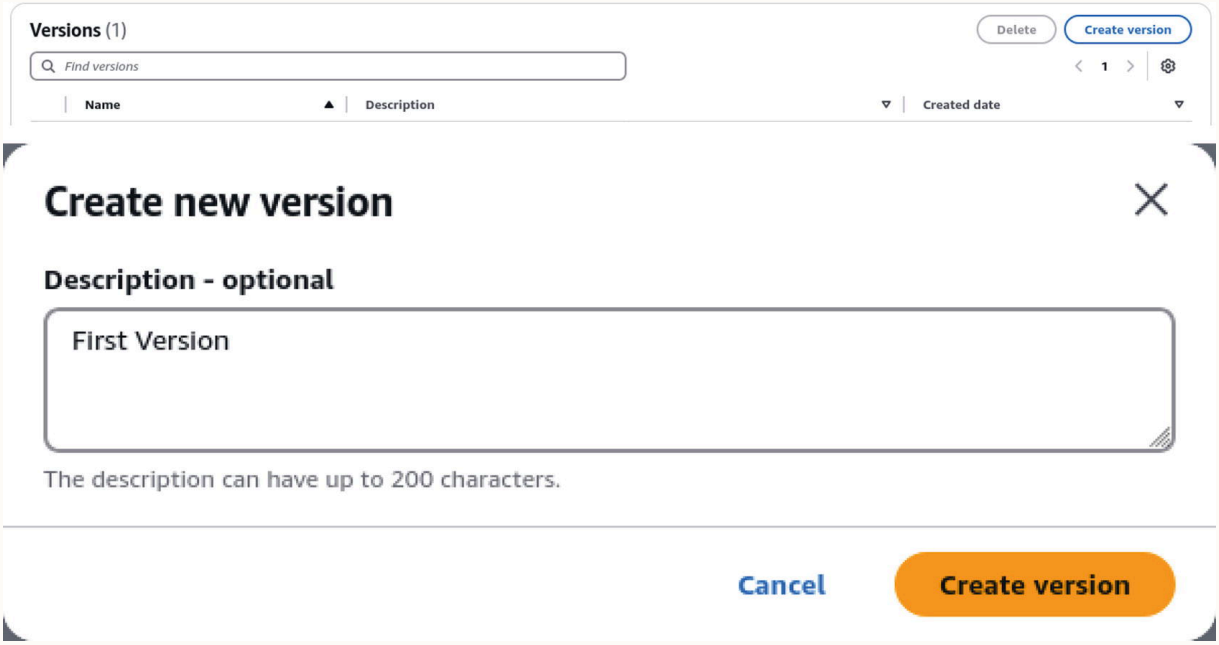


Figure 16-39 Guardrail versions

Once the version is created, it will show in ready state for usage. You will need the arn property of the guardrail by clicking the guardrail and using the arn property in the detail page (see Figure 16-40).

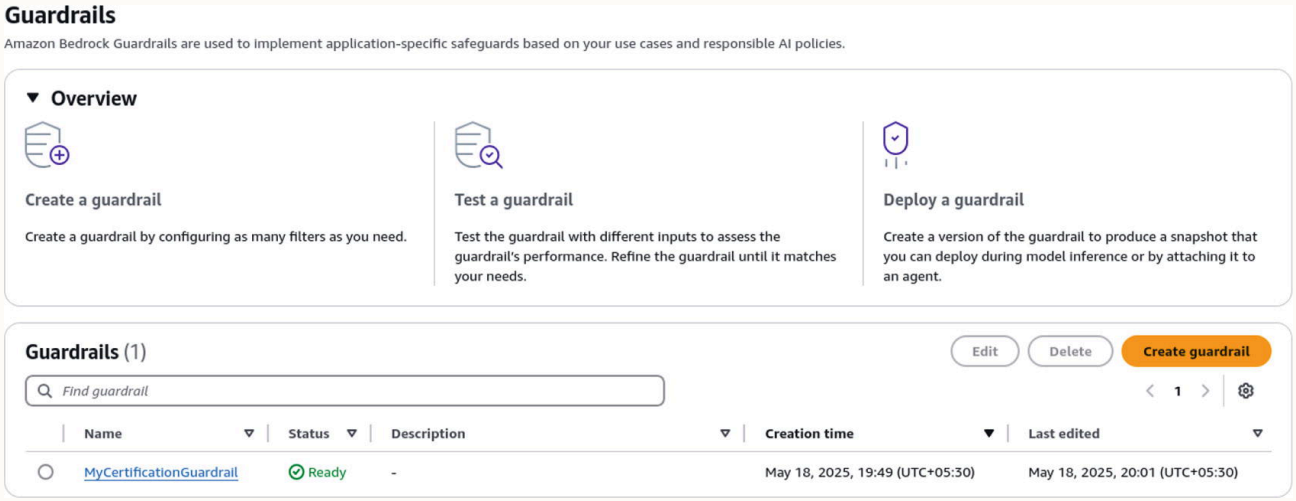


Figure 16-40 Guardrails listing overview

Now that you have the guardrail for use, let's use it in the application you developed previously here. The first step is to note the arn property and version of your guardrail and set it in application.properties for later use in the service component.

File: ./src/main/resource/application.properties

```
spring.application.name=demo
aws.region=ap-south-1
aws.bedrock.modelId=arn:aws:bedrock:ap-south-1:xxxxxxxxxxxx:inference-profile/apac.amazon.nova-micro-v1:0
```

```
aws.bedrock.guardrailIdentifier=arn:aws:bedrock:ap-south-1:xxxxxxxxxxxx:guardrail/x
xxxxxxxxxxx
aws.bedrock.guardrailVersion=1
```

Once these are set, you can proceed to the service file to set the properties in variables.

```
@Value("${aws.bedrock.modelId}")
private String modelId;
@Value("${aws.bedrock.guardrailIdentifier}")
private String guardrailIdentifier;
@Value("${aws.bedrock.guardrailVersion}")
private String guardrailVersion;
```

The ConverseRequest that you used previously additionally accepts a guardrailConfiguration object, which you can create using the GuardrailConfiguration builder API. It accepts the guardrail identifier, which is your guardrail arn property and the guardrail version.

```
GuardrailConfiguration guardrailConfiguration = GuardrailConfiguration.builder()
        .guardrailIdentifier(guardrailIdentifier)
        .guardrailVersion(guardrailVersion)
        .build();
ConverseRequest request = ConverseRequest.builder()
        .modelId(modelId)
        .guardrailConfig(guardrailConfiguration)
        .messages(message)
        .inferenceConfig(cfg -> cfg
                .maxTokens(500)
                .temperature(0.5F)
        )
        .build();
```

The complete code change for the service is shown here:

File: ./src/main/java/com/example/demo/service/BedrockService.java

```
package com.example.demo.service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;
```

```
@Service
public class BedrockService {
    private final BedrockRuntimeClient client;
    @Value("${aws.bedrock.modelId}")
    private String modelId;
    @Value("${aws.bedrock.guardrailIdentifier}")
    private String guardrailIdentifier;
    @Value("${aws.bedrock.guardrailVersion}")
    private String guardrailVersion;
    public BedrockService(BedrockRuntimeClient client) {
        this.client = client;
    }
   /**
     * Sends the given text to Bedrock and returns the model's reply.
     */
    public String converse(String inputText) {
        Message message = Message.builder()
                .role(ConversationRole.USER)
                .content(ContentBlock.fromText(inputText))
                .build();
       GuardrailConfiguration guardrailConfiguration = GuardrailConfiguration.builder()
                .guardrailIdentifier(guardrailIdentifier)
                .guardrailVersion(guardrailVersion)
                .build();
        ConverseRequest request = ConverseRequest.builder()
                .modelId(modelId)
                .guardrailConfig(guardrailConfiguration)
                .messages(message)
                .inferenceConfig(cfg -> cfg
                        .maxTokens(500)
                        .temperature(0.5F)
                )
                .build();
        try {
            ConverseResponse response = client.converse(request);
            return response.output().message().content().get(0).text();
        } catch (SdkClientException e) {
            throw new RuntimeException("Failed to invoke Bedrock model: " + e.getMessa
ge(), e);
        }
    }
}
```

With this simple method addition, you are able to extend your application with guardrail protection. Let's test this.

```
curl -X POST localhost:8080/api/bedrock/converse   -H "Content-Type: application/json"   -d '{"inputText":"Give me a patent idea in certification for FAA."}'
```

Output:

```
{"outputText":"Sorry, the model cannot answer this question."}
```

As you can see in the output, your guardrail was able to catch the filtered topic and respond with the previously set response.

Extending and Integrating RAG into Your Application

You will next learn how to integrate RAG into the application by extending it with your source, which is the FAA certification website, as shared at the starting of this chapter. You will create a PDF of the document at https://www.ecfr.gov/current/title-14/chapter-III/subchapter-A/part-400/section-400.2 by clicking the Print/PDF button on the website, as shown in Figure 16-41.

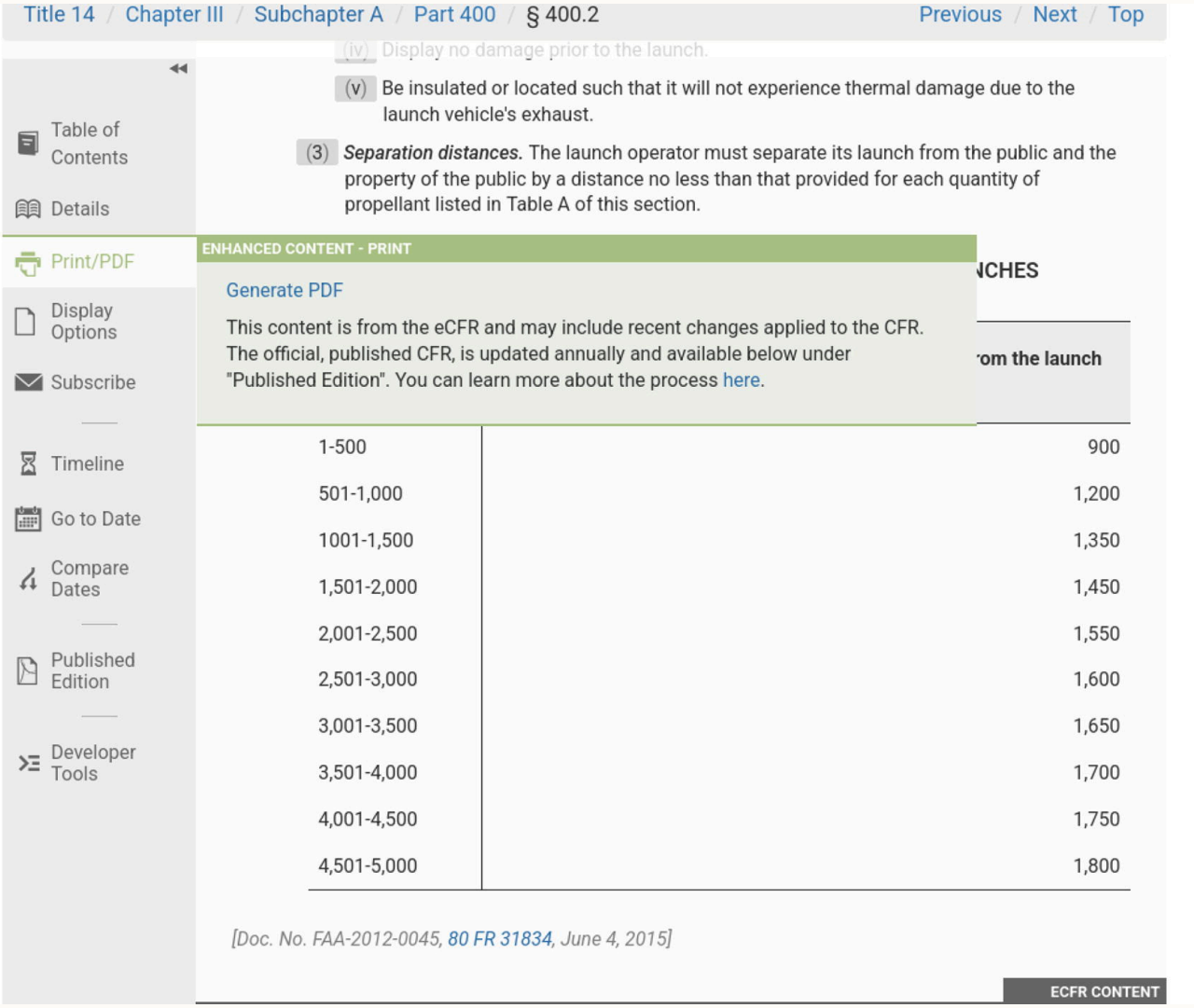


Figure 16-41 PDF generation

Amazon Bedrock abstracts how to implement this in knowledge bases (see Figure 16-42).

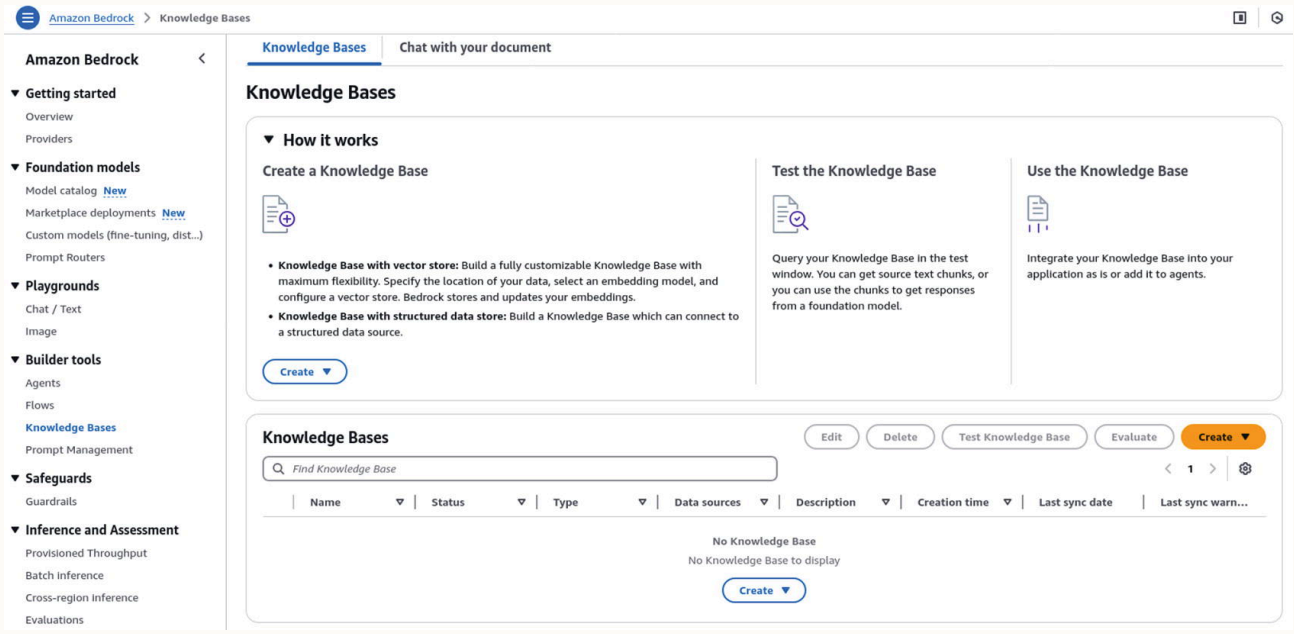


Figure 16-42 AWS Bedrock knowledge bases

Amazon Bedrock knowledge bases let you bring your own data into the same environment as foundation models so you can ask natural-language questions against documents, databases, or other content you control. When you build a vector-store knowledge base, you begin by pointing Bedrock at your unstructured source material—PDFs, text files, web pages, or anything else—and selecting an embedding model along with the storage backend where your vectors will live. Bedrock then reads your data, transforms each piece into high-dimensional embeddings, and keeps those embeddings up-to-date as your content changes. For structured-data knowledge bases, you instead connect Bedrock directly to a database or other tabular store; Bedrock catalogs the schema and ingests the records, turning rows and columns into retrievable objects without manual preprocessing.

Once your knowledge base is built, Bedrock provides a test interface where you can type in queries and immediately see what snippets or records it retrieves. You can inspect these "raw" results to verify that your data is being indexed correctly, or you can feed them straight into a foundation model to observe how the model synthesizes a conversational answer from the retrieved context. This rapid feedback loop helps you tune your embedding choices, storage settings, and query parameters until you're confident in the relevance and accuracy of the results.

When you're ready to go live, you simply call Bedrock's retrieval or chat endpoints from your application—either embedding knowledge-base queries directly into your user interface or wiring them into an agent framework that can orchestrate multiturn dialogues

or complex workflows. In either case, Bedrock handles the heavy lifting of vector similarity search or structured querying behind the scenes, so your code remains clean and focused on delivering a smooth, intelligent experience to your users.

We will proceed with creating a knowledge base using a vector store. Click the Create button and select to use vector store. This leads us to step 1 where you need to provide knowledge base details. Most of the form fields are self-explanatory and filled in for quick startup. Feel free to update any of them to your choice (see Figure 16-43).

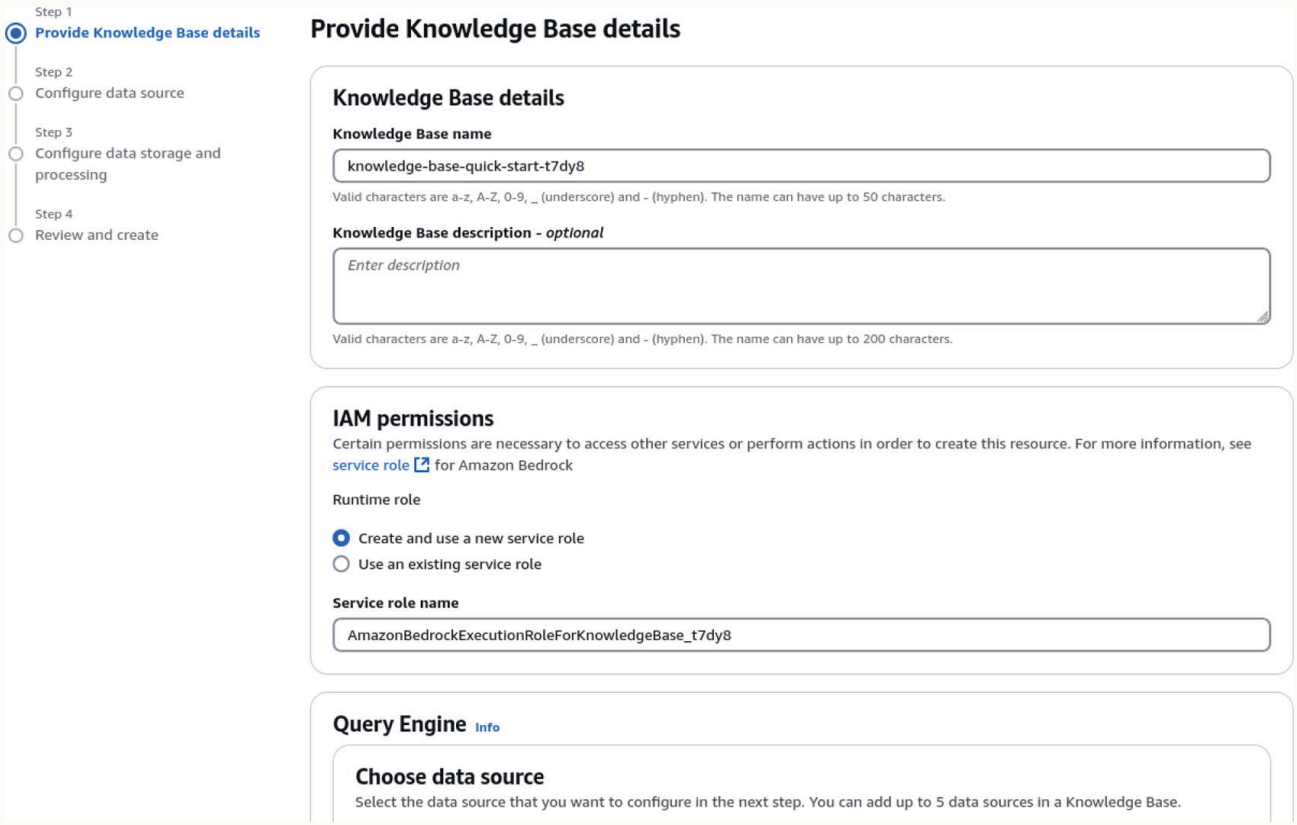


Figure 16-43 Knowledge base details section

Since in this case the page is public, you can use the web crawler as your data source (see Figure 16-44).

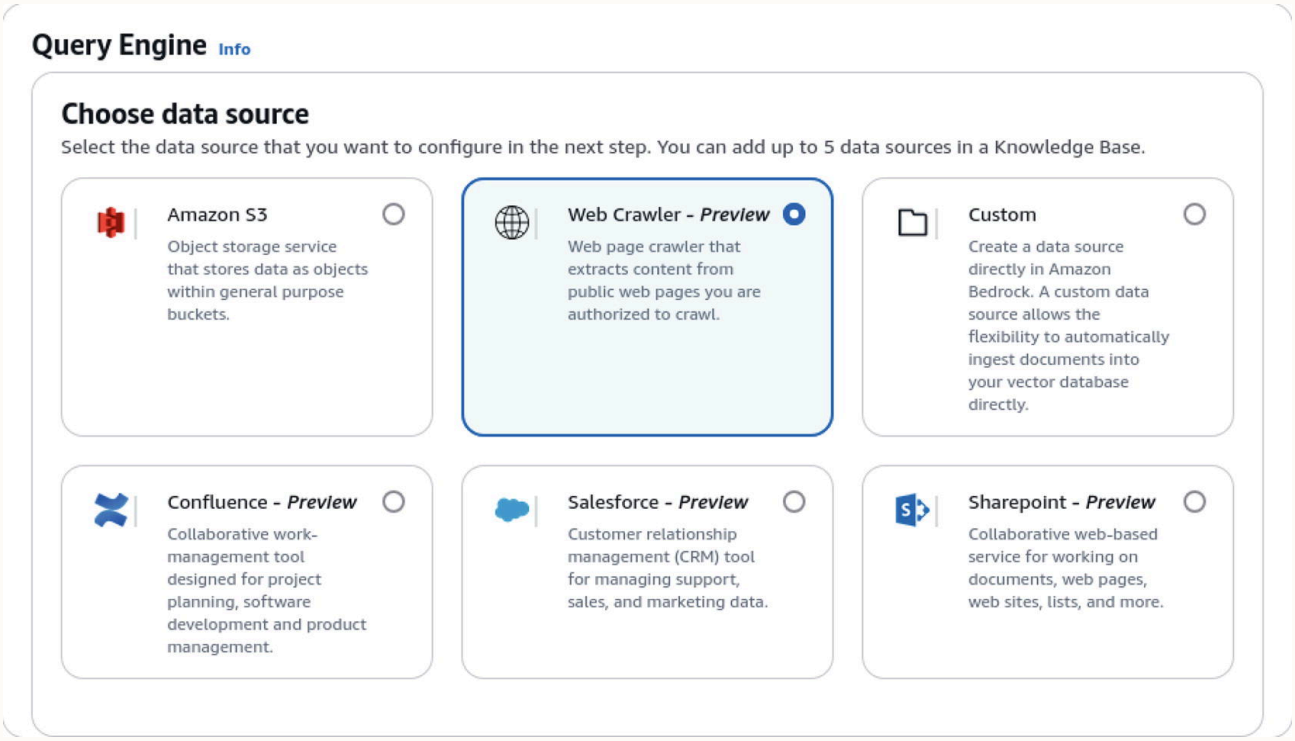


Figure 16-44 Data source choices for knowledge base

In step 2, you will continue to configure the data source where you will need to set the base URL of the website you want as your data source. You can use the URL https://www.ecfr.gov/current/title-14, but for quick setup, you can instead go with only a single subpage in the Source URLs field (see Figure 16-45).

> *"Remember that you must only use the Web Crawler to index your own web pages, or web pages that you have authorization to crawl and must respect robots.txt configurations. The web crawler will respect robots.txt in accordance to the RFC 9309. It's not recommended to crawl large websites, such as wikipedia.org, without filters or scope limits. Crawling large websites will take a very long time to crawl."*

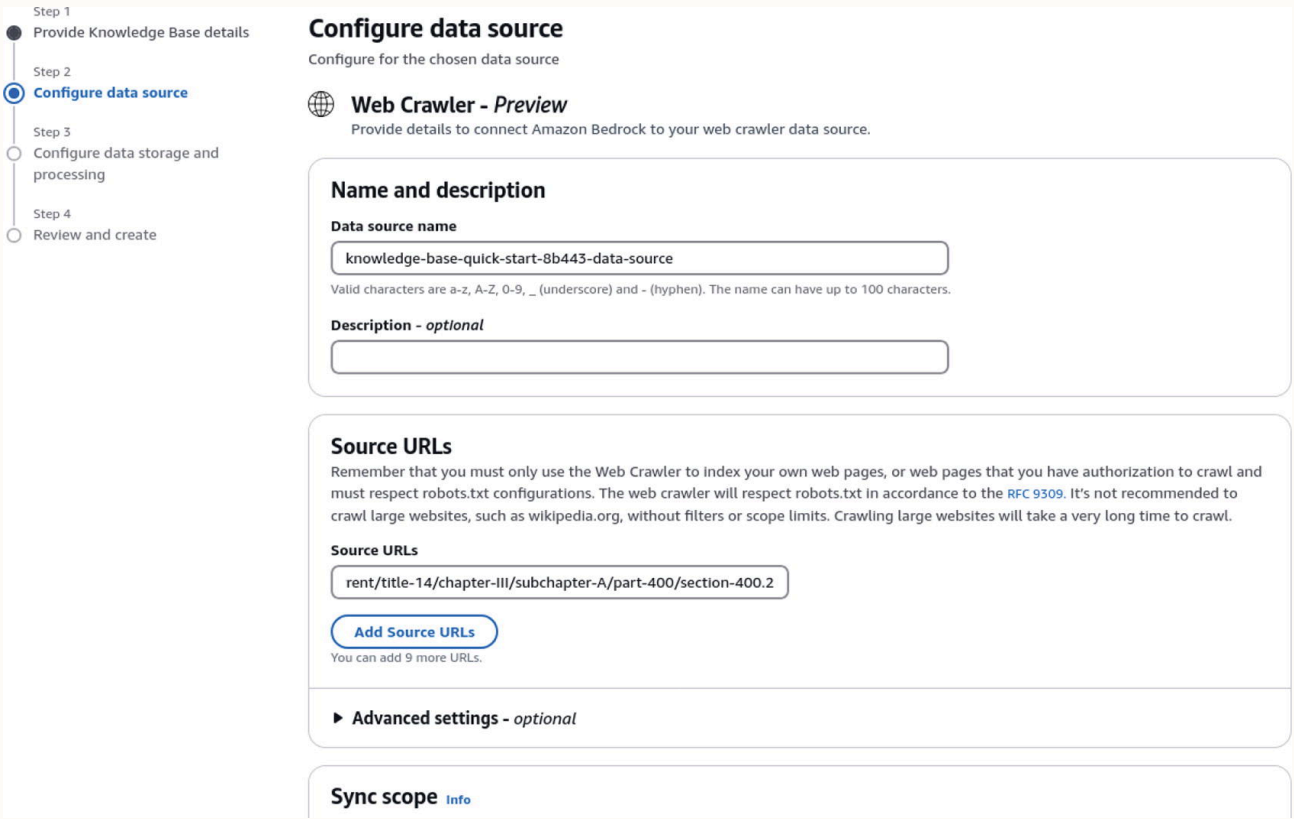


Figure 16-45 Web crawling of data source URLs

We are allowed at most 10 URLs to crawl from.

Then you can select which scope you want to crawl from, such as the same host and initial URL, the same host only, or the same primary domain. For now let's use the default. There are other settings you can set as shown in Figure 16-46 that are self-explanatory.

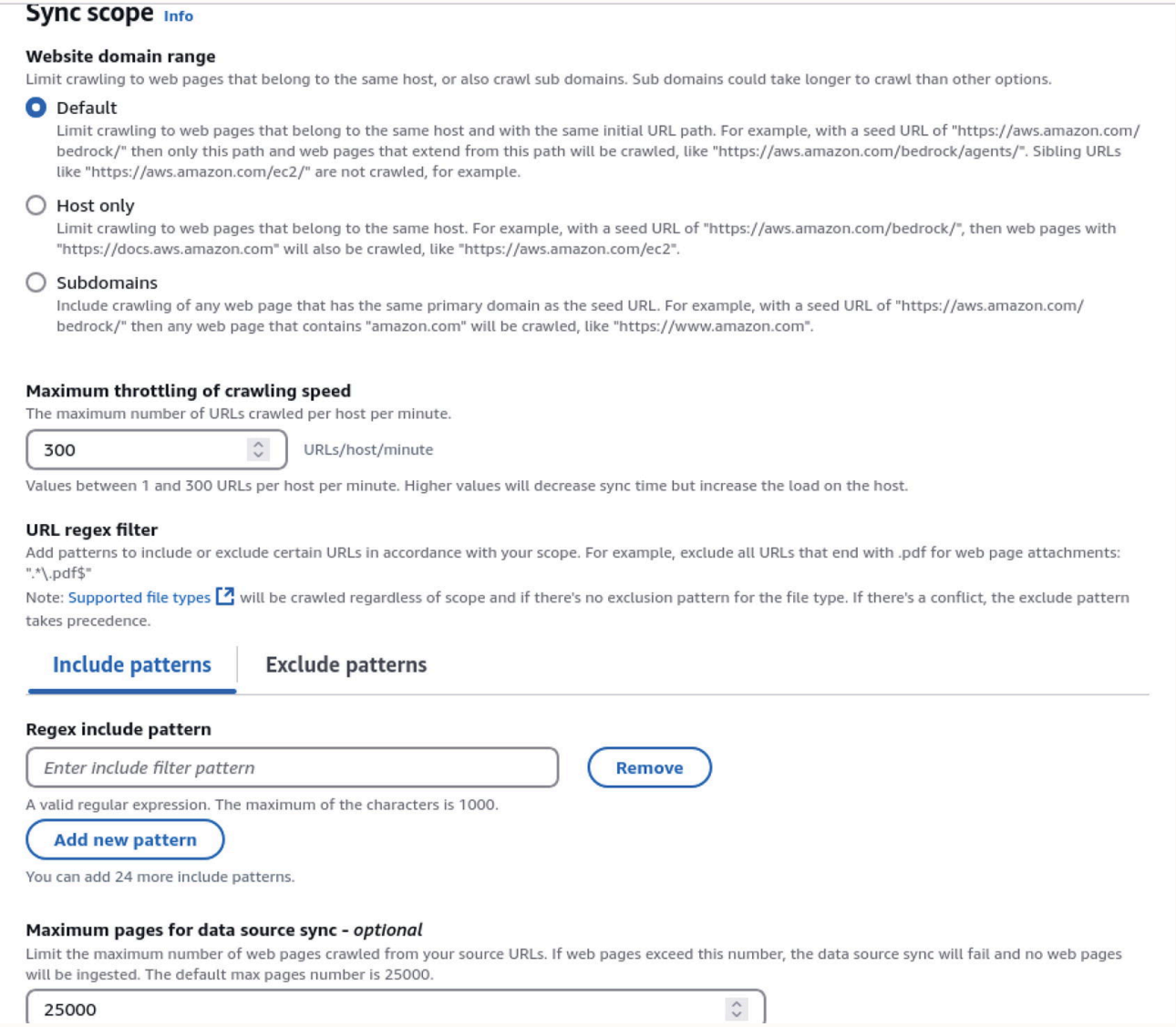


Figure 16-46 Web crawling settings

Since the website does not allow crawling the contents of the website, the crawler fails. I have chosen to use the PDF format, which can be downloaded and then uploaded to S3. Uploaded the downloaded document to S3 and then provide the S3 path to the data source settings. See Figure 16-47.

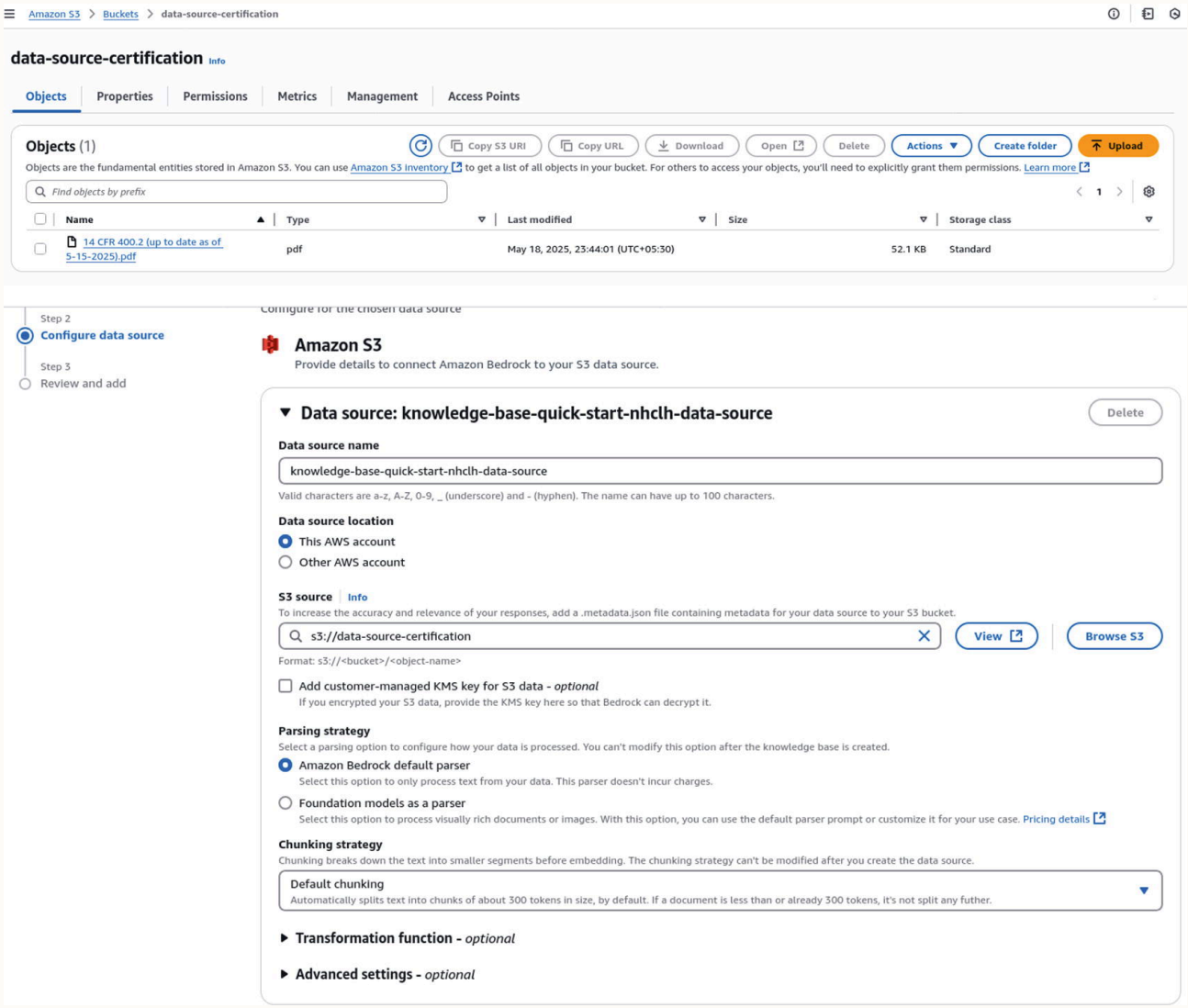


Figure 16-47 S3 data source

The next step is to select the chunking strategy. Go ahead with the default setting, which is fine for this case. You can also use a custom lambda function if you have a custom chunking strategy (see Figure 16-48).

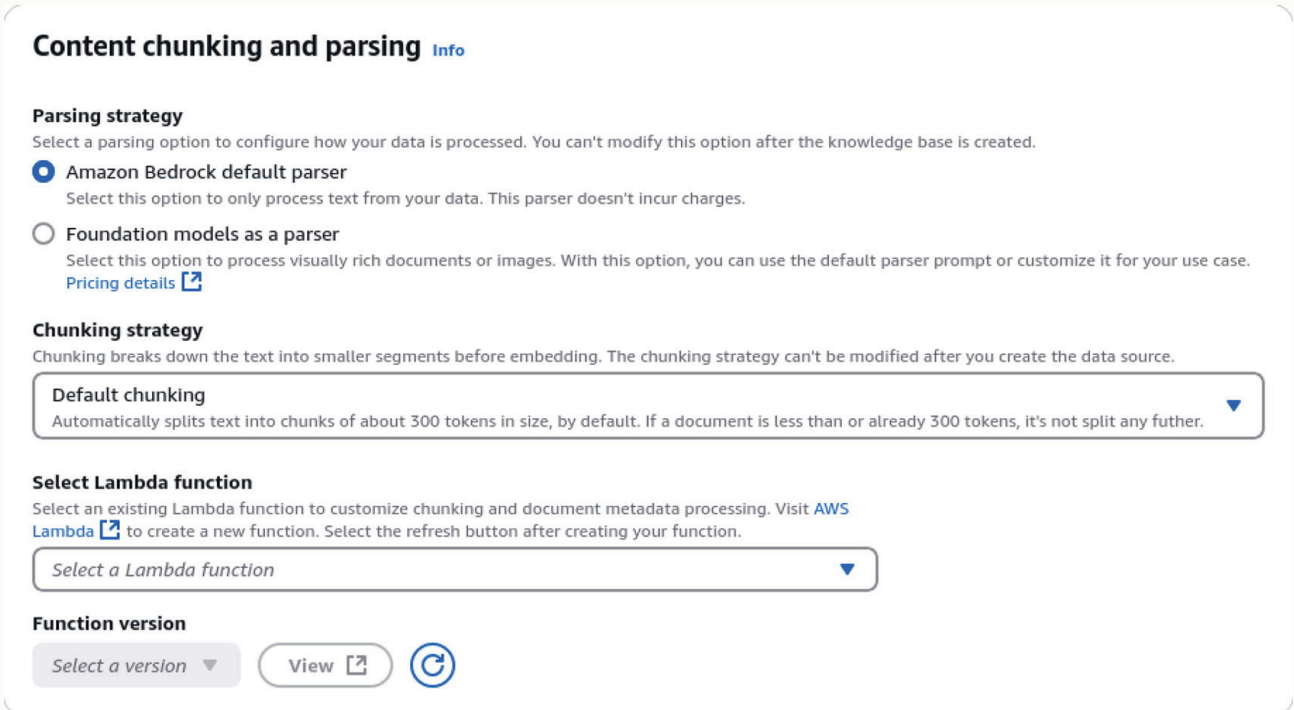


Figure 16-48 Chunking strategy configuration

Step 3 is to configure data storage and processing where you will choose an embedding model from Amazon called Tital Text Embeddings V2 and OpenSearch Serverless as the vector store.

For this step, to actually work later, you have to make sure you have requested access to it for embeddings generation. This can be done similar to what you did for the Nova Micro model (see Figure 16-49).

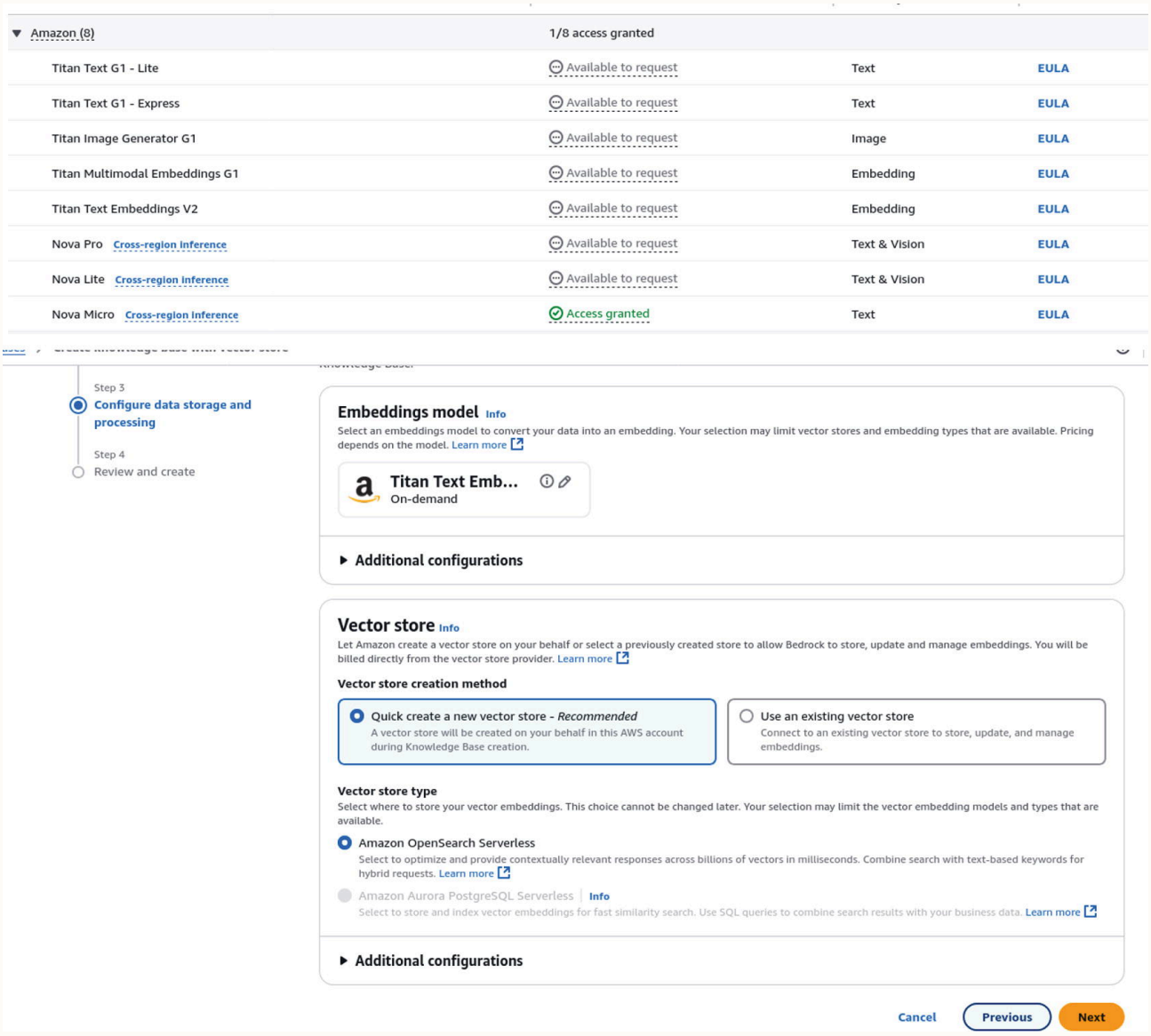


Figure 16-49 Vector store configuration for knowledge base

The final step is to review and create your knowledge base (see Figure 16-50).

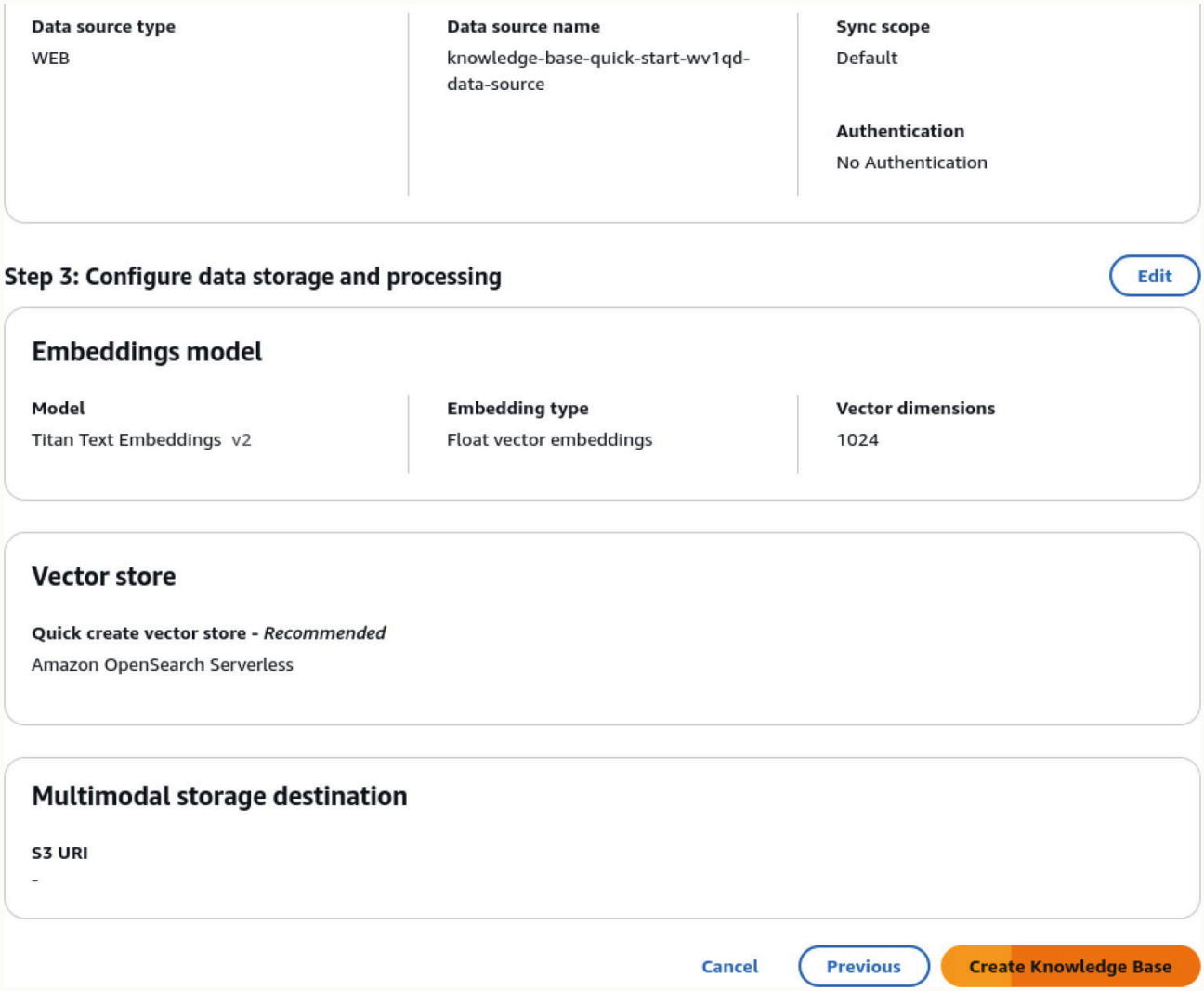


Figure 16-50 Knowledge base overview and create page

It takes some time to complete this step (see Figure 16-51).

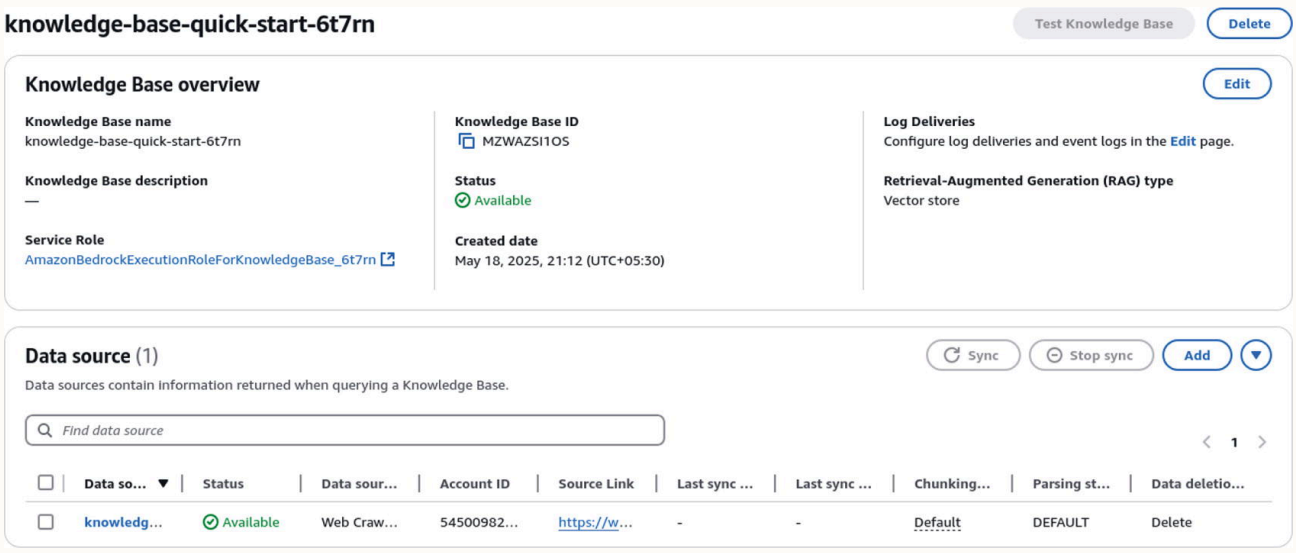


Figure 16-51 Knowledge base listing view

Once complete, you should sync your data source once so that the embeddings are populated in open search for RAG to work. In Figure 16-52, you can see the data source has already been synced. If there are no other issues, the Test Knowledge Base at top right will be enabled for your testing.

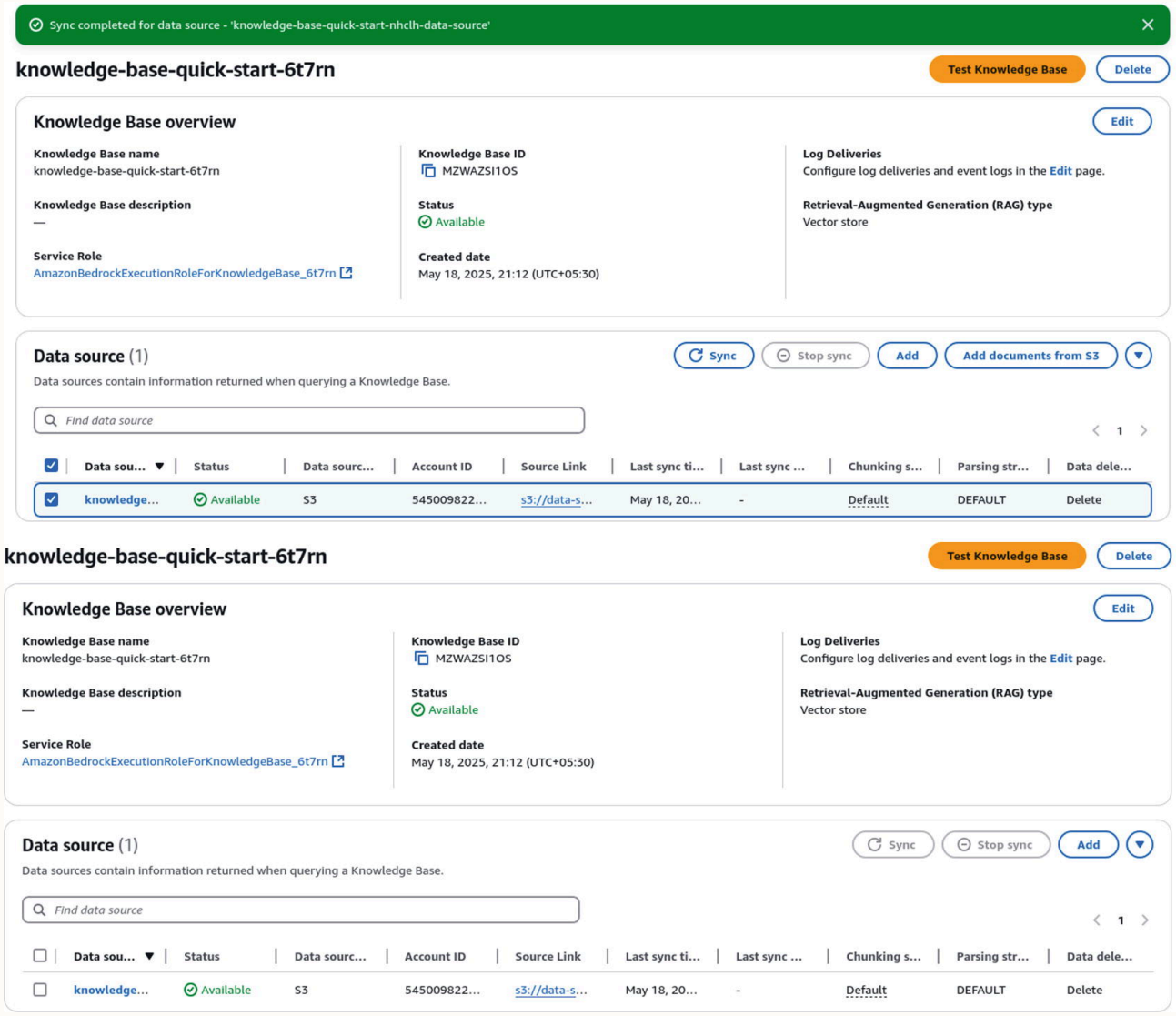


Figure 16-52 Test knowledge base and status views

Now you can go ahead and test your knowledge base and ask it a question to share with the scope for 400.2, which is present in your document. If asked to select a model, select your Nova Micro model for the generation component (see Figure 16-53).

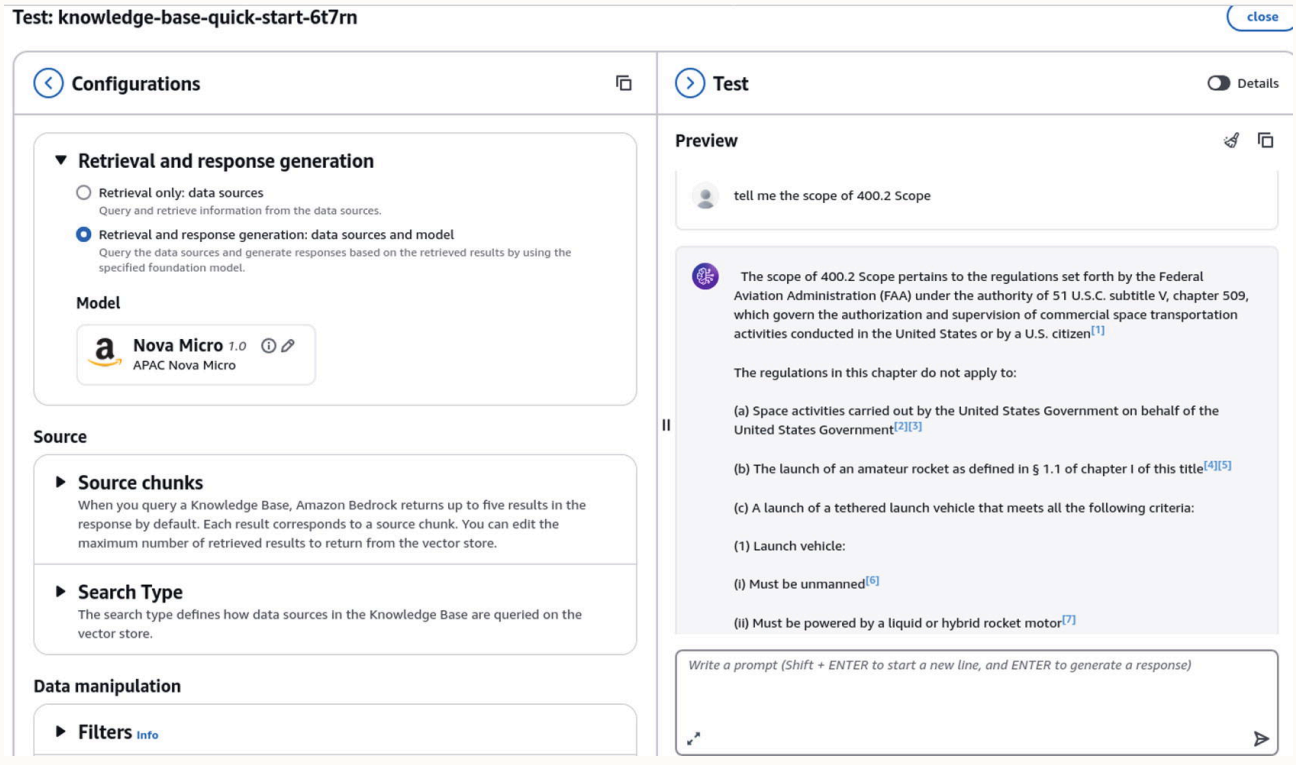


Figure 16-53 Test knowledge base page

Now that you have your knowledge base, let's use it to integrate RAG into your previous application. Since this uses a new dependency called bedrockagentruntime, you will need to include it into your build.gradle file.

```
// https://mvnrepository.com/artifact/software.amazon.awssdk/bedrockagentruntime
implementation 'software.amazon.awssdk:bedrockagentruntime:2.31.45'
```

We will need to set the knowledge base id in the application.properties file to be used later in your service component. This can be fetched from the knowledge base page (see Figure 16-54).

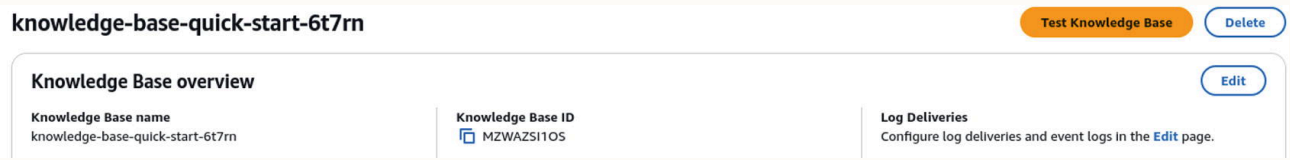


Figure 16-54 The application.properties information for knowledge base config

File: ./src/main/resources/application.properties

```
spring.application.name=demo
aws.region=ap-south-1
aws.bedrock.modelId=arn:aws:bedrock:ap-south-1:xxxxxxxxxxxx:inference-profile/apac.amazon.nova-micro-v1:0
aws.bedrock.guardrailIdentifier=arn:aws:bedrock:ap-south-1:xxxxxxxxxxxx:guardrail/xxxxxxxxxx
aws.bedrock.guardrailVersion=1
aws.bedrock.knowledgeBaseId=XXXXXXXXXX
```

Since this uses the BedrockAgentRuntimeClient for your final request for RAG, you will have to create another bean for this in the configuration file.

File: ./src/main/java/com/example/demo/config/AwsConfig.java

```
package com.example.demo.config;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockagentruntime.BedrockAgentRuntimeClient;
```

```
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
@Configuration
public class AwsConfig {
   @Value("${aws.region}")
   private String awsRegion;
   @Bean
   public BedrockRuntimeClient bedrockRuntimeClient() {
      return BedrockRuntimeClient.builder()
            .credentialsProvider(DefaultCredentialsProvider.create())
            .region(Region.of(awsRegion))
            .build();
   }
   @Bean
   public BedrockAgentRuntimeClient bedrockAgentRuntimeClient() {
      return BedrockAgentRuntimeClient.builder()
            .credentialsProvider(DefaultCredentialsProvider.create())
            .region(Region.of(awsRegion))
            .build();
   }
}
```

This bean will then be injected at runtime in your service component.

```
   private final BedrockService bedrockService;
   public BedrockController(BedrockService bedrockService) {
      this.bedrockService = bedrockService;
   }
```

We will create another method in your service component since the API for this differs from the Converse API you used previously. You will name this retrieve, which takes a similar inputText as an argument. Then you create an object of the RetrieveAndGenerateInput type using the builder pattern, which takes in as a text argument the inputText value.

```
public String retrieve(String inputText) {
   RetrieveAndGenerateInput retrieveAndGenerateInput = RetrieveAndGenerateInput.
builder()
         .text(inputText)
         .build();
```

We will then create the KnowledgeBaseRetrieveAndGenerateConfiguration object, which takes as argument the knowledgeBaseId and the modelId of your Nova Micro model, which is already set.

```
@Value("${aws.bedrock.knowledgeBaseId}")
private String knowledgeBaseId;
. . .
. . .
KnowledgeBaseRetrieveAndGenerateConfiguration knowledgeBaseConfiguration = Kn
owledgeBaseRetrieveAndGenerateConfiguration.builder()
    .knowledgeBaseId(knowledgeBaseId)
    .modelArn(modelId)
    .build();
```

After this, you will need to create the RetrieveAndGenerateConfiguration object that uses the previous knowledgeBaseConfiguration and the type of KNOWLEDGE_BASE value to build the object.

```
RetrieveAndGenerateConfiguration retrieveAndGenerateConfiguration = RetrieveAndG
enerateConfiguration.builder()
    .knowledgeBaseConfiguration(knowledgeBaseConfiguration)
    .type(RetrieveAndGenerateType.KNOWLEDGE_BASE)
    .build();
```

You can finally use this configuration to create the request object for the retrieveAndGenerate method call on the agentClient.

```
RetrieveAndGenerateRequest retrieveAndGenerateRequest = RetrieveAndGenerateRe
quest.builder()
    .input(retrieveAndGenerateInput)
    .retrieveAndGenerateConfiguration(retrieveAndGenerateConfiguration)
    .build();
    try {
        RetrieveAndGenerateResponse retrieveAndGenerateResponse = agentClient
            .retrieveAndGenerate(retrieveAndGenerateRequest);
        return retrieveAndGenerateResponse.output().text();
        } catch (SdkClientException e) {
    throw new RuntimeException("Failed to invoke Bedrock agent model: " + e.getMess
age(), e);
    }
```

Then you will add your controller route to access this new method.

```
package com.example.demo.controller;
import com.example.demo.dto.ConverseRequestDto;
import com.example.demo.dto.ConverseResponseDto;
import com.example.demo.service.BedrockService;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/bedrock")
public class BedrockController {
    private final BedrockService bedrockService;
    public BedrockController(BedrockService bedrockService) {
        this.bedrockService = bedrockService;
    }
    @PostMapping("/converse")
    public ResponseEntity<ConverseResponseDto> converse(@RequestBody ConverseR
equestDto req) {
        String reply = bedrockService.converse(req.getInputText());
        return ResponseEntity.ok(new ConverseResponseDto(reply));
    }
    @PostMapping("/retrieve")
    public ResponseEntity<ConverseResponseDto> retrieve(@RequestBody ConverseRe
questDto req) {
        String reply = bedrockService.retrieve(req.getInputText());
        return ResponseEntity.ok(new ConverseResponseDto(reply));
    }
}
```

Now your RAG has been set up. Let's boot up your application and test it with the same query as you tested in the playground.

```
curl -X POST localhost:8080/api/bedrock/retrieve  -H "Content-Type: application/jso
n"   -d '{"inputText":"Tell me the scope of 400.2."}'
```

Output:


```
{"outputText":"The scope of 400.2 pertains to the regulations under which commercia
l space transportation activities are authorized and supervised in the United States. Sp
ecifically, the regulations in this chapter apply to commercial space transportation acti
vities conducted in the United States or by a U.S. citizen, but do not cover:\n\n(a) Spac
e activities carried out by the United States Government on behalf of the United States
Government.\n\n(b) The launch of an amateur rocket as defined in § 1.1 of chapter I of
this title.\n\n(c) A launch of a tethered launch vehicle that meets all the following crit
eria:\n\n(1) Launch vehicle. The launch vehicle must—\n\n(i) Be unmanned.\n\n(ii) Be
powered by a liquid or hybrid rocket motor.\n\n(iii) Not use any of the toxic propellant
s of Table I417-2 and Table I417-3 in Appendix I of part 417 of this chapter.\n\n(iv) Carr
y no more than 5,000 pounds of propellant.\n\n(2) Tether system. The tether system
must—\n\n(i) Not yield or fail under—\n\n(A) The maximum dynamic load on the syste
m; or\n\n(B) A load equivalent to two times the maximum potential engine thrust.\n\n
```

(ii) Have a minimum safety factor of 3.0 for yield stress and 5.0 for ultimate stress.\n \n(iii) Constrain the launch vehicle within 75 feet above ground level as measured from the ground to the attachment point of the vehicle to the tether.\n\n(iv) Display no da mage prior to the launch.\n\nAdditionally, the launch operator must separate its launc h from the public and the property of the public by a distance no less than that provid ed for each quantity of propellant listed in Table A of this section. The separation dista nces are as follows:\n\n- 1-500 lbs: 900 ft\n- 501-1,000 lbs: 1,200 ft\n- 1,001-1,500 lbs: 1,350 ft\n- 1,501-2,000 lbs: 1,450 ft\n- 2,001-2,500 lbs: 1,550 ft\n- 2,501-3,000 lbs: 1,600 ft\n- 3,001-3,500 lbs: 1,650 ft\n- 3,501-4,000 lbs: 1,700 ft\n- 4,001-4,50 0 lbs: 1,750 ft\n- 4,501-5,000 lbs: 1,800 ft."}

You can compare the output to the text in your document and find the relevance there. As you can see, it's an interesting experience bringing together all the tooling in Amazon Bedrock in order to plug it in and build your generative application.

NoteRemember to delete all the resources in AWS created for this chapter so as not to incur additional unknown costs.

Reengineering Your Application Using Spring AI Integration with Amazon Bedrock

You have seen how to build an application using the SDK from AWS. In this section, you will explore how to build a similar application but leveraging a framework like Spring AI to leverage the same resource infra in AWS in the backend. The advantage of this is that Spring AI uses some similar APIs to interact with different LLMs whether they are hosted in AWS or in OpenAI. This brings common ground of learning regardless of which LLM or infra provider you are working with. So the goal here is to interact with an LLM in AWS through Spring AI.

You will see how to build a simple application to send prompt queries to AWS and return responses to users, as shown in Figure 16-55.

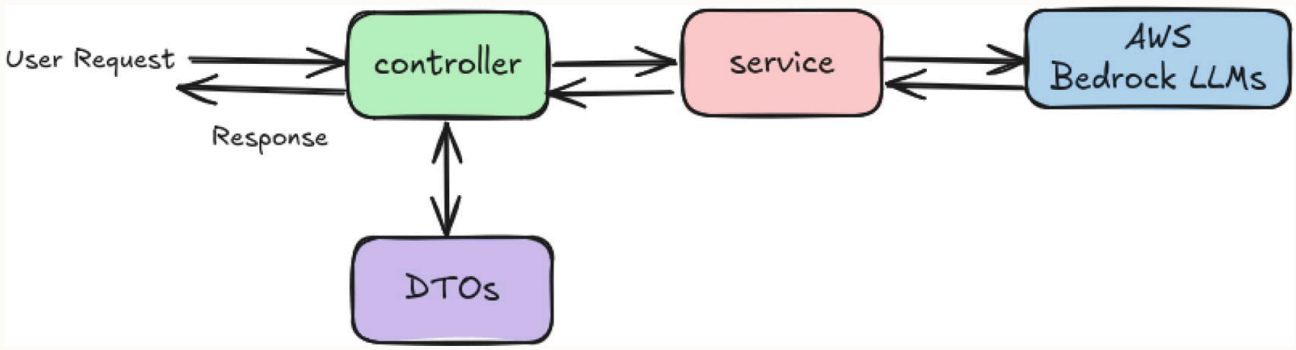


Figure 16-55 Application architecture and control flow

Let's start by visiting the Spring Initializr website and downloading an artifact with dependencies list, as shown in Figure 16-56.

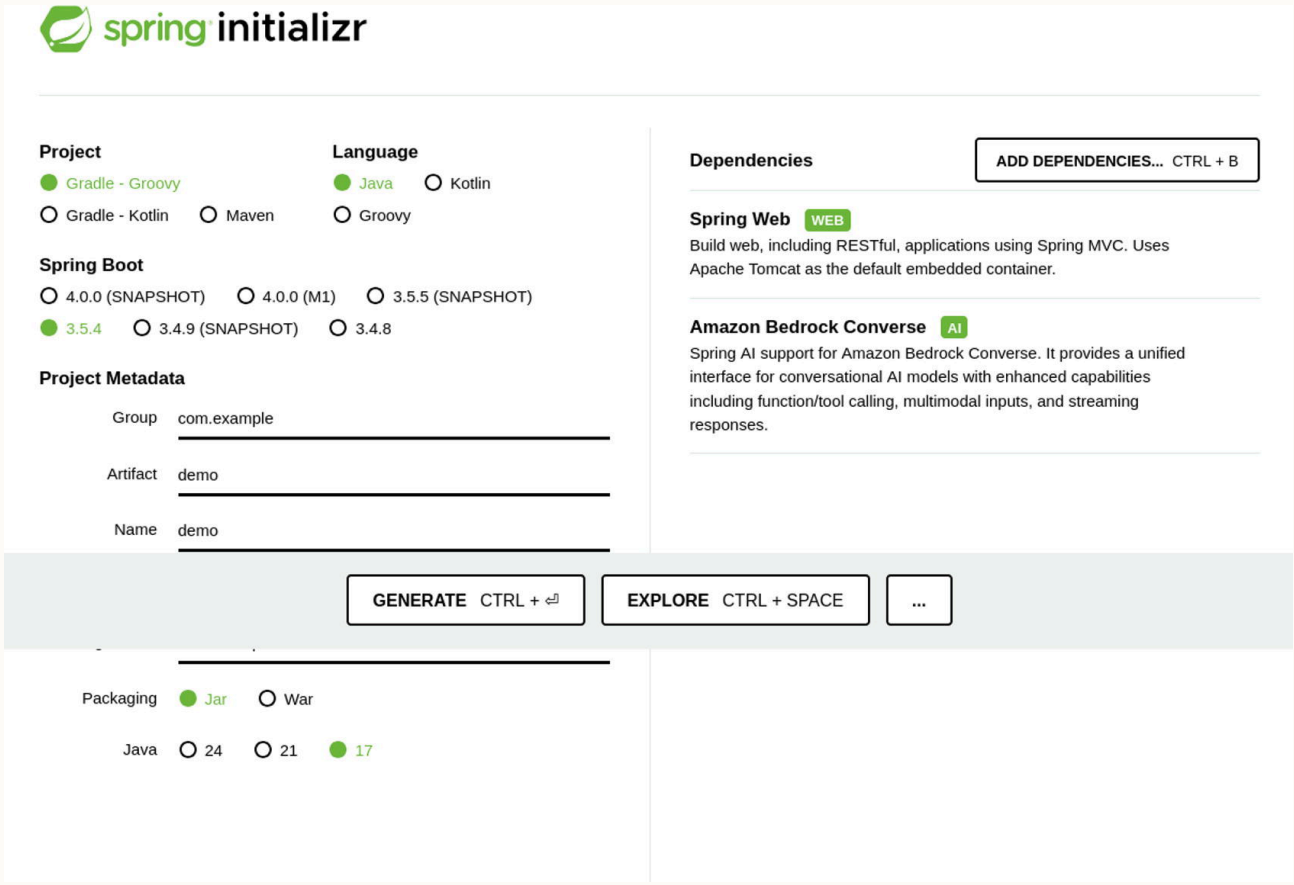


Figure 16-56 Spring Initializr setup

Let's download the artifact, unzip it at the location of your choice, and open it with the IDE of your choice. Please make sure you are able to boot it up to see the familiar Spring Boot welcome message.

For Spring AI to work with AWS Bedrock, it needs a few properties to be set in the application.properties file, as shown here:

File: ./src/main/resources/application.properties

```
spring.application.name=aws-bedrock-spring-ai
spring.ai.bedrock.aws.region=ap-south-1
spring.ai.bedrock.aws.timeout=5m
spring.ai.bedrock.converse.chat.options.model=apac.amazon.nova-micro-v1:0
spring.ai.bedrock.aws.access-key=
spring.ai.bedrock.aws.secret-key=
```

As you have seen, you need to make sure to share the region where you have enabled access to a particular LLM of your choice. You will want to use the model apac.amazon.nova-micro-v1:0 that you enabled previously. This value can be retrieved from the cross-region inference section, as shown in Figure 16-57.

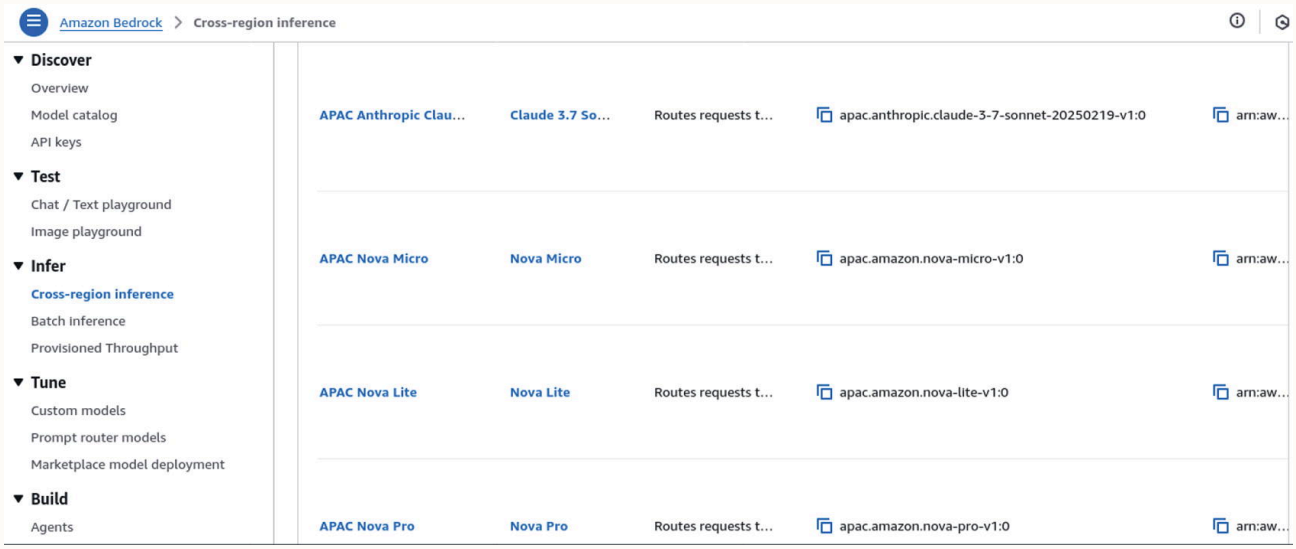


Figure 16-57 Inference page listing for LLM models

Here I have assumed five minutes as the connection timeout with AWS Bedrock. For the access key and secret key, they can be generated from AWS IAM. For this particular application, I will be using one that has administrator access for quick prototyping. But make sure to use a role that has fine-grained access to AWS Bedrock based on the tasks it is supposed to do.

There are other properties specific to the Spring AI AWS converse, as shown next. In this section, stick to the default values.

```
spring.ai.bedrock.converse.chat.options.temperature
Controls the randomness of the output. Values can range over [0.0,1.0]
spring.ai.bedrock.converse.chat.options.top-p
The maximum cumulative probability of tokens to consider when sampling.
spring.ai.bedrock.converse.chat.options.top-k
Number of token choices for generating the next token.
spring.ai.bedrock.converse.chat.options.max-tokens
Maximum number of tokens in the generated response.
```

These values can be overridden at runtime based on need using the ToolCallingChatOptions API. You will create the service component first so you can receive a prompt from the controller and make a request to the AWS Bedrock LLM.

So, start building a service component named ChatService.java. You will define a public constructor with a ChatModel API instance argument, which will be injected by Spring AI at runtime.

```
public ChatService(ChatModel chatModel) {
  this.chatModel = chatModel;
}
```

Then you can use this chatModel with the ChatClient API to interact with your Bedrock model, as shown next. This method takes as argument the text prompt from the user and makes a request with the call() method call and finally to content() to fetch the text response.

```
public String chat(String prompt) {
  return ChatClient
        .create(this.chatModel)
        .prompt(prompt)
        .call()
        .content();
}
```

File: ./src/main/java/com/example/aws_bedrock_spring_ai/service/ChatService.java

```
package com.example.aws_bedrock_spring_ai.service;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.stereotype.Service;
@Service
public class ChatService {
  private final ChatModel chatModel;
  public ChatService(ChatModel chatModel) {
     this.chatModel = chatModel;
  }
  public String chat(String prompt) {
     return ChatClient
           .create(this.chatModel)
           .prompt(prompt)
           .call()
           .content();
  }
}
```

Next, you will create a few DTO objects to map your controller request and response objects, as shown here:

File: ./src/main/java/com/example/aws_bedrock_spring_ai/dto/ChatRequestDto.java

```
package com.example.aws_bedrock_spring_ai.dto;
import jakarta.validation.constraints.NotBlank;
public class ChatRequestDto {
  @NotBlank
```

```
  private String prompt;
  public String getPrompt()          { return prompt; }
  public void setPrompt(String prompt) { this.prompt = prompt; }
}
```

File: /media/user/persistence/aws-bedrock-spring-ai/src/main/java/com/example/aws_bedrock_spring_ai/dto/ChatResponseDto.java

```
package com.example.aws_bedrock_spring_ai.dto;
public class ChatResponseDto {
  private final String content;
  public ChatResponseDto(String content) {
     this.content = content;
  }
  public String getContent() {
     return content;
  }
}
```

These DTO objects are self-explanatory, so I will not cover them in depth here. The final step is to use the service in your controller class to close the request response cycle. You will create a controller class called ChatController.java with the following content:

File: ./src/main/java/com/example/aws_bedrock_spring_ai/controller/ChatController.java

```
package com.example.aws_bedrock_spring_ai.controller;
import com.example.aws_bedrock_spring_ai.dto.ChatRequestDto;
import com.example.aws_bedrock_spring_ai.dto.ChatResponseDto;
import com.example.aws_bedrock_spring_ai.service.ChatService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import javax.validation.Valid;
@RestController
@RequestMapping("/api/v1/chat")
public class ChatController {
  private final ChatService chatService;
  public ChatController(ChatService chatService) {
     this.chatService = chatService;
  }
  @PostMapping
  public ResponseEntity<ChatResponseDto> chat(@RequestBody @Valid ChatRequest
Dto body) {
     String reply = chatService.chat(body.getPrompt());
```

```
        return ResponseEntity.ok(new ChatResponseDto(reply));
    }
}
```

You have autowired the ChatService inside your constructor and used it inside your chat method to get a response to the user prompt query. Let's boot up your application and make a curl request to see it in action.

```
curl -X POST http://localhost:8080/api/v1/chat -H "Content-Type: application/json" -
d '{"prompt": "Tell me a joke"}'
{"content":"Sure, here's a light-hearted joke for you:\n\nWhy did the tomato turn red?
\n\nBecause it saw the salad dressing!\n\nHope that brought a smile to your face!"}
```

In this section, you learned about how to use Spring AI APIs to connect with AWS Bedrock LLMs, which is just the beginning. There are more features like tool calling and RAG that are available for use and exploration. If you now want to change it to OpenAI, you can simply change some of the application properties, and it works without any major changes. This is where the flexibility and simplicity of Spring AI API interfaces come into effect. These same steps could be used with other cloud vendors like GCP and Azure.

Generative AI in GCP

Google Cloud has been making some big strides in my opinion toward developing both state-of-the-art proprietary and open-source models. It has SDKs available for Gen AI in four major languages: Java, Python, Go, and Node.js. In this section, you will explore the Java SDK. Please make sure you have a Google account or register for a three-month subscription, which they are kind enough to give credits for.

GCP groups Gen AI together with its Vertex AI platform for machine learning. When you visit Vertex AI page, you will see Model Garden in the Tools section, as shown in Figure 16-58. In this section, you will see many popular state-of-the-art models that can be used with GCP's SDK, as shown in Figure 16-59.

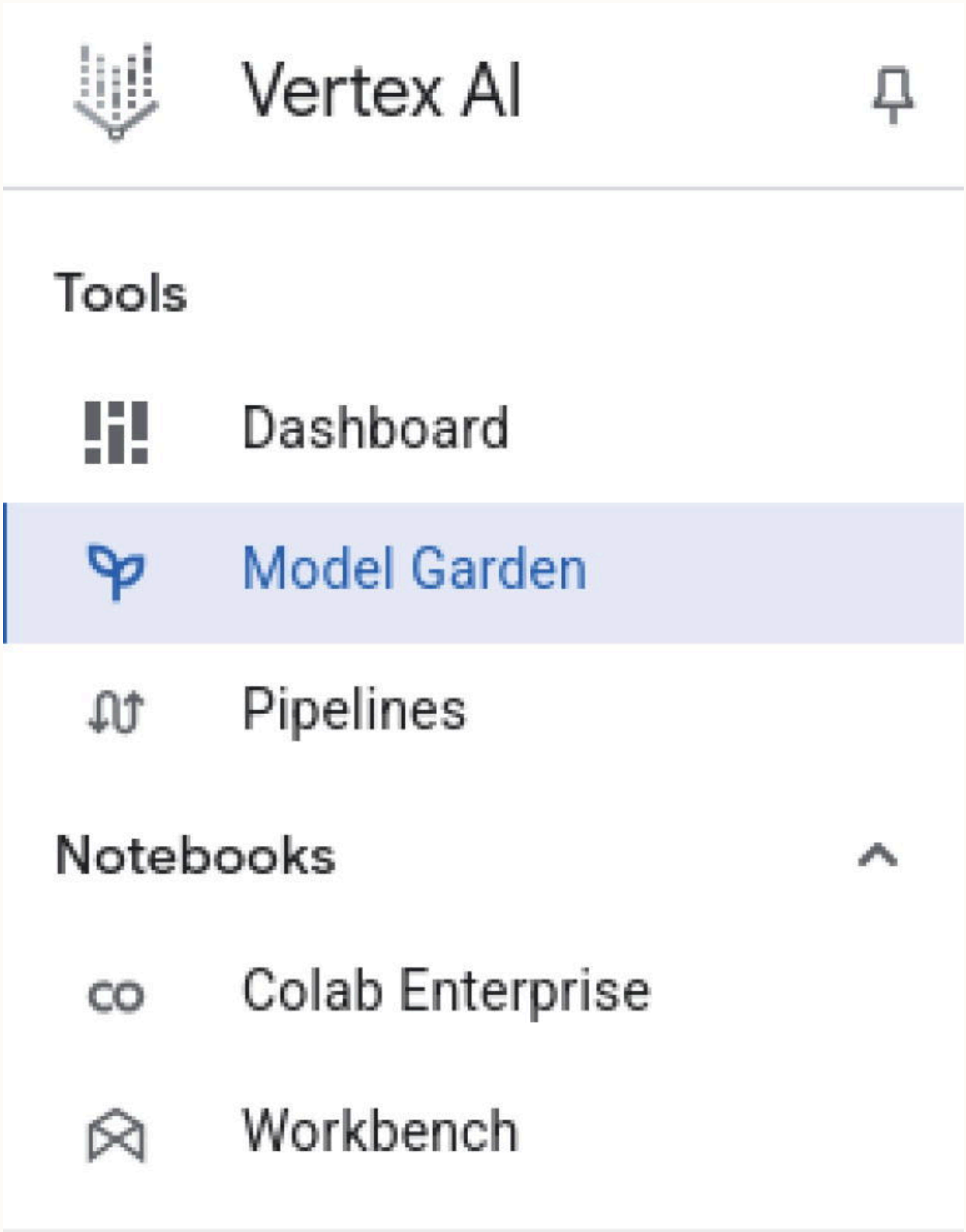


Figure 16-58 Tools section in Vertex AI

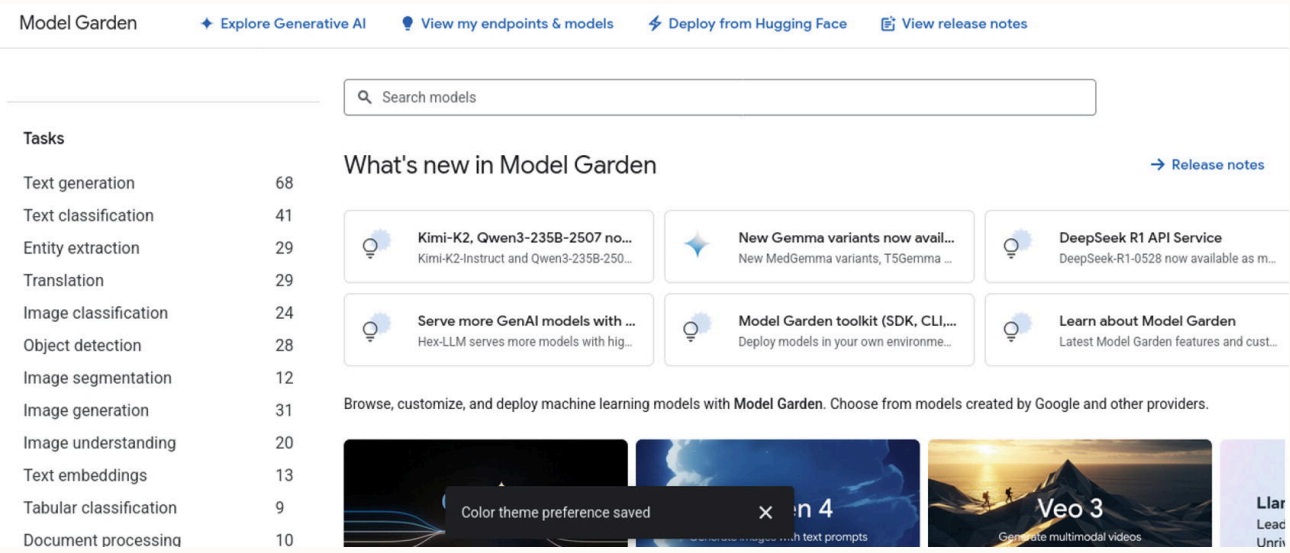


Figure 16-59 Model Garden in Vertex AI

In the sidebar you'll see Vertex AI Studio, where you will find many of the offerings relevant to prompt engineering, prompt management, tuning a LLM, etc. You can explore these at your leisure. Here, we will focus on Create Prompt, which will let you play with prompts including multimodal prompts with different models (see Figure 16-60).

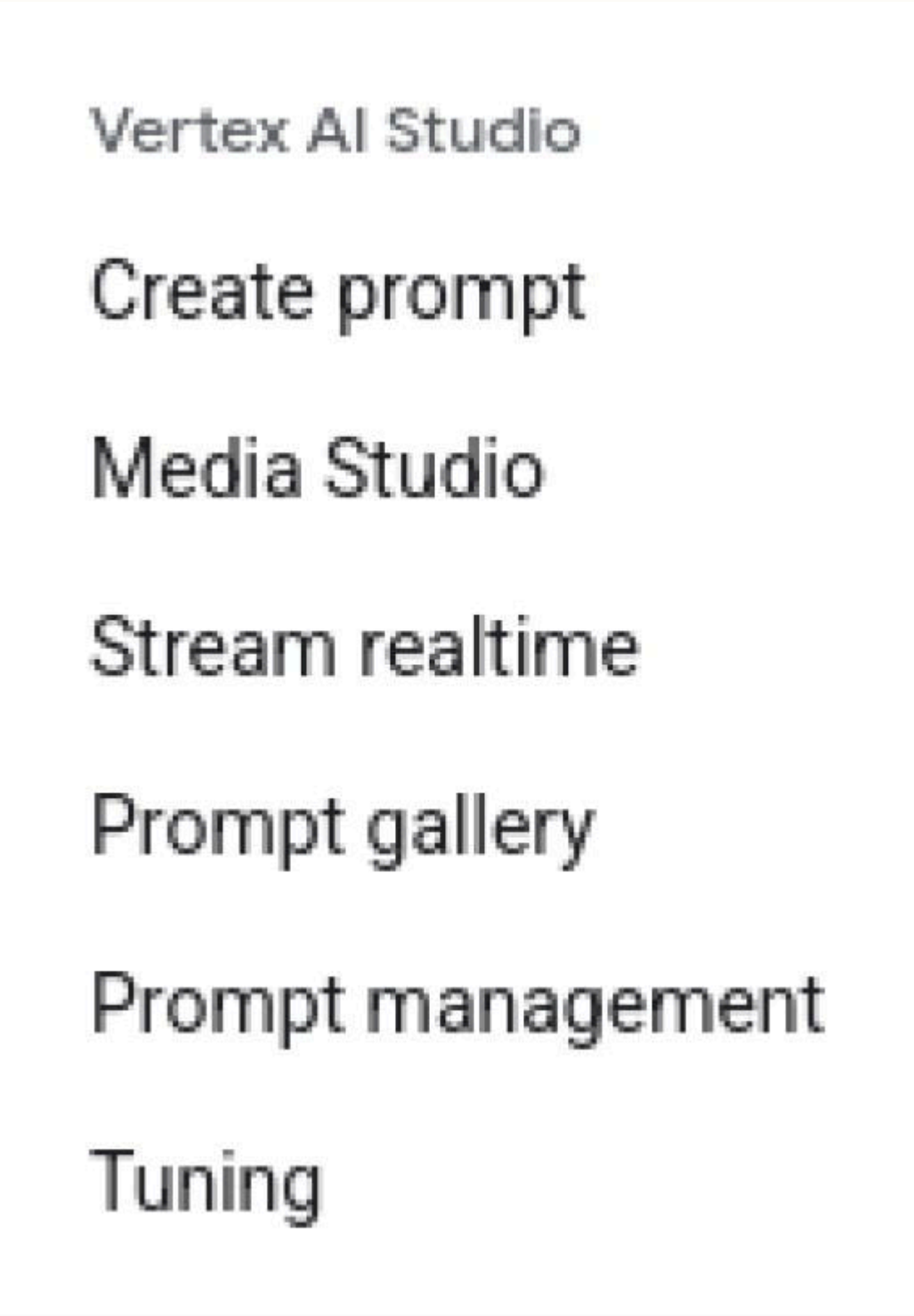


Figure 16-60 Vertex AI Studio services

As you can see in Figure 16-61, you can provide a prompt as well as attach different media content like images, audio, etc. Let's try a prompt to ask it for a joke and see its response. On the right, you can change and try the many models it provides based on your use case and play with the configuration settings for the LLM like temperature, etc.

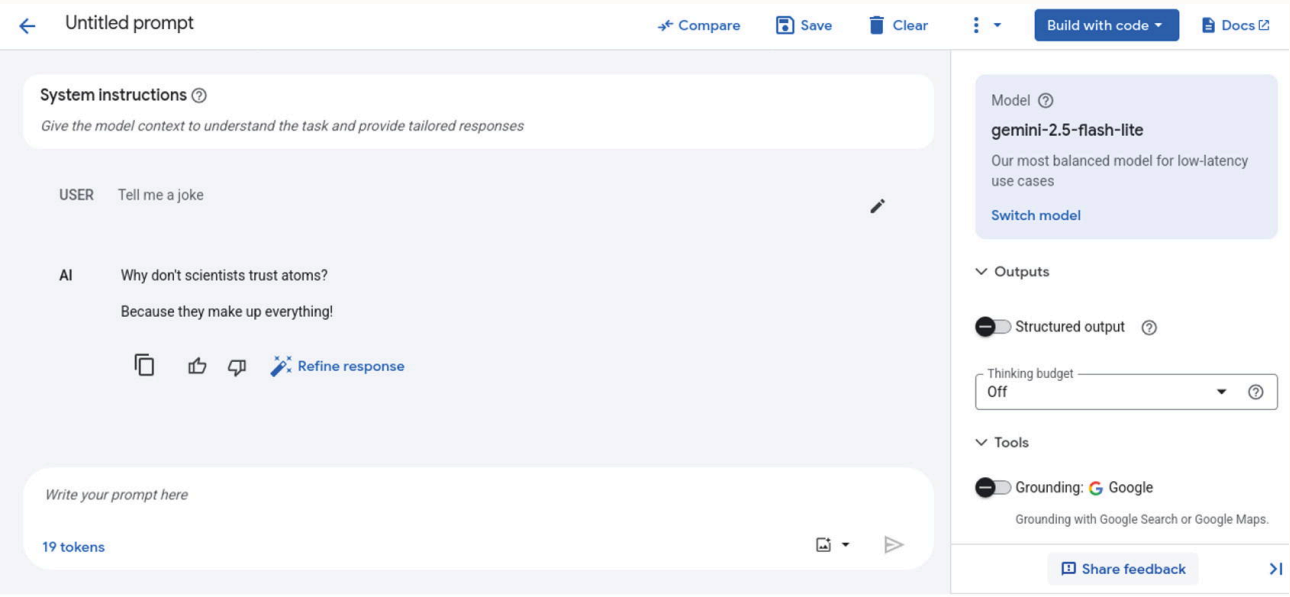


Figure 16-61 Create Prompt page

One of the cool features is the "Build with code" button at the top right; when you click this button, you will see the code used to generate this output. This makes it simple to try and then deploy in code. It currently supports many languages, as shown in Figure 16-62.

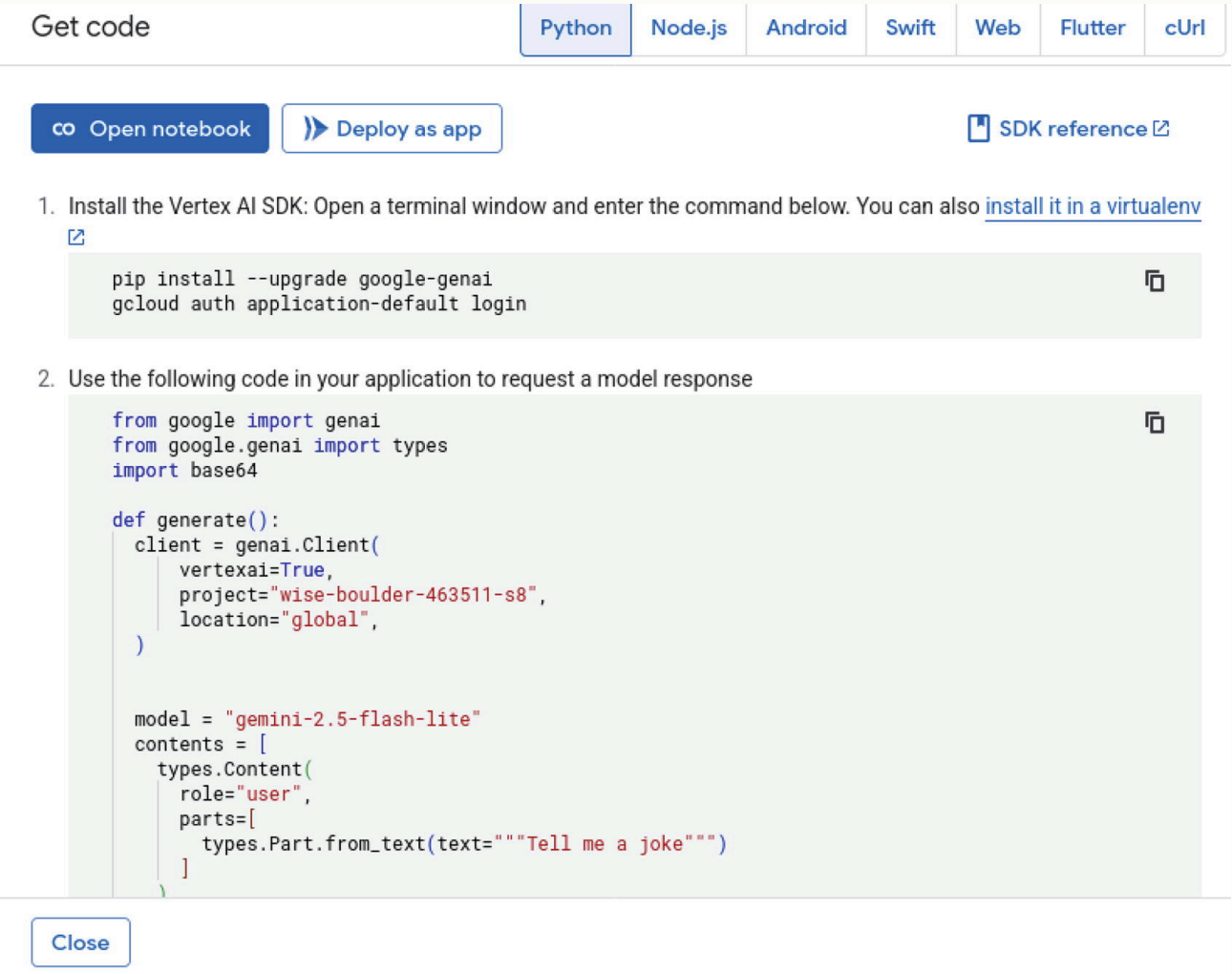


Figure 16-62 Build with code feature

Farther down in the Vertex AI sidebar, you will see some more advanced tools that are focused on agentic and RAG development (Figure 16-63). I will not cover these

features in this section.

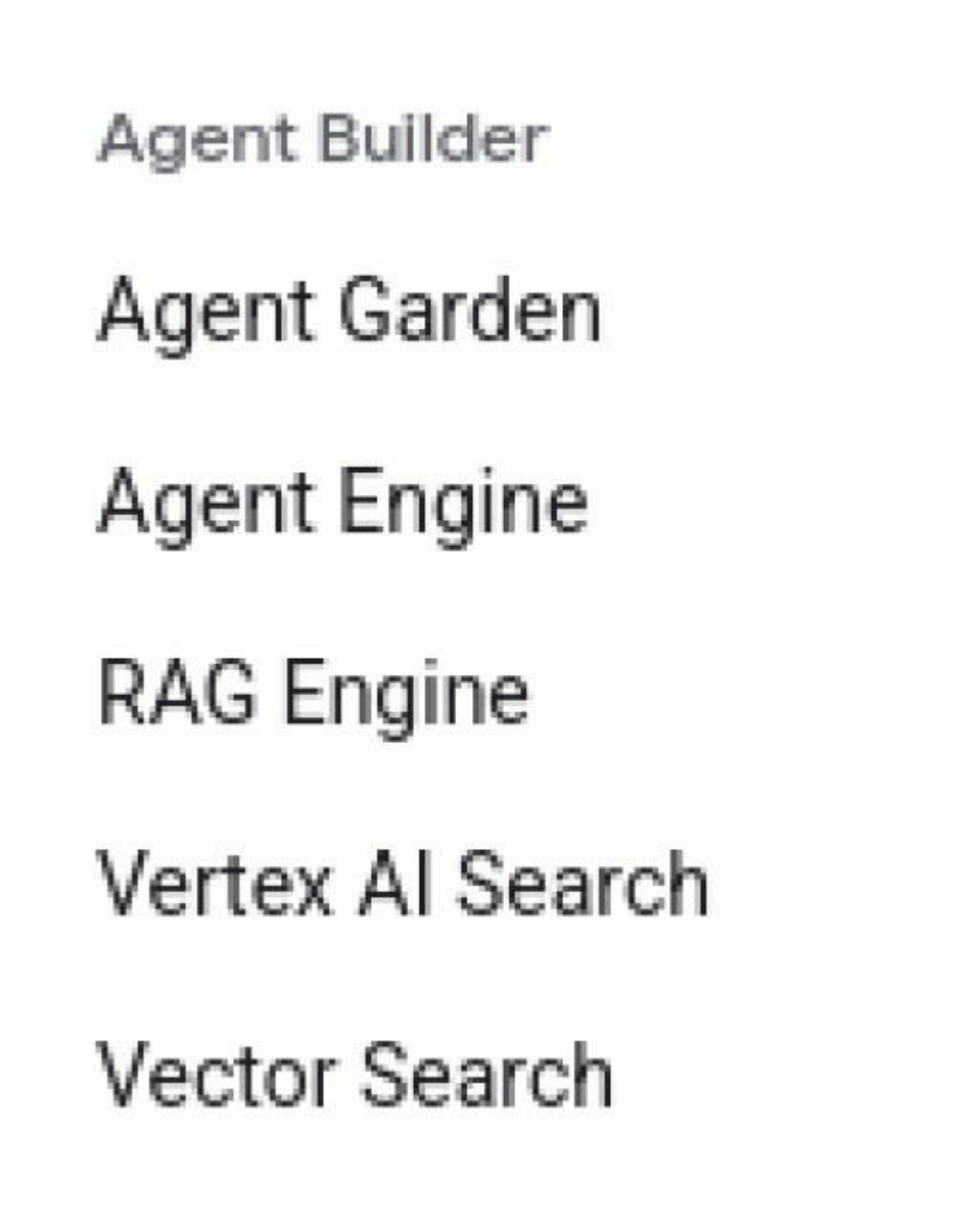


Figure 16-63 Agent builder services

Now that you have had a brief look at the Gen AI features of GCP, let's get our hands dirty with some code and application building. You will use the Java SDK of GCP for Gen AI to build a simple application that prompts for a joke.

You can review the Gen AI SDK documentation at **https://cloud.google.com/vertex-ai/generative-ai/docs/sdks/overview**.

Let's initialize a Spring Boot application using the Spring Initializr website, as shown in Figure 16-64.

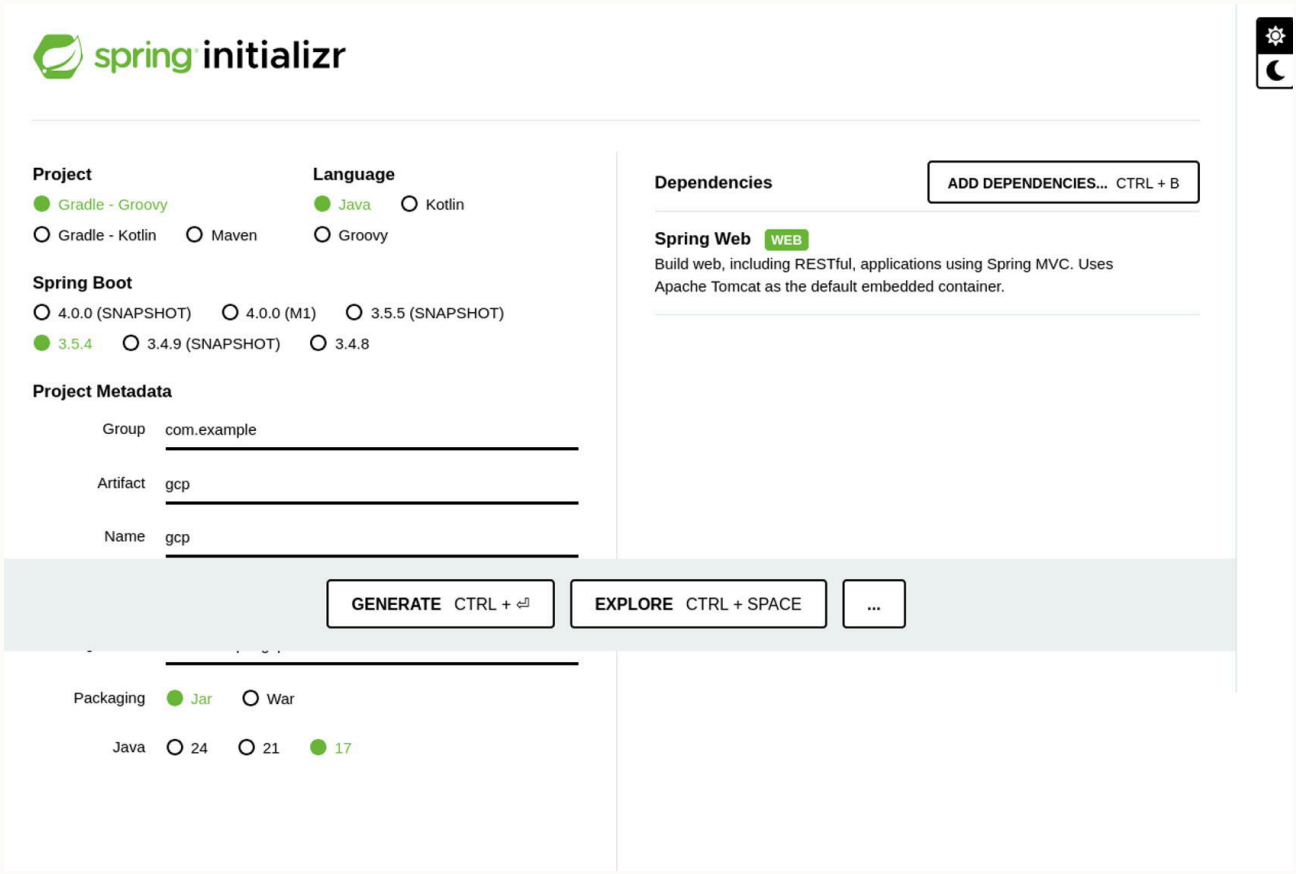


Figure 16-64 Spring Initializr for the base setup of GCP Gen AI application

Download and extract the zip file at the location of your choice and open it with your favorite IDE of choice. When you boot it up, you should see the familiar Spring Boot welcome start output, as shown in the following output:

```
2025-07-29T17:36:51.852Z  INFO 36012 --- [gcp] [main] com.example.gcp.GcpApplic
ation          : Started GcpApplication in 2.653 seconds (process running for 3.838)
```

Next, to work with Gen AI SDK of Google, you will need to include the Gradle package definition for Gen AI in your build.gradle file, as shown here:

```
// https://mvnrepository.com/artifact/com.google.genai/google-genai
implementation("com.google.genai:google-genai:1.10.0")
```

File: build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.4'
    id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
```

```
    toolchain {
      languageVersion = JavaLanguageVersion.of(17)
    }
  }
  repositories {
    mavenCentral()
  }
  dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'com.google.genai:google-genai:1.10.0'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
  }
  tasks.named('test') {
    useJUnitPlatform()
  }
```

The different components you will be creating to build your application are shown in Figure 16-65.

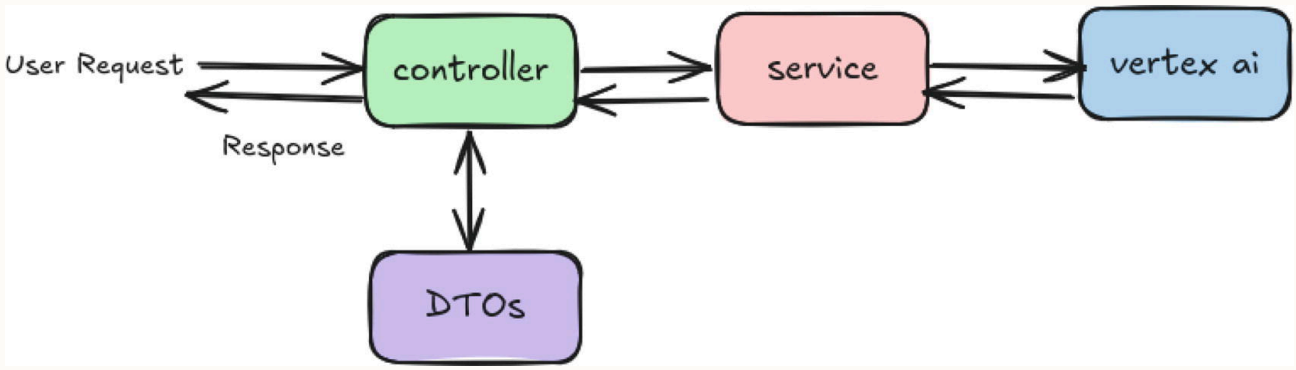


Figure 16-65 Application architecture and control flow

Let's start with the service component named GenAiService.java in a package called service. Inside the service class, define a constructor and initialize the Gen AI client, as shown here:

```
  private final Client client;
  public GenAiService() {
    // Default ctor picks up env vars
    this.client = new Client();
  }
```

Next, define a public method called generate, which takes two string parameters, called prompt and modelId, to generate the content response. The modelId defaults to the gemini-2.0-flash-001 model. You can choose any model of your choice. Inside the method, you make a call to the models.generateContent method of the client API by

passing it the modelId as the first parameter argument and the prompt text as the second. The third argument is the configuration argument, which you can leave as null for this example, but you can try pass temperature and other configuration arguments for your use case.

```
/**
 * Calls Gemini (or Vertex) and returns the text response.
 *
 * @param prompt  the user prompt
 * @param modelId optional model (defaults to "gemini-2.0-flash-001")
 */
public String generate(String prompt, String modelId) {
   String m = (modelId == null || modelId.isBlank())
         ? "gemini-2.0-flash-001"
         : modelId;
   GenerateContentResponse res = client.models.generateContent(m, prompt, null);
   return res.text();         // convenient helper
}
```

File: ./src/main/java/com/example/gcp/service/GenAiService.java

```
package com.example.gcp.service;
import com.google.genai.Client;
import com.google.genai.types.GenerateContentResponse;
import org.springframework.stereotype.Service;
@Service
public class GenAiService {
   private final Client client;
  public GenAiService() {
      // Default ctor picks up env vars
      this.client = new Client();
   }
  /**
    * Calls Gemini (or Vertex) and returns the text response.
    *
    * @param prompt  the user prompt
    * @param modelId optional model (defaults to "gemini-2.0-flash-001")
    */
   public String generate(String prompt, String modelId) {
      String m = (modelId == null || modelId.isBlank())
            ? "gemini-2.0-flash-001"
            : modelId;
     GenerateContentResponse res = client.models.generateContent(m, prompt, null);
      return res.text();         // convenient helper
```

```
    }
}
```

With this you have completed your service layer and will move to the controller layer to accept requests and return responses. The controller will need a few DTO objects for your request and response types. These are shared for reference here:

File: ./src/main/java/com/example/gcp/dto/GenAiRequest.java

```
package com.example.gcp.dto;
/** Simple request body for /api/genai POST. */
public record GenAiRequest(String prompt, String modelId) { }
```

File: ./src/main/java/com/example/gcp/dto/GenAiResponse.java

```
package com.example.gcp.dto;
/** JSON response wrapper. */
public record GenAiResponse(String text) { }
```

These DTO objects will be referenced in your controller file named GenAiController.java. You will reference the service you created earlier in the constructor to get an instance of it injected and use it to generate a response and send it as response to your user's request. The code for same is shown here:

File: ./src/main/java/com/example/gcp/controller/GenAiController.java

```
package com.example.gcp.controller;
import com.example.gcp.dto.GenAiRequest;
import com.example.gcp.dto.GenAiResponse;
import com.example.gcp.service.GenAiService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/genai")
public class GenAiController {
    private final GenAiService genAiService;
    public GenAiController(GenAiService genAiService) {
        this.genAiService = genAiService;
    }
    @PostMapping(consumes = "application/json", produces = "application/json")
    public ResponseEntity<GenAiResponse> generatePost(@RequestBody GenAiReques
t req) {
```

```
        String answer = genAiService.generate(req.prompt(), req.modelId());
        return ResponseEntity.ok(new GenAiResponse(answer));
    }
}
```

At this point, although you have completed your source code implementation, it won't work since you have a few prerequisites to complete. The first one is to make sure the Vertex AI API is enabled, as shown in Figure 16-66.

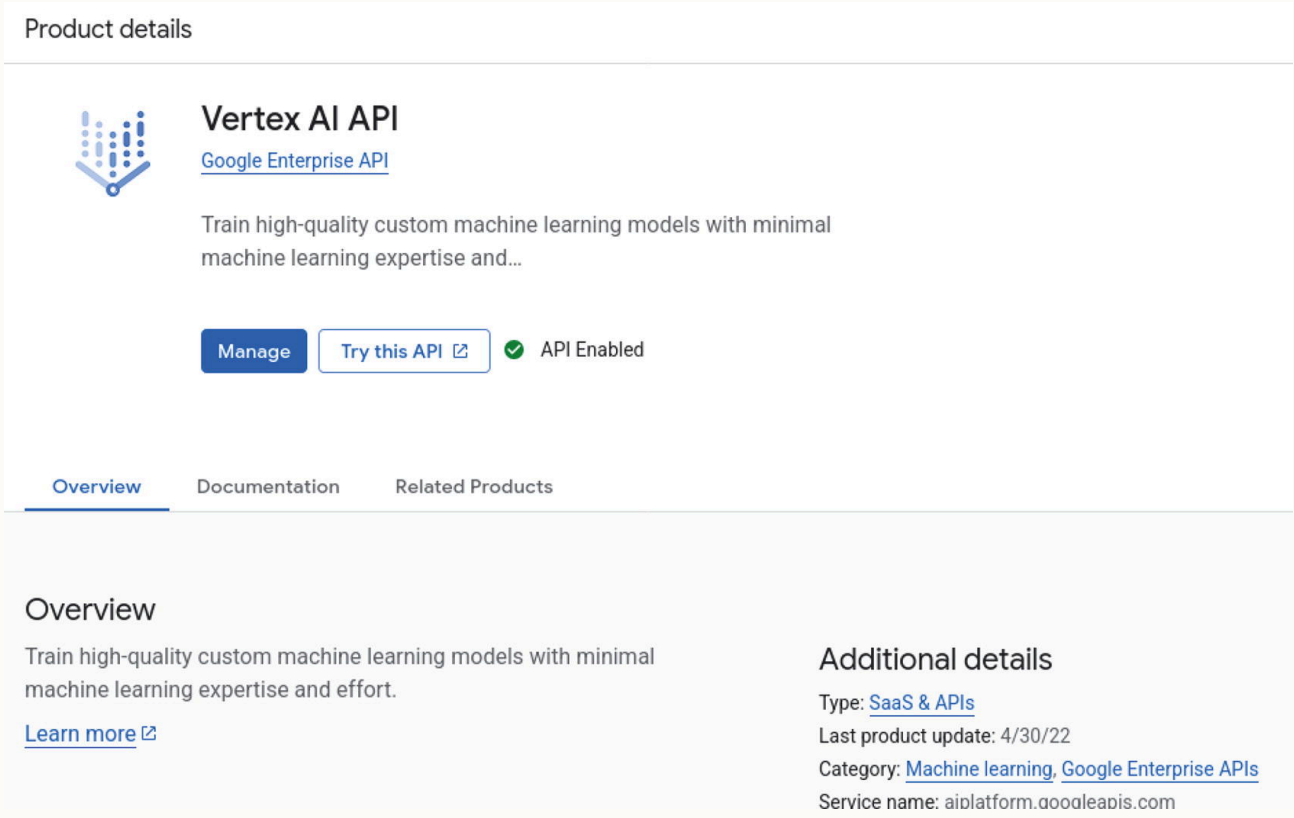


Figure 16-66 Vertex AI API enable page

The Gen AI client needs GEMINI_API_KEY or GOOGLE_API_KEY set as an environment to make calls to Vertex AI. This API can be generated by visiting https://aistudio.google.com/app/apikey and creating an API key by following the steps highlighted in Figure 16-67, Figure 16-68, and Figure 16-69. The process involves clicking the blue "Create API key" button, then selecting your existing project or create if one does not exist, and finally copying the key for your reference.



Figure 16-67 Create API key page in Google AI Studio

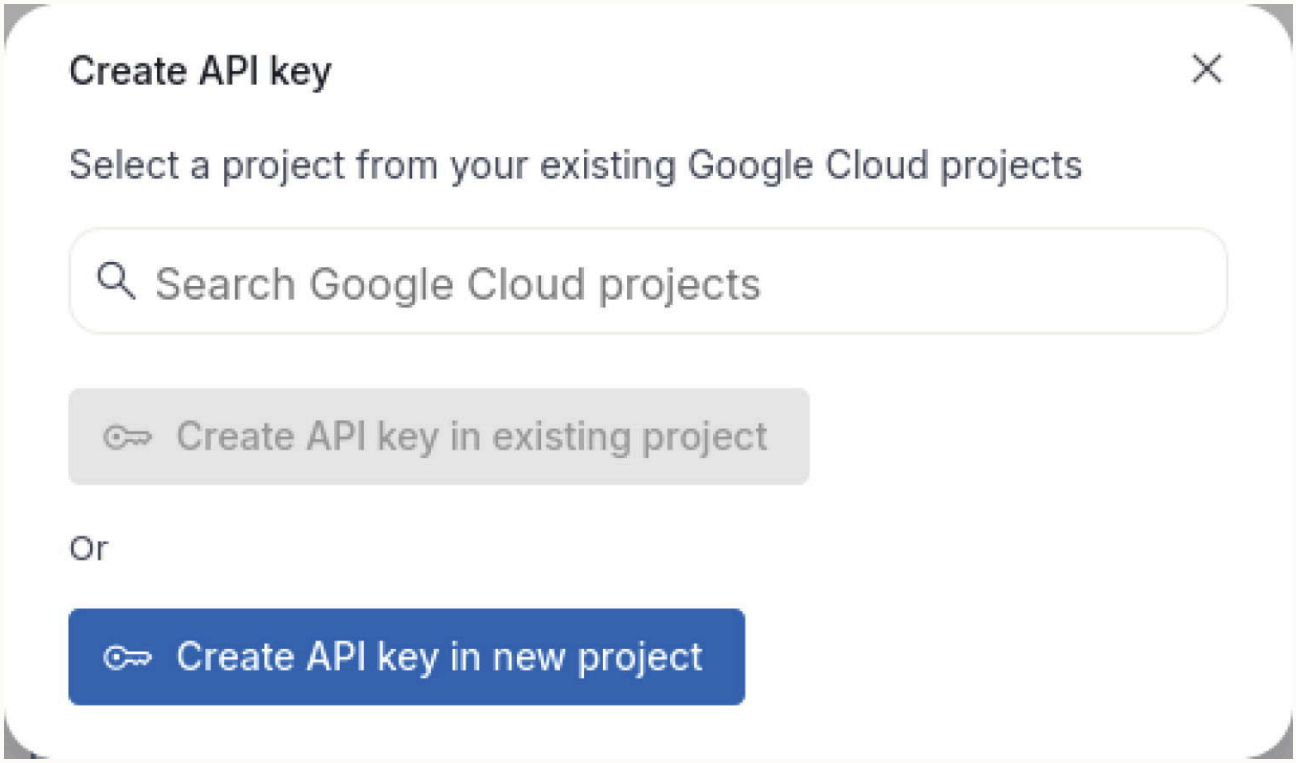


Figure 16-68 Create API key dialog

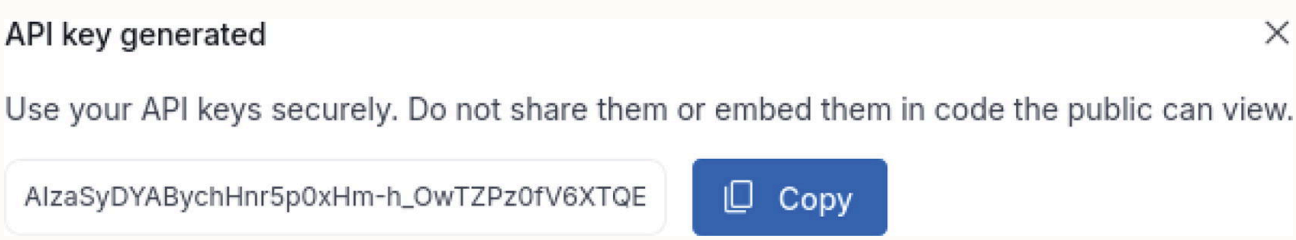


Figure 16-69 Copying generated API key

Make sure to set the environment variable with the name GEMINI_API_KEY or GOOGLE_API_KEY in your environment or the IntelliJ configuration section, as shown in Figure 16-70.

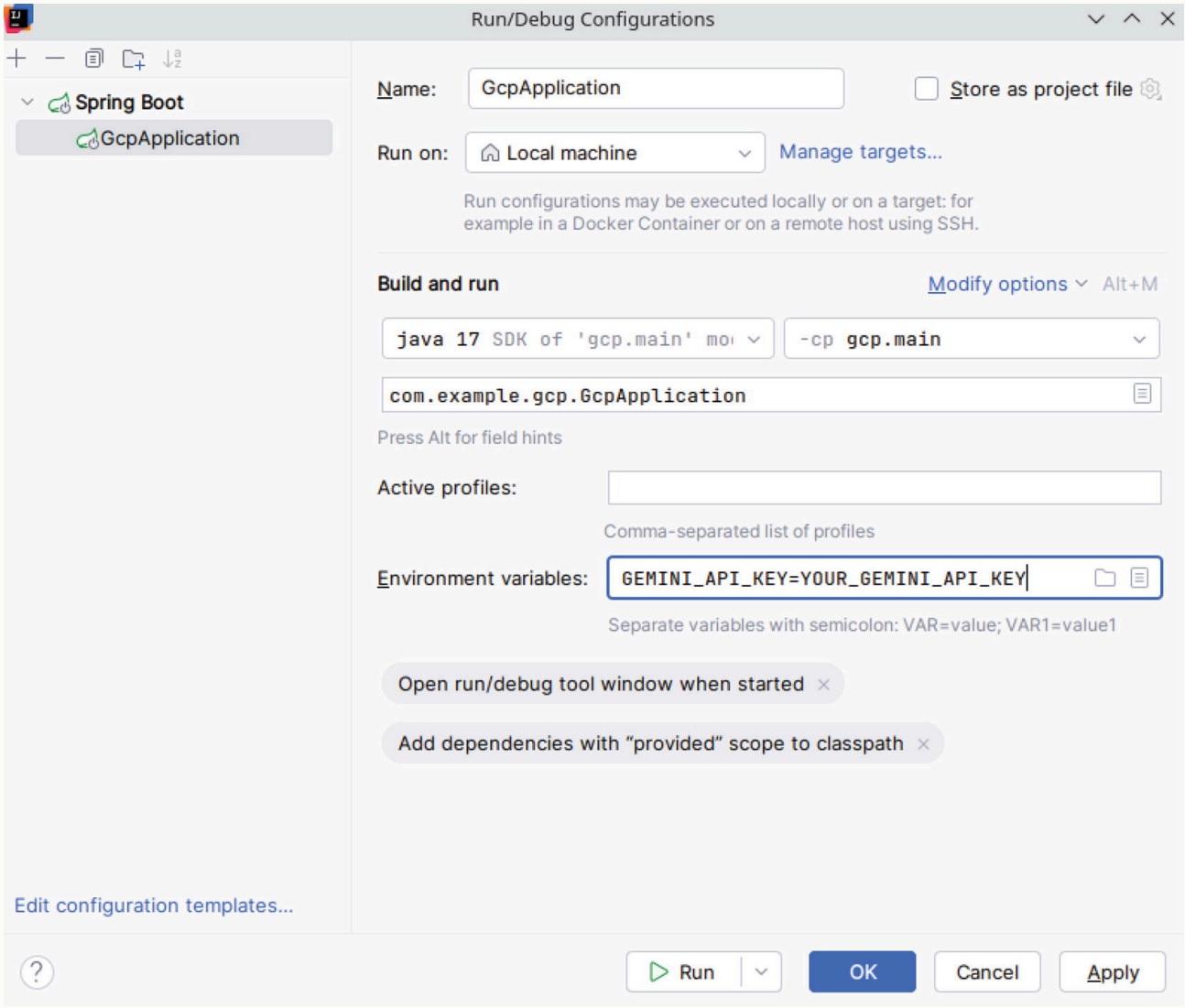


Figure 16-70 Environment configuration in Spring Boot

Now that your prerequisites are complete, let's boot up your application and see it in action by trying the following curl requests.

```
curl -X POST http://localhost:8080/api/genai
  -H "Content-Type: application/json" \
 -d '{"prompt":"Tell me a joke","modelId":"gemini-2.0-flash-001"}'
{"text":"Why don't scientists trust atoms? Because they make up everything!"}
curl -X POST http://localhost:8080/api/genai \
  -H "Content-Type: application/json" \
 -d '{"prompt":"Tell me a joke"}'
{"text":"Why don't scientists trust atoms? Because they make up everything!"}
```

In this example, you explored the different tools in the GCP Vertex AI service relevant to Gen AI and used the Java SDK of Gen AI to build a small application to interact with LLMs.

Generative AI in Azure

Azure AI has many services relevant to machine learning, some of which are shown in Figure 16-71. The service you will be exploring in this section is Azure OpenAI for Gen AI.

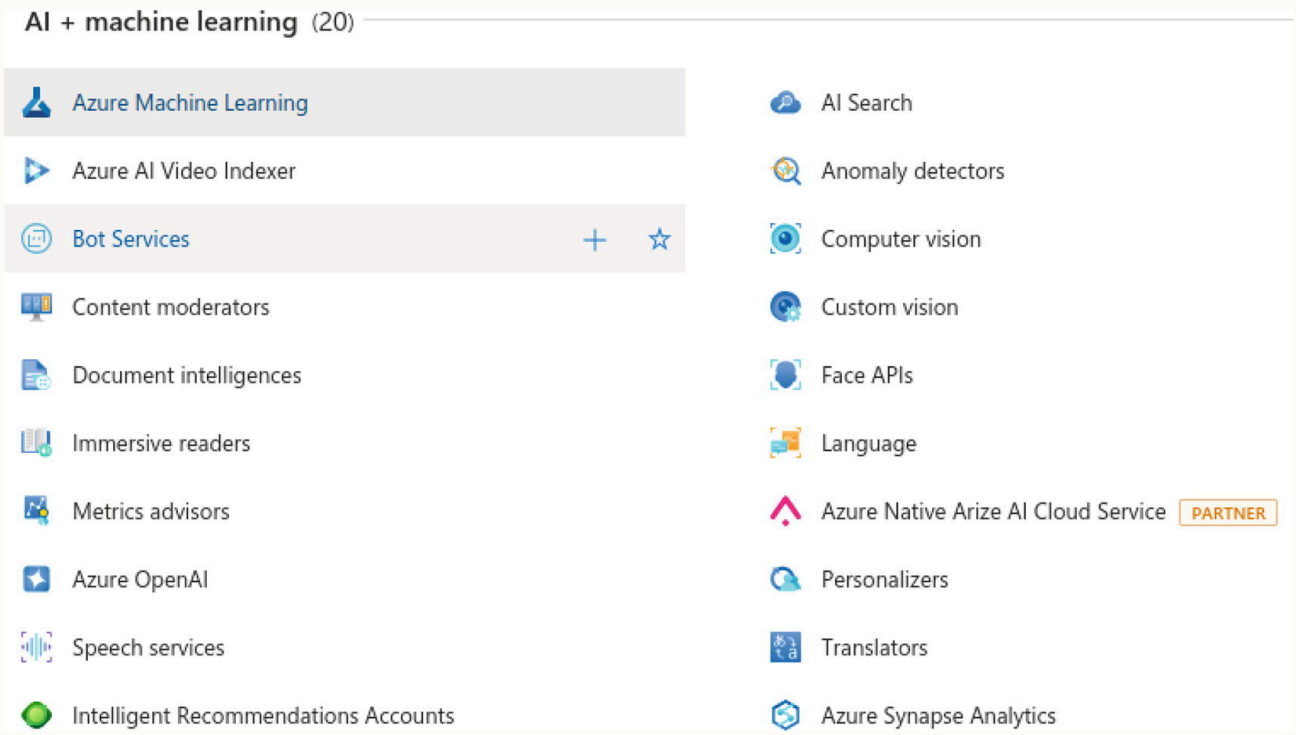


Figure 16-71 Azure machine learning services

AI Foundry is the umbrella tool under which Azure provides different Gen AI services, as shown in Figure 16-72.

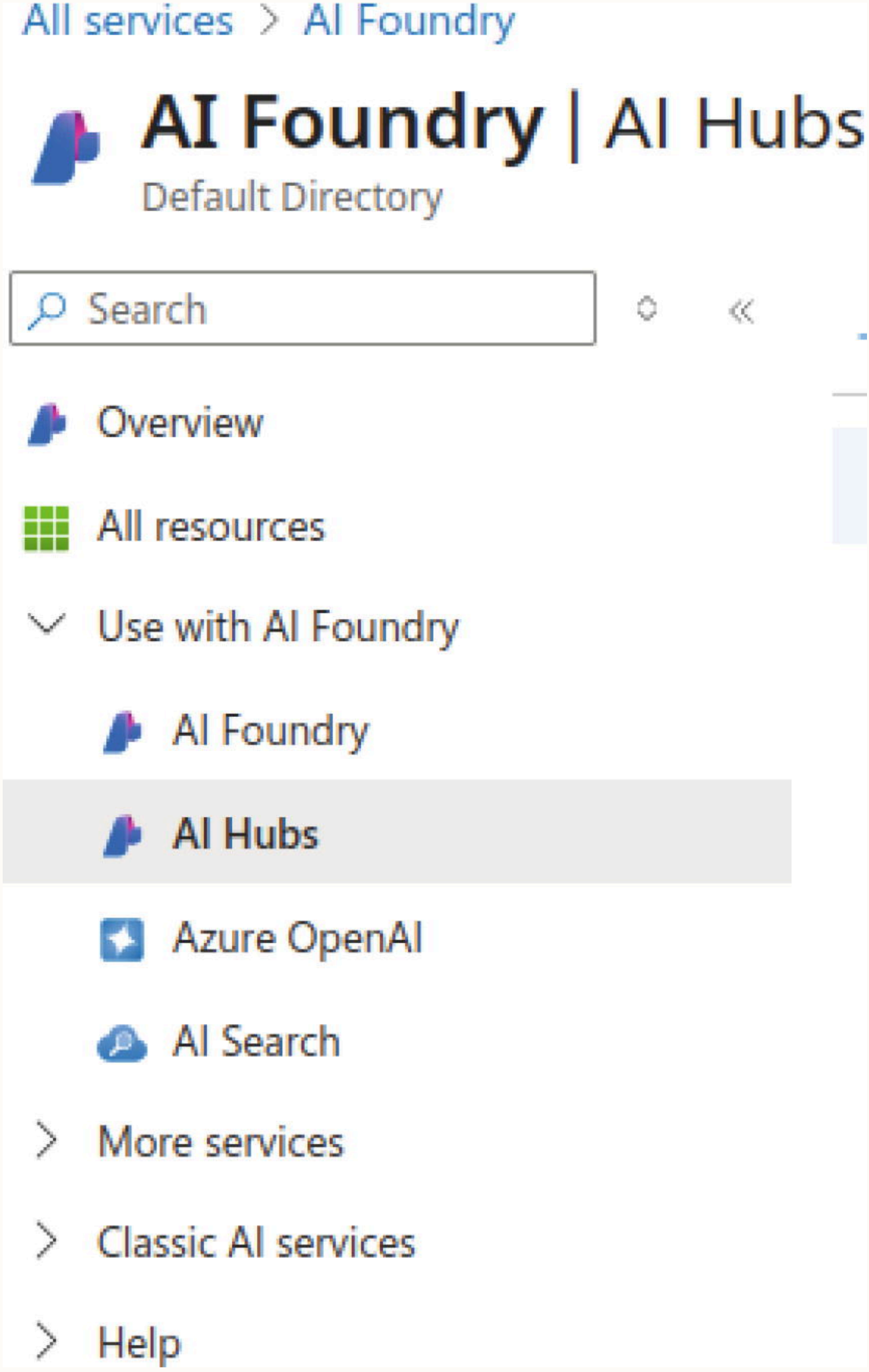


Figure 16-72 AI Foundry Services for Gen AI

Let's start by exploring the Azure OpenAI service. Click this service, you will you see a starter page asking you to create an Azure OpenAI service, as shown in Figure 16-73.

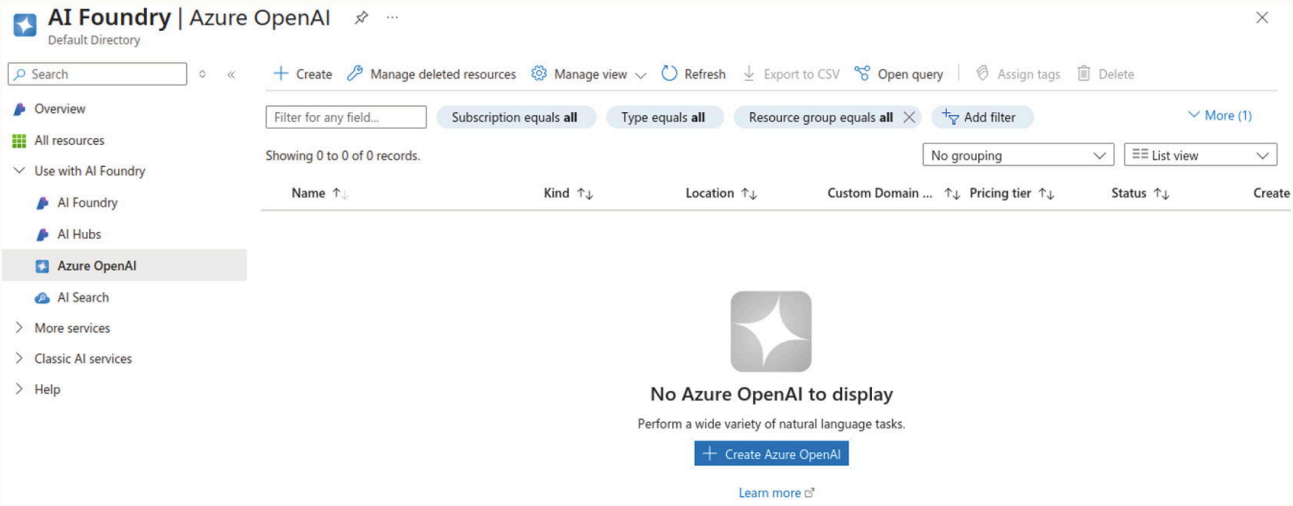


Figure 16-73 Azure OpenAI service creation page

Then select your subscription and resource group and fill in the region, name, and pricing tier details, as shown in Figure 16-74.

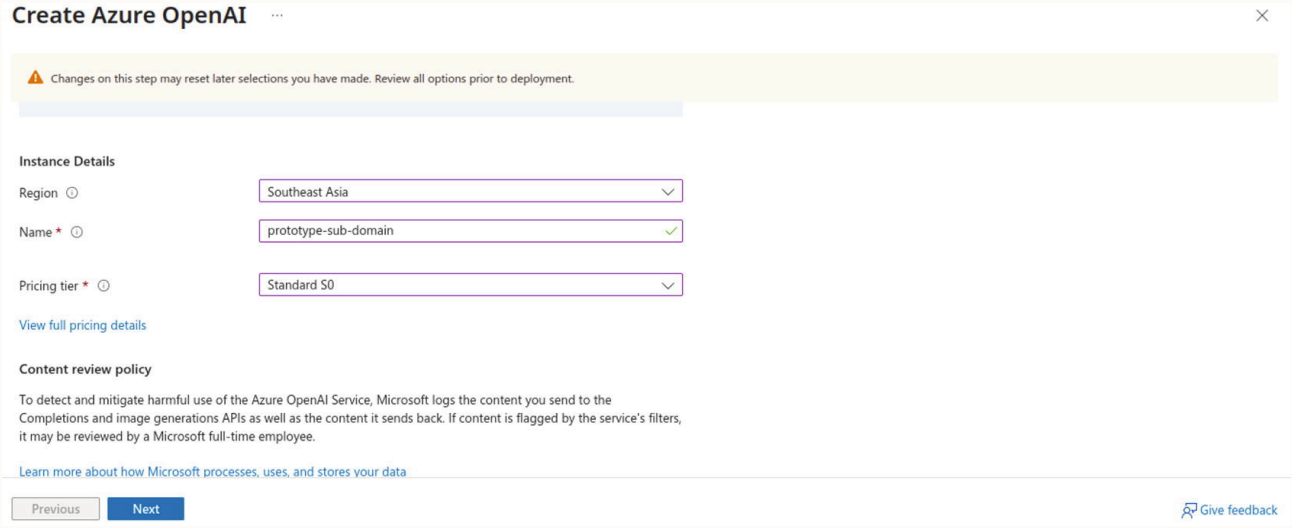


Figure 16-74 Region, name, pricing tier configuration for Azure OpenAI

For the next two action requesting pages, select the default selections and proceed to the Review + submit page to review all the choices made and submit them for the resource creation. Then you will wait for a couple of minutes for the deployment to complete. Once the deployment is complete, visit the Azure OpenAI page in Azure AI Foundry, as shown in Figure 16-75.

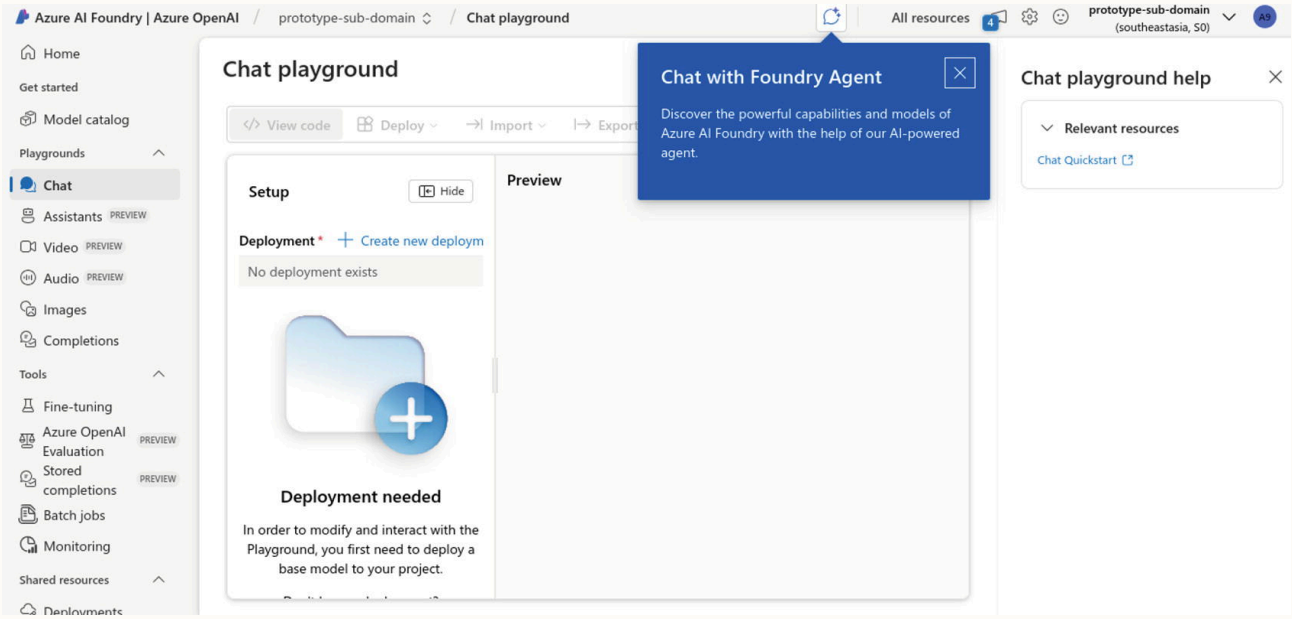


Figure 16-75 Azure OpenAI Chat Playground

As you can see, different tools are provided for Gen AI in this playground. The Playgrounds section allows you to play with chat, assistants, and multimodal prompts. In the Tools section, you can see offerings for fine-tuning, evaluation, stored completions, batch jobs, and monitoring, which are important for the Gen AI ecosystem. You will see this area getting more and more focus in the future. For the moment, you will explore the chat service and then move on to write a sample application using the Azure SDK.

To start with chat playground, you will need to create a deployment first (see Figure 16-76).

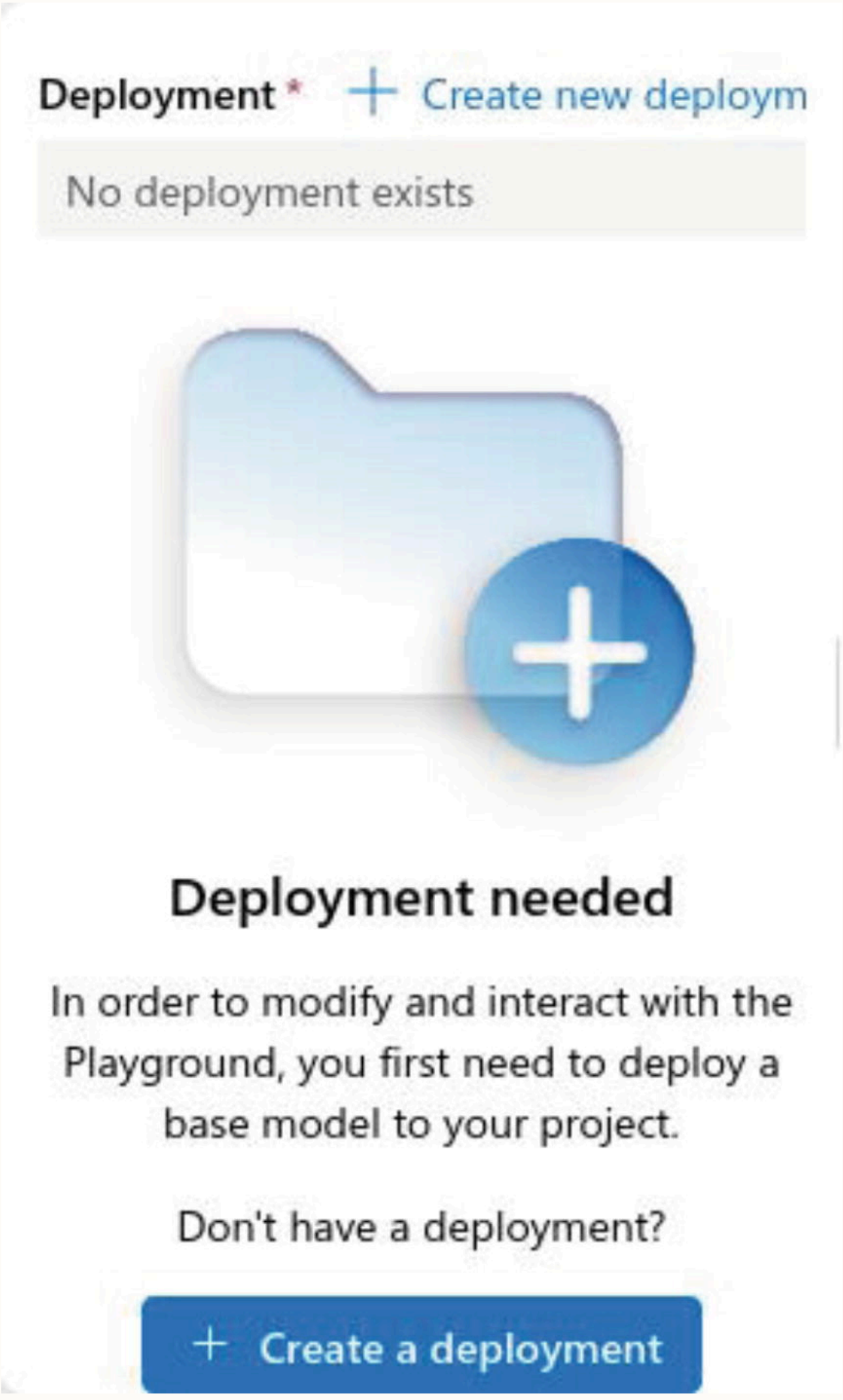


Figure 16-76 Base model deployment creation section

The first step is to select a model from the list provided. You can use gpt-4.1-mini to practice. On selecting the model, it shows the details specific to the model, which is very helpful (Figure 16-77).

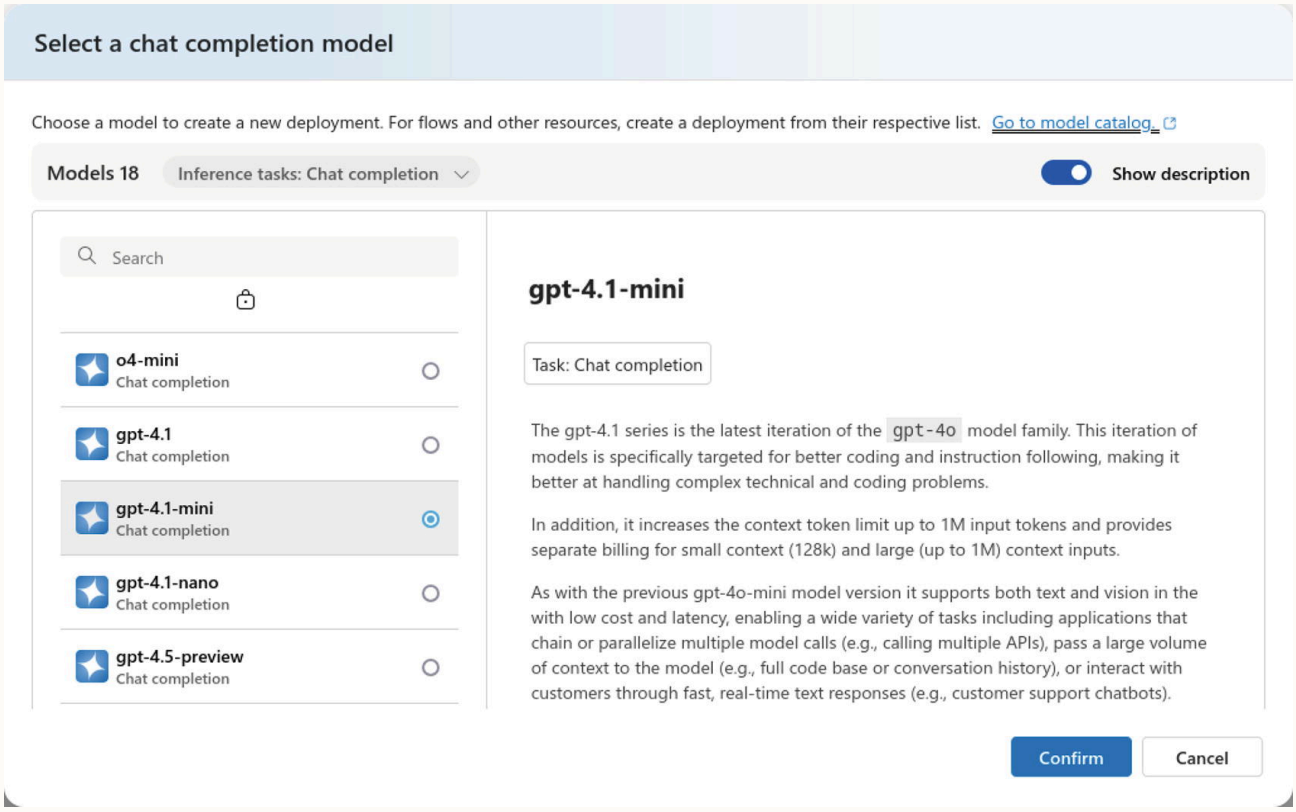


Figure 16-77 Base model selection section

Once you confirm and proceed, it will show you the deployment details, as shown in Figure 16-78. Review the details and then click the "Create resource and deploy" button.

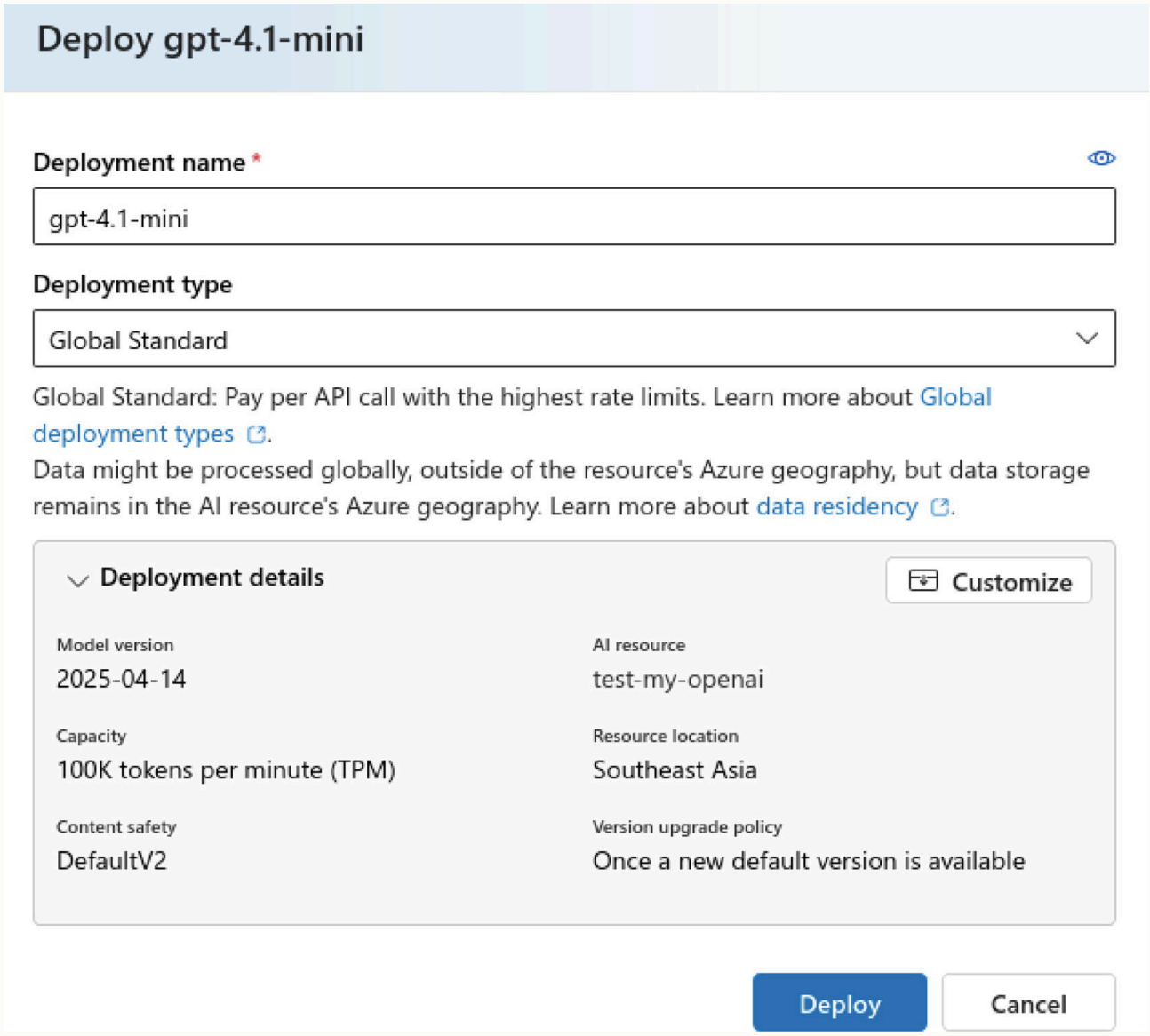


Figure 16-78 Deployment name and type selection section

The deployment is quick, and then you can play with the LLM model in the playground. Let's ask for a joke, as shown in Figure 16-79.

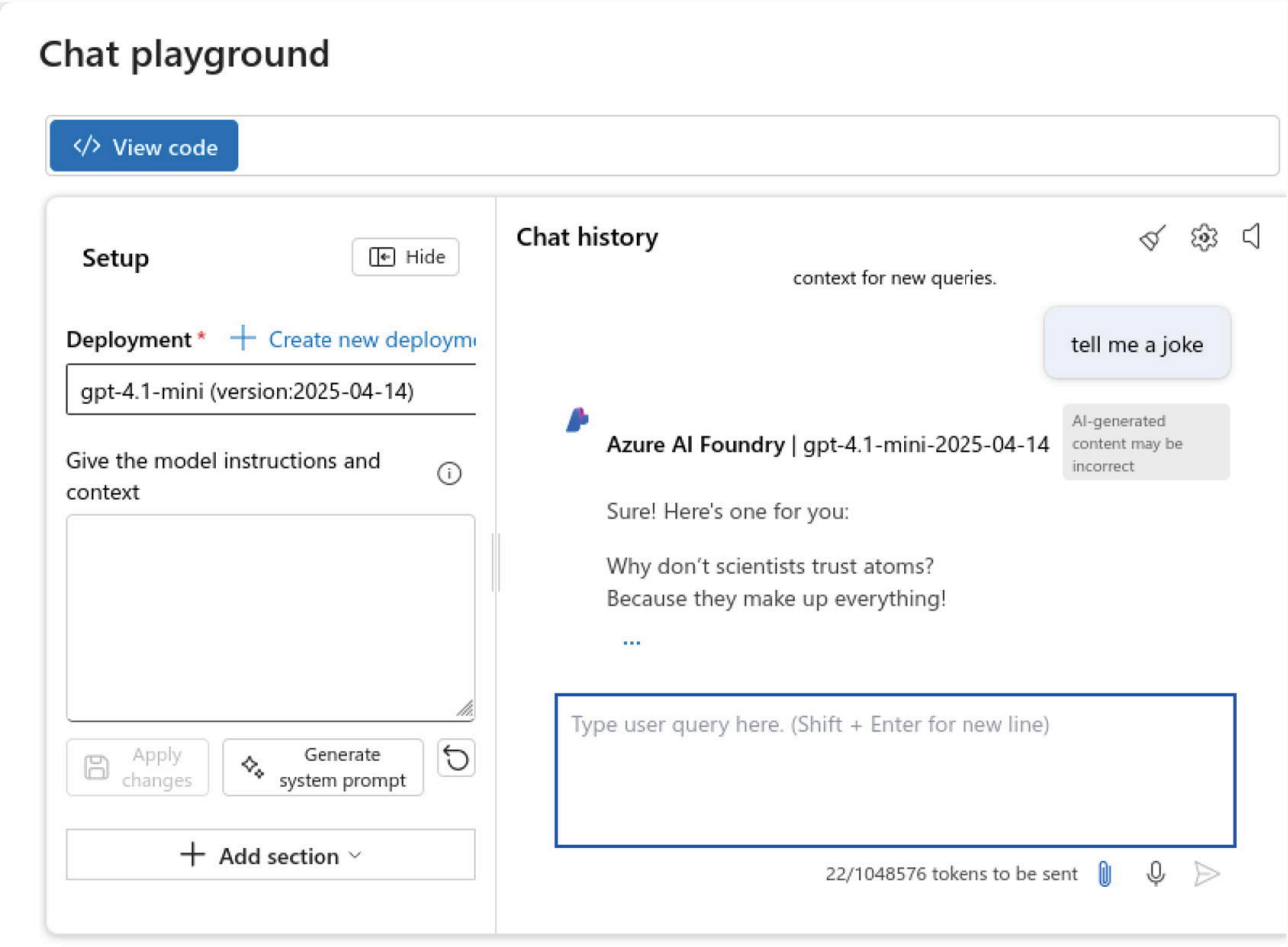


Figure 16-79 Chat playground

There is a handy blue "View code" button that when clicked shows the code responsible for generating this output. It is similar to GCP but with the Java language as an option, which was missing from GCP (see Figure 16-80).

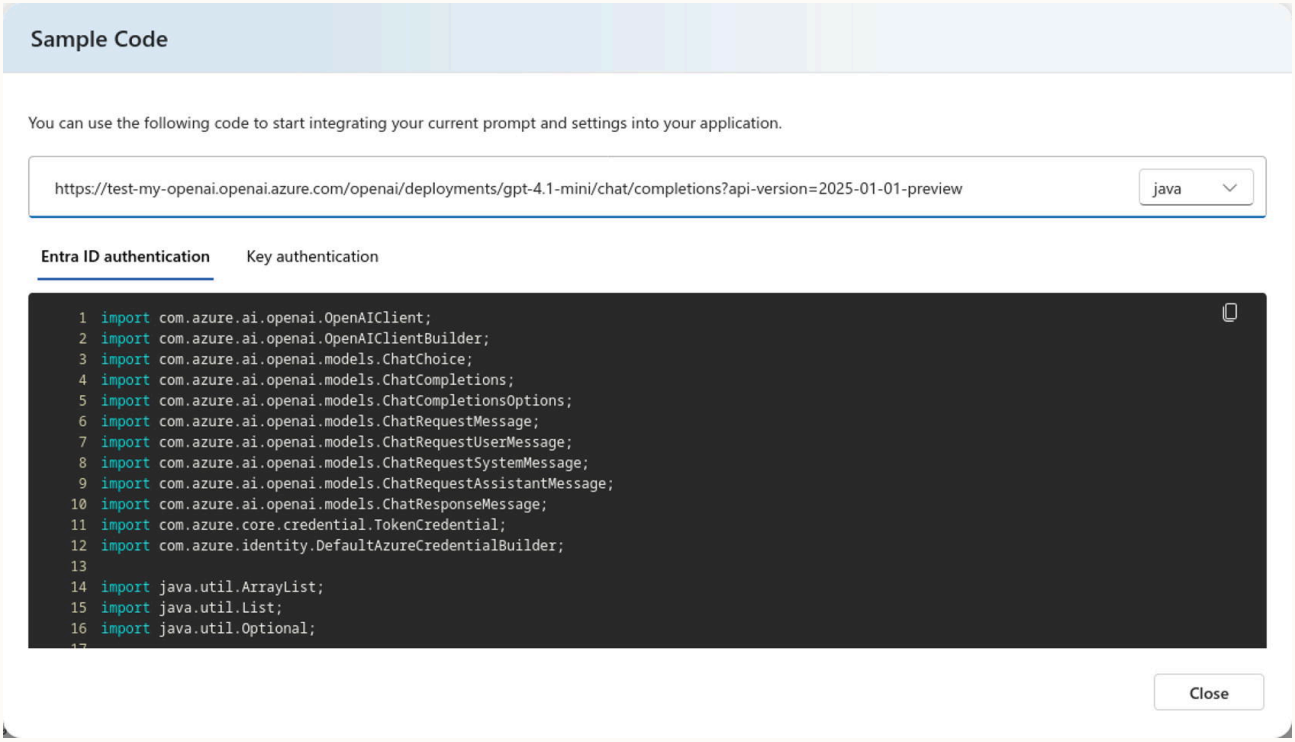


Figure 16-80 Sample code generation section in different languaages

Now let's explore how to build a Spring Boot application using the sample code from the Java example. Let's start with the same setup as for GCP by starting from Spring Initializr and unzipping it at a location of your choice. The components for your application would be similar to the one you built earlier. The only difference now is you're using Azure OpenAI instead of Vertex AI, as shown in Figure 16-81.

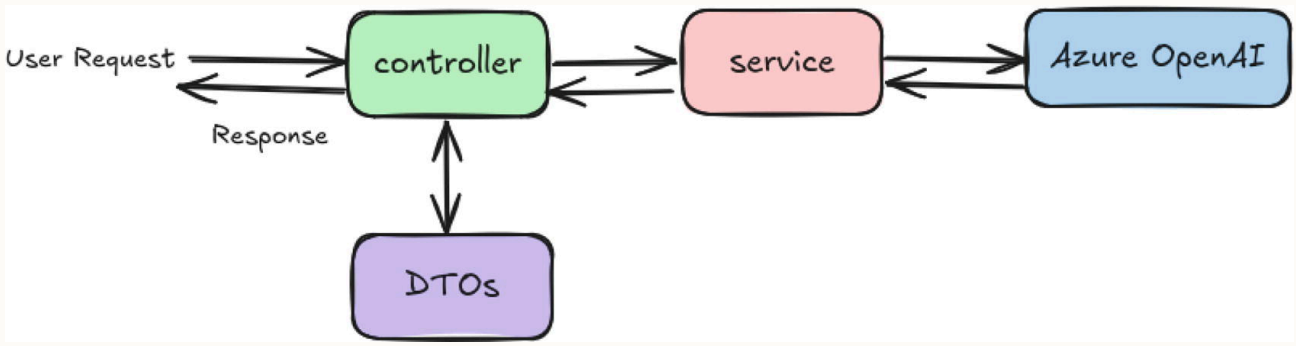


Figure 16-81 Application architecture and control flow

Most of the code from the previous setup is reusable, so it will not be repeated here. This section will focus on the service component since it now interacts with Azure OpenAI and any build.gradle file-specific changes along with any environment variable needed to interact with Azure OpenAI.

Let's start by updating your build.gradle file to include a dependency for Azure OpenAI SDK. (Note that at this moment of writing the SDK is in beta stage.)

```
// https://mvnrepository.com/artifact/com.azure/azure-ai-openai
implementation("com.azure:azure-ai-openai:1.0.0-beta.16")
```

File: ./build.gradle

```
plugins {
  id 'java'
  id 'org.springframework.boot' version '3.5.4'
  id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
  toolchain {
    languageVersion = JavaLanguageVersion.of(17)
  }
}
repositories {
  mavenCentral()
}
```

```
dependencies {
  implementation 'org.springframework.boot:spring-boot-starter-web'
  implementation 'com.azure:azure-ai-openai:1.0.0-beta.16'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
  testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') {
  useJUnitPlatform()
}
```

Now you will move on to updating your service component in the GenAiService.java file. You will first create two attributes to hold the client and deployment instance entities, as shown here:

```
private final OpenAIClient client;
private final String defaultDeployment;
```

Then inside the public constructor, you will instantiate these as shown next. Many of the APIs are similar to the OpenAI ones you used earlier, so they should be self-explanatory. You can use the OpenAIClientBuilder API to build your client instance, which will be used to make API calls to Azure OpenAI.

```
public GenAiService(
    @Value("${azure.openai.endpoint}")   String endpoint,
    @Value("${azure.openai.api-key}")    String apiKey,
    @Value("${azure.openai.deployment}") String defaultDeployment) {
  this.defaultDeployment = defaultDeployment;
  this.client = new OpenAIClientBuilder()
        .endpoint(endpoint)
        .credential(new AzureKeyCredential(apiKey))
        .buildClient();
}
```

Finally, update the generate method to generate the ChatRequestUserMessage using the prompt sent from the user and generate the response using the getChatCompletions API, as shown here:

```
public String generate(String prompt, String modelId) {
  String deployment = (modelId == null || modelId.isBlank())
        ? defaultDeployment
        : modelId;
  List<ChatRequestMessage> messages = List.of(new ChatRequestUserMessage(prom
pt));
```

```
  ChatCompletionsOptions opts = new ChatCompletionsOptions(messages)
        .setMaxTokens(1_024)   // pick a sensible number; adjust to your policy/quota
        .setTemperature(0.7)
        .setTopP(0.95);
  ChatCompletions completions = client.getChatCompletions(deployment, opts);
  /* In production, you should add defensive checks for:
   *   - completions == null
   *   - completions.getChoices().isEmpty()
   *   - multiple choices (pick best by log-prob, etc.)
   */
  return completions.getChoices()
        .get(0)
        .getMessage()
        .getContent();
}
```

File: ./src/main/java/com/example/azureopenai/service/GenAiService.java

```
package com.example.azureopenai.service;
import com.azure.ai.openai.OpenAIClient;
import com.azure.ai.openai.OpenAIClientBuilder;
import com.azure.ai.openai.models.ChatCompletions;
import com.azure.ai.openai.models.ChatCompletionsOptions;
import com.azure.ai.openai.models.ChatRequestMessage;
import com.azure.ai.openai.models.ChatRequestUserMessage;
import com.azure.core.credential.AzureKeyCredential;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class GenAiService {
  private final OpenAIClient client;
  private final String defaultDeployment;
  public GenAiService(
        @Value("${azure.openai.endpoint}")   String endpoint,
        @Value("${azure.openai.api-key}")    String apiKey,
        @Value("${azure.openai.deployment}") String defaultDeployment) {
    this.defaultDeployment = defaultDeployment;
    this.client = new OpenAIClientBuilder()
          .endpoint(endpoint)
          .credential(new AzureKeyCredential(apiKey))
          .buildClient();
  }
  public String generate(String prompt, String modelId) {
    String deployment = (modelId == null || modelId.isBlank())
```

```
            ? defaultDeployment
            : modelId;
        List<ChatRequestMessage> messages = List.of(new ChatRequestUserMessage(pro
mpt));
        ChatCompletionsOptions opts = new ChatCompletionsOptions(messages)
                .setMaxTokens(1_024)  // pick a sensible number; adjust to your policy/quota
                .setTemperature(0.7)
                .setTopP(0.95);
        ChatCompletions completions = client.getChatCompletions(deployment, opts);
        /* In production, you should add defensive checks for:
         *  - completions == null
         *  - completions.getChoices().isEmpty()
         *  - multiple choices (pick best by log-prob, etc.)
         */
        return completions.getChoices()
                .get(0)
                .getMessage()
                .getContent();
    }
}
```

Before booting up your application, you will need a few resource attributes in your application.properties file. All these three attributes can be easily found in the shared resources menu in the sidebar at the bottom left. Click Deployments and from the list click the deployment that you want to use from the ones created earlier (see Figure 16-82).

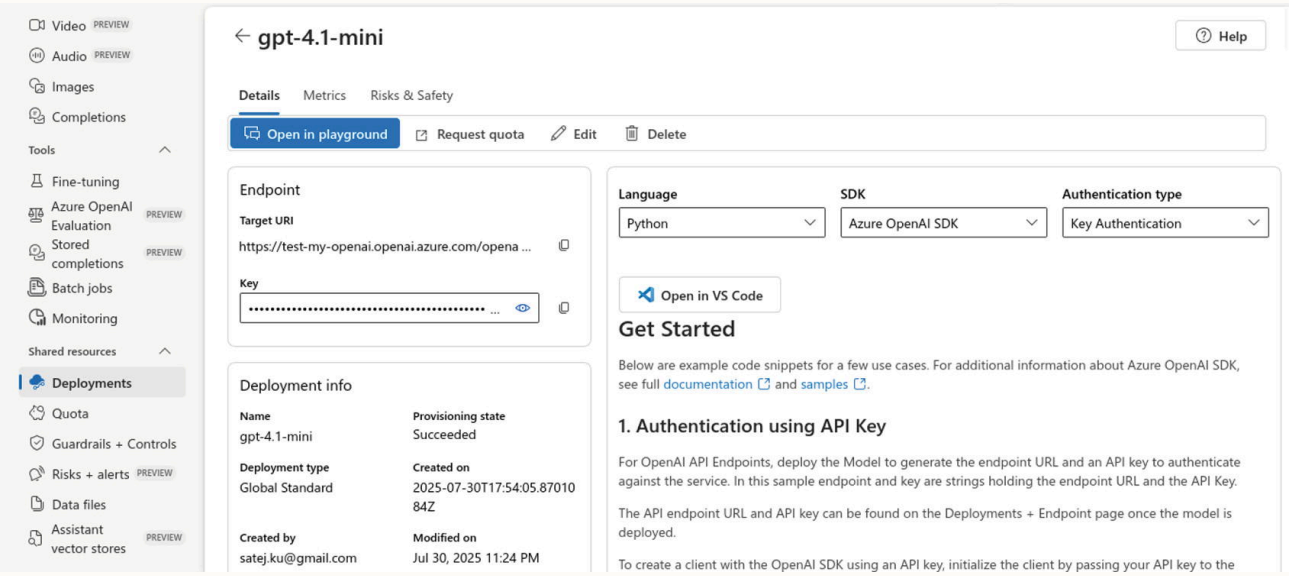


Figure 16-82 Deployment-specific configuration information page

The Get Started guide is very helpful. In the Endpoint section in the center screen, you can find information for the target URI, which is your endpoint; the key, which is your

API key; and finally the name in the Deployment info section, which is your deployment value.

File: ./src/main/resources/application.properties

```
spring.application.name=azure-openai
azure.openai.endpoint=https://test-my-openai.openai.azure.com/
azure.openai.api-key=YOUR-API-KEY
azure.openai.deployment=gpt-4.1-mini
```

Now that you have completed all the setup, let's boot up your application and test it.

```
curl -X POST http://localhost:8080/api/genai    -H "Content-Type: application/jso
n"    -d '{"prompt":"Tell me a joke"}'
{"text":"Sure! Here's a joke for you:\n\nWhy don't scientists trust atoms?  \nBecause t
hey make up everything!"}
```

In this section, you were able to learn about the different services of Azure OpenAI and also build a sample application using the Azure OpenAI SDK for Java.

Conclusion

In this chapter, you learned how AWS Amazon Bedrock enables Java developers to harness Gen AI within the familiar AWS ecosystem. The chapter began by exploring the broader landscape of Gen AI services across the cloud and then focused on Bedrock's foundational models, playground experimentation, and SDK-driven application development. Building on that foundation, the chapter introduced guardrails to ensure safety and compliance, extended the solution with retrieval-augmented generation using Bedrock knowledge bases, and finally showcased how Spring AI integration can simplify and standardize generative AI in Java.

By understanding these core components—model selection, prompt tuning, safety frameworks, RAG, and framework integration, you're well equipped to architect robust, scalable, and responsible Gen AI applications on AWS. As Gen AI continues to evolve, the patterns and practices covered here will help you adapt and extend your solutions, whether in aerospace certification, customer support, content creation, or any domain where AI-driven intelligence can unlock new possibilities. In addition, I implore you to try other services such as fine-tuning monitoring with focus within the ecosystem surrounding LLMs.

The next chapter covers LLM Communication protocols.

Previous chapter

< 15. Does It Even Work? Testing and Evaluating LLM Apps

Next chapter

17. Talking in Protocols: The MCP Revolution >